

```
#
=====
=====
# TELEGRAM QUIZ BOT
# PART 1/4
# Config + Database + Question Management
#
=====
=====

import os
import re
import json
import time
import uuid
import html
import hashlib
import logging
import sqlite3
import threading
import asyncio
from pathlib import Path
from datetime import datetime, timezone,
timedelta

import requests
from bs4 import BeautifulSoup
from flask import Flask

from groq import Groq
```

```

from telegram import (
    Update,
    Poll,
    InlineKeyboardButton,
    InlineKeyboardMarkup,
)
from telegram.ext import (
    Application,
    CommandHandler,
    MessageHandler,
    CallbackQueryHandler,
    PollAnswerHandler,
    ContextTypes,
    TypeHandler,
    filters,
)

#
=====
=====
# CONFIG
#
=====
=====

BOT_TOKEN = os.getenv("BOT_TOKEN",
    "").strip()
GROQ_API_KEY = os.getenv("GROQ_API_KEY",
    "").strip()
SERPAPI_API_KEY =
os.getenv("SERPAPI_API_KEY", "").strip()

```

```
CHANNEL_ID = os.getenv("CHANNEL_ID",
    "").strip()

ADMIN_IDS = {
    int(x.strip())
    for x in os.getenv("ADMIN_IDS",
    "").split(",")
    if x.strip().lstrip("-").isdigit()
}

PORT = int(os.getenv("PORT", "8000"))

DB_PATH = os.getenv(
    "DB_PATH",
    "quizbot.db"
)

DATA_DIR = Path(
    os.getenv(
        "DATA_DIR",
        "data"
    )
)

PDF_DIR = DATA_DIR / "pdfs"

DATA_DIR.mkdir(
    parents=True,
    exist_ok=True
)

PDF_DIR.mkdir(
    parents=True,
    exist_ok=True
)
```

```

)

GROQ_MODEL = os.getenv(
    "GROQ_MODEL",
    "openai/gpt-oss-120b"
)

# Scanned/image PDF OCR model. Override
with GROQ_VISION_MODEL if needed.
GROQ_VISION_MODEL = os.getenv(
    "GROQ_VISION_MODEL",
    "qwen/qwen3.6-27b"
)

# SerpAPI web-search settings, used by
# /autoquiz to ground AI question
# generation in fresh web results instead
of relying only on model memory.
SERPAPI_RESULTS_COUNT =
int(os.getenv("SERPAPI_RESULTS_COUNT",
"8"))
SERPAPI_FETCH_PAGES =
int(os.getenv("SERPAPI_FETCH_PAGES", "3"))
SERPAPI_CONTEXT_CHAR_LIMIT =
int(os.getenv("SERPAPI_CONTEXT_CHAR_LIMIT",
"20000"))

# Wikipedia settings, used by /wikiquiz. No
API key needed – Wikipedia's
# REST/MediaWiki API is free and open.
WIKIPEDIA_LANG =
os.getenv("WIKIPEDIA_LANG", "hi").strip()
or "hi"
WIKIPEDIA_RESULTS_COUNT =

```

```
int(os.getenv("WIKIPEDIA_RESULTS_COUNT",
"3"))
WIKIPEDIA_CONTEXT_CHAR_LIMIT =
int(os.getenv("WIKIPEDIA_CONTEXT_CHAR_LIMIT
", "20000"))

# Google Cloud Text-to-Speech settings,
used by /speak to turn text into
# a Telegram voice note.
GOOGLE_TTS_API_KEY =
os.getenv("GOOGLE_TTS_API_KEY", "").strip()
GOOGLE_TTS_LANGUAGE_CODE =
os.getenv("GOOGLE_TTS_LANGUAGE_CODE", "hi-
IN").strip() or "hi-IN"
GOOGLE_TTS_VOICE_NAME =
os.getenv("GOOGLE_TTS_VOICE_NAME",
 "").strip()
GOOGLE_TTS_CHAR_LIMIT =
int(os.getenv("GOOGLE_TTS_CHAR_LIMIT",
"800"))

# Exam/subject saved-link library settings,
used by /examquiz and
# /subjectquiz to build a quiz from user-
curated website links.
TOPIC_LINKS_MAX_PER_QUIZ =
int(os.getenv("TOPIC_LINKS_MAX_PER_QUIZ",
"8"))
TOPIC_LINKS_CHARS_PER_PAGE =
int(os.getenv("TOPIC_LINKS_CHARS_PER_PAGE",
"4000"))
TOPIC_LINKS_MAX_SAVED =
int(os.getenv("TOPIC_LINKS_MAX_SAVED",
"30"))
```

```
# Automatic document-to-MCQ settings. No
question count is requested from the user.
AUTO_IMPORT_CHUNK_CHARS =
int(os.getenv("AUTO_IMPORT_CHUNK_CHARS",
"12000"))
AUTO_IMPORT_QUESTIONS_PER_CHUNK =
int(os.getenv("AUTO_IMPORT_QUESTIONS_PER_CHUNK", "12"))
AUTO_IMPORT_MAX_QUESTIONS =
int(os.getenv("AUTO_IMPORT_MAX_QUESTIONS",
os.getenv("MAX_IMPORT_QUESTIONS", "500")))
# Lightweight reliability controls. These
do not increase normal DB/query cost.
GROQ_MAX_RETRIES =
int(os.getenv("GROQ_MAX_RETRIES", "2"))
GROQ_RETRY_BASE_SECONDS =
float(os.getenv("GROQ_RETRY_BASE_SECONDS",
"1.5"))

REQUEST_TIMEOUT = int(
    os.getenv(
        "REQUEST_TIMEOUT",
        "30"
    )
)

DEFAULT_QUIZ_COUNT = int(
    os.getenv(
        "DEFAULT_QUIZ_COUNT",
        "10"
    )
)
```

```
DEFAULT_NEGATIVE_MARK = float(
    os.getenv(
        "NEGATIVE_MARK",
        "0.25"
    )
)

MAX_IMPORT_QUESTIONS = int(
    os.getenv(
        "MAX_IMPORT_QUESTIONS",
        "500"
    )
)

DEFAULT_QUIZ_TIME_SECONDS = int(
    os.getenv(
        "QUIZ_TIME_SECONDS",
        "30"
    )
)

MIN_QUIZ_TIME_SECONDS = 5

MAX_QUIZ_TIME_SECONDS = 600

#
=====
=====
# LOGGING
#
=====
=====
```

```

logging.basicConfig(
    level=logging.INFO,
    format=(
        "%(asctime)s | "
        "%(levelname)s | "
        "%(name)s | "
        "%(message)s"
    )
)

logging.getLogger("httpx").setLevel(logging
.WARNING)

logger = logging.getLogger(
    "quizbot"
)


#
=====
=====
# GLOBAL STATE
#
=====
=====

DB_LOCK = threading.RLock()

ACTIVE_QUIZZES = {}

ACTIVE_POLLS = {}

PENDING = {}

```



```

QUIZ_CREATE = {}

CLONE_QUEUE = []

CLONE_LOCK = threading.Lock()


#
=====
=====
# FLASK HEALTH SERVER
#
=====
=====

health_app = Flask(__name__)


@health_app.route("/")
def health():
    return "Quiz Bot is running", 200


@health_app.route("/health")
def health_check():
    return {
        "status": "ok",
        "service": "telegram-quiz-bot"
    }, 200


def run_health_server():
    try:

```

```
        health_app.run(
            host="0.0.0.0",
            port=PORT,
            debug=False,
            use_reloader=False
        )
    except Exception as e:
        logger.exception(
            "Health server error: %s",
            e
        )
```

```
#
```

```
=====
```

```
=====
```

```
# INTERNET SOURCES
```

```
#
```

```
=====
```

```
=====
```

```
DEFAULT_SOURCES = [
```

```
    (
```

```
        "rajras",
```

```
        "https://www.rajras.in/",
```

```
    ),
```

```
    (
```

```
        "rbse",
```

```
        "https://rajeduboard.rajasthan.gov.in/",
```

```
    ),
```

```

        (
            "samyak_rbse",
            "https://www.samyakias.com/",
        ),

        (
            "ncert",
            "https://ncert.nic.in/",
        ),

        (
            "online2study",
            "https://www.online2study.in/",
        ),
    ]

#
=====
=====
# DEFAULT EXAM/SUBJECT LINK LIBRARY
# (verified free, static,
# no-login MCQ sources – seeded
# automatically on startup so
# /subjectquiz and /examquiz work without
# running /addsubjectlink
# or /addexamlink first). Each tuple:
# (kind, topic_name, url).
#
=====
=====

DEFAULT_TOPIC_SOURCES = [

```

```
(
    "subject",
    "GK",
    "https://www.edudose.com/gk/",
),

(
    "subject",
    "GK",
    "https://www.indiabix.com/general-
knowledge/questions-and-answers/",
),

(
    "subject",
    "Current Affairs",
    "https://dailygk.indgovtjobs.net/current-
affairs",
),

(
    "subject",
    "GK",
    "https://www.freeonlinetest.in/questions-
and-answers/general-knowledge",
),

(
    "subject",
    "Indian Polity",
    "https://www.examveda.com/general-
knowledge/practice-mcq-question-on-indian-
```

```

politics/",
    ),
]

#
=====
=====
# DATABASE CONNECTION
#
=====
=====

def db():
    conn = sqlite3.connect(
        DB_PATH,
        timeout=30,
        check_same_thread=False
    )

    conn.row_factory = sqlite3.Row

    conn.execute(
        "PRAGMA journal_mode=WAL"
    )

    conn.execute(
        "PRAGMA foreign_keys=ON"
    )

    return conn

#

```

```

=====
=====
# SAFE COLUMN MIGRATION HELPER
#
=====
=====

def _add_column_if_missing(
    conn,
    table,
    column,
    coldef
):

    existing = {
        row["name"]
        for row in conn.execute(
            f"PRAGMA table_info({table})"
        ).fetchall()
    }

    if column in existing:
        return

    try:
        conn.execute(
            f"ALTER TABLE {table} "
            f"ADD COLUMN {column} {coldef}"
        )
    except Exception:
        logger.exception(
            "Column migration failed:
%s.%s",
            table,

```

```

        column
    )

#
=====
=====
# DATABASE INITIALIZATION
#
=====
=====

def init_db():

    with DB_LOCK:

        conn = db()

        conn.executescript(
            """

            CREATE TABLE IF NOT EXISTS
users (

                id INTEGER PRIMARY KEY,

                username TEXT,

                first_name TEXT,

                last_name TEXT,

                questions_answered INTEGER
                DEFAULT 0,

```

```
        correct_answers INTEGER
        DEFAULT 0,

        wrong_answers INTEGER
        DEFAULT 0,

        skipped_questions INTEGER
        DEFAULT 0,

        score REAL
        DEFAULT 0,

        quizzes_completed INTEGER
        DEFAULT 0,

        created_at TEXT,

        updated_at TEXT

    );
```

```
CREATE TABLE IF NOT EXISTS
questions (

        id INTEGER PRIMARY KEY
        AUTOINCREMENT,

        question TEXT NOT NULL,

        option_a TEXT NOT NULL,

        option_b TEXT NOT NULL,
```



```
        option_c TEXT NOT NULL,  
        option_d TEXT NOT NULL,  
        answer TEXT NOT NULL,  
        explanation TEXT,  
        exam TEXT,  
        subject TEXT,  
        source TEXT,  
        hash TEXT UNIQUE,  
        active INTEGER DEFAULT 1,  
        created_at TEXT,  
        updated_at TEXT  
    );
```

```
CREATE TABLE IF NOT EXISTS  
quiz_history (  
    id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
    user_id INTEGER NOT NULL,
```

```

        question_id INTEGER NOT
NULL,

        selected_answer TEXT,

        correct INTEGER DEFAULT 0,

        score REAL DEFAULT 0,

        answered_at TEXT,

        UNIQUE (
            user_id,
            question_id
        ),

        FOREIGN KEY(user_id)
            REFERENCES users(id)
            ON DELETE CASCADE,

        FOREIGN KEY(question_id)
            REFERENCES
questions(id)
            ON DELETE CASCADE

    );

```

```

CREATE TABLE IF NOT EXISTS
quizzes (

        id INTEGER PRIMARY KEY
AUTOINCREMENT,

```

```

        name TEXT NOT NULL,

        description TEXT,

        created_by INTEGER,

        active INTEGER DEFAULT 1,

        created_at TEXT,

        updated_at TEXT

    );

CREATE TABLE IF NOT EXISTS
quiz_questions (

        id INTEGER PRIMARY KEY
    AUTOINCREMENT,

        quiz_id INTEGER NOT NULL,

        question_id INTEGER NOT
    NULL,

        position INTEGER DEFAULT 0,

    UNIQUE (
        quiz_id,
        question_id
    ),

    FOREIGN KEY(quiz_id)

```

```

REFERENCES quizzes(id)
ON DELETE CASCADE,

FOREIGN KEY(question_id)
REFERENCES
questions(id)
ON DELETE CASCADE

);

```

```

CREATE TABLE IF NOT EXISTS
pdf_files (

    id INTEGER PRIMARY KEY
    AUTOINCREMENT,

    filename TEXT,

    path TEXT,

    imported_by INTEGER,

    question_count INTEGER
    DEFAULT 0,

    created_at TEXT

);

```

```

CREATE TABLE IF NOT EXISTS
sources (

```

```
        id INTEGER PRIMARY KEY  
AUTOINCREMENT,  
  
        name TEXT UNIQUE,  
  
        url TEXT,  
  
        enabled INTEGER DEFAULT 1,  
  
        created_at TEXT  
  
    );
```

```
CREATE TABLE IF NOT EXISTS  
topic_sources (  
  
    id INTEGER PRIMARY KEY  
AUTOINCREMENT,  
  
    kind TEXT NOT NULL,  
  
    topic_name TEXT NOT NULL,  
  
    url TEXT NOT NULL,  
  
    added_by INTEGER,  
  
    created_at TEXT  
  
);
```

```
CREATE TABLE IF NOT EXISTS
```

```
settings (  
    key TEXT PRIMARY KEY,  
    value TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS  
import_jobs (  
    id INTEGER PRIMARY KEY  
    AUTOINCREMENT,  
    user_id INTEGER,  
    filename TEXT,  
    status TEXT,  
    question_count INTEGER  
    DEFAULT 0,  
    error TEXT,  
    created_at TEXT,  
    updated_at TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS
```

```
document_import_cache (  
  
            id INTEGER PRIMARY KEY  
AUTOINCREMENT,  
  
            file_hash TEXT UNIQUE NOT  
NULL,  
  
            filename TEXT,  
  
            source_type TEXT,  
  
            question_ids TEXT DEFAULT  
'[]',  
  
            status TEXT DEFAULT  
'completed',  
  
            created_at TEXT,  
  
            updated_at TEXT  
  
        );  
  
        CREATE INDEX IF NOT EXISTS  
idx_document_cache_hash  
        ON  
document_import_cache(file_hash);  
  
        CREATE TABLE IF NOT EXISTS  
bookmarks (  

```

```

        id INTEGER PRIMARY KEY
AUTOINCREMENT,

        user_id INTEGER NOT NULL,

        question_id INTEGER NOT
NULL,

        created_at TEXT,

        UNIQUE (
            user_id,
            question_id
        ),

        FOREIGN KEY(user_id)
            REFERENCES users(id)
            ON DELETE CASCADE,

        FOREIGN KEY(question_id)
            REFERENCES
questions(id)
            ON DELETE CASCADE

    );

```

```

CREATE TABLE IF NOT EXISTS
badges (

        id INTEGER PRIMARY KEY
AUTOINCREMENT,

        user_id INTEGER NOT NULL,

```



```
        badge_code TEXT NOT NULL,

        earned_at TEXT,

        UNIQUE (
            user_id,
            badge_code
        )

    );

CREATE TABLE IF NOT EXISTS
feedback (

        id INTEGER PRIMARY KEY
        AUTOINCREMENT,

        user_id INTEGER,

        question_id INTEGER,

        message TEXT,

        created_at TEXT

    );

CREATE INDEX IF NOT EXISTS
idx_questions_active
ON questions(active);
```

```
CREATE INDEX IF NOT EXISTS  
idx_questions_exam  
ON questions(exam);
```

```
CREATE INDEX IF NOT EXISTS  
idx_questions_subject  
ON questions(subject);
```

```
CREATE INDEX IF NOT EXISTS  
idx_history_user  
ON quiz_history(user_id);
```

```
CREATE INDEX IF NOT EXISTS  
idx_history_question  
ON quiz_history(question_id);
```

```
CREATE INDEX IF NOT EXISTS  
idx_quiz_questions_quiz  
ON quiz_questions(quiz_id);
```

```
CREATE INDEX IF NOT EXISTS  
idx_bookmarks_user  
ON bookmarks(user_id);
```

```
CREATE INDEX IF NOT EXISTS  
idx_feedback_user  
ON feedback(user_id);
```

```

CREATE INDEX IF NOT EXISTS
idx_topic_sources_lookup
ON topic_sources(kind,
topic_name);

    ""
)

# -----
-----
# Column migrations (safe on
existing databases)
# -----
-----

_add_column_if_missing(
    conn, "users",
"current_streak",
    "INTEGER DEFAULT 0"
)
_add_column_if_missing(
    conn, "users", "max_streak",
    "INTEGER DEFAULT 0"
)
_add_column_if_missing(
    conn, "users",
"last_quiz_date",
    "TEXT"
)
_add_column_if_missing(
    conn, "users", "referred_by",

```

```

        "INTEGER"
    )
    _add_column_if_missing(
        conn, "users",
"referral_count",
        "INTEGER DEFAULT 0"
    )
    _add_column_if_missing(
        conn, "questions",
"difficulty",
        "TEXT DEFAULT 'medium'"
    )

    now = utcnow()

    for name, url in DEFAULT_SOURCES:

        conn.execute(
            """
            INSERT OR IGNORE INTO
sources
            (
                name,
                url,
                enabled,
                created_at
            )
            VALUES (?, ?, 1, ?)
            """,
            (
                name,
                url,
                now
            )
        )

```

```

    )
)

conn.execute(
    """
    INSERT OR IGNORE INTO settings
    (
        key,
        value
    )
    VALUES (?, ?)
    """,
    (
        "negative_mark",
        str(DEFAULT_NEGATIVE_MARK)
    )
)

conn.commit()

conn.close()

logger.info(
    "Database initialized"
)

#
=====
=====
# TIME
#

```

```

=====
=====

def utcnow():

    return datetime.now(
        timezone.utc
    ).isoformat()

#
=====
=====
# SETTINGS
#
=====
=====

def get_setting(
    key,
    default=None
):

    with DB_LOCK:

        conn = db()

        row = conn.execute(
            """
            SELECT value
            FROM settings
            WHERE key = ?
            """,
            (key,)

```

```

        ).fetchone()

        conn.close()

    if not row:
        return default

    return row["value"]

def set_setting(
    key,
    value
):
    with DB_LOCK:

        conn = db()

        conn.execute(
            """
            INSERT INTO settings
            (
                key,
                value
            )
            VALUES (?, ?)

            ON CONFLICT(key)
            DO UPDATE SET
                value = excluded.value
            """,
            (
                key,

```

```

        str(value)
    )
)

conn.commit()

conn.close()

def get_negative_mark():

    value = get_setting(
        "negative_mark",
        DEFAULT_NEGATIVE_MARK
    )

    try:
        return float(value)

    except Exception:
        return DEFAULT_NEGATIVE_MARK

def clamp_quiz_time(seconds):

    try:
        seconds = int(seconds)
    except Exception:
        seconds = DEFAULT_QUIZ_TIME_SECONDS

    return max(
        MIN_QUIZ_TIME_SECONDS,
        min(
            seconds,

```



```

        MAX_QUIZ_TIME_SECONDS
    )
)

def get_quiz_time():

    value = get_setting(
        "quiz_time_seconds",
        DEFAULT_QUIZ_TIME_SECONDS
    )

    try:
        seconds = int(float(value))

    except Exception:
        seconds = DEFAULT_QUIZ_TIME_SECONDS

    return clamp_quiz_time(seconds)

#
=====
=====
# TEXT NORMALIZATION
#
=====
=====

def normalize_text(text):

    if text is None:
        return ""

```

```

text = str(text)

text = text.replace(
    "\u200b",
    ""
)

text = text.replace(
    "\xa0",
    " "
)

text = re.sub(
    r"\s+",
    " ",
    text
)

return text.strip().lower()

```

```

#
=====
=====
# QUESTION HASH
#
=====
=====

```

```

def question_hash(
    question,
    option_a,
    option_b,
    option_c,

```

```

        option_d
    ):

    raw = "|".join(
        [
            normalize_text(question),
            normalize_text(option_a),
            normalize_text(option_b),
            normalize_text(option_c),
            normalize_text(option_d),
        ]
    )

    return hashlib.sha256(
        raw.encode(
            "utf-8"
        )
    ).hexdigest()

#
=====
=====
# CLEAN OPTION
#
=====
=====
#
=====
=====
# PART 2/4
# Question Validation + Question Management
# Groq + Web Crawl + PDF/TXT Import
#

```

```

=====
=====

#
=====
=====
# CLEAN OPTION
#
=====
=====

def clean_option(option):
    """
    Option text को साफ करता है।
    A) / A. / (A) जैसे prefixes हटाता है।
    """

    if option is None:
        return ""

    option = str(option).strip()

    option = re.sub(
        r"^\\s*[\\(\\[]?[A-Za-d1-4]
[\\)\\].:\\-]\\s*",
        "",
        option
    )

    option = re.sub(
        r"^\\s*Option\\s*[A-Za-d1-4]\\s*
[:.\\-]\\s*",
        "",

```

```

        option,
        flags=re.IGNORECASE
    )

    return option.strip()

#
=====
=====
# VALIDATE QUESTION PAYLOAD
#
=====
=====

def valid_question_payload(data):
    """
    Question dictionary validate करता है।
    """

    if not isinstance(data, dict):
        return False

    required = [
        "question",
        "option_a",
        "option_b",
        "option_c",
        "option_d",
        "answer",
    ]

    for key in required:

```

```
value = data.get(key)

if value is None:
    return False

if not str(value).strip():
    return False

answer = str(
    data.get("answer")
).strip().upper()

answer = answer.replace(
    "OPTION ",
    ""
)

answer = answer.replace(
    "(",
    ""
).replace(
    ")",
    ""
)

answer = answer.replace(
    ".",
    ""
).replace(
    ":",
    ""
)

if answer not in {
```

```
        "A",
        "B",
        "C",
        "D",
    }:
        return False

    options = [
        clean_option(data.get("option_a")),
        clean_option(data.get("option_b")),
        clean_option(data.get("option_c")),
        clean_option(data.get("option_d")),
    ]

    if any(
        not option
        for option in options
    ):
        return False

    normalized_options = [
        normalize_text(x)
        for x in options
    ]

    if len(
        set(normalized_options)
    ) != 4:
        return False

    question = str(
        data.get("question")
    ).strip()
```

```

if len(question) < 5:
    return False

data["question"] = question

data["option_a"] = options[0]
data["option_b"] = options[1]
data["option_c"] = options[2]
data["option_d"] = options[3]

data["answer"] = answer

data["explanation"] = str(
    data.get("explanation") or ""
).strip()

data["exam"] = str(
    data.get("exam") or ""
).strip()

data["subject"] = str(
    data.get("subject") or ""
).strip()

data["source"] = str(
    data.get("source") or ""
).strip()

return True

```

```

#

```

```

=====
=====

```



```

# ADD QUESTION
#
=====

def add_question(data):
    """
    Database में question add करता है।

    Duplicate question hash के आधार पर रोका
    जाता है।
    """

    if not valid_question_payload(data):
        return None, "invalid"

    q_hash = question_hash(
        data["question"],
        data["option_a"],
        data["option_b"],
        data["option_c"],
        data["option_d"],
    )

    now = utcnow()

    with DB_LOCK:

        conn = db()

        existing = conn.execute(
            """
            SELECT id
            FROM questions

```

```
        WHERE hash = ?
        LIMIT 1
        """
        (q_hash,)
    ).fetchone()

    if existing:

        conn.close()

        return (
            existing["id"],
            "duplicate"
        )

    cursor = conn.execute(
        """
        INSERT INTO questions
        (
            question,
            option_a,
            option_b,
            option_c,
            option_d,
            answer,
            explanation,
            exam,
            subject,
            source,
            hash,
            active,
            created_at,
            updated_at
        )
```

```

VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?,
?, ?, ?, 1, ?, ?)
    "",
    (
        data["question"],
        data["option_a"],
        data["option_b"],
        data["option_c"],
        data["option_d"],
        data["answer"],
        data.get("explanation",
""),
        data.get("exam", ""),
        data.get("subject", ""),
        data.get("source", ""),
        q_hash,
        now,
        now,
    )
)

question_id = cursor.lastrowid

conn.commit()
conn.close()

return question_id, "added"

#
=====
=====
# GET QUESTION
#

```

```

=====
=====

def get_question(question_id):

    try:
        question_id = int(
            question_id
        )
    except Exception:
        return None

    with DB_LOCK:

        conn = db()

        row = conn.execute(
            """
            SELECT *
            FROM questions
            WHERE id = ?
            """,
            (question_id,)
        ).fetchone()

        conn.close()

    return row

#
=====
=====
# GET ALL QUESTIONS

```

```

#
=====

def get_all_questions(
    active_only=True
):

    with DB_LOCK:

        conn = db()

        if active_only:

            rows = conn.execute(
                """
                SELECT *
                FROM questions
                WHERE active = 1
                ORDER BY id ASC
                """
            ).fetchall()

        else:

            rows = conn.execute(
                """
                SELECT *
                FROM questions
                ORDER BY id ASC
                """
            ).fetchall()

    conn.close()

```

```

        return rows

#
=====
=====
# DELETE QUESTION
#
=====
=====

def delete_question(
    question_id
):

    try:
        question_id = int(
            question_id
        )
    except Exception:
        return False

    with DB_LOCK:

        conn = db()

        cursor = conn.execute(
            """
            UPDATE questions
            SET active = 0,
              updated_at = ?
            WHERE id = ?
            """,

```

```

        (
            utcnow(),
            question_id
        )
    )

    conn.commit()

    changed = (
        cursor.rowcount > 0
    )

    conn.close()

    return changed

```

```

#
=====
=====
# UPDATE QUESTION
#
=====
=====

def update_question(
    question_id,
    data
):

    try:
        question_id = int(
            question_id
        )

```

```

except Exception:
    return False, "invalid_id"

if not valid_question_payload(data):
    return False, "invalid"

q_hash = question_hash(
    data["question"],
    data["option_a"],
    data["option_b"],
    data["option_c"],
    data["option_d"],
)

with DB_LOCK:

    conn = db()

    duplicate = conn.execute(
        """
        SELECT id
        FROM questions
        WHERE hash = ?
        AND id != ?
        LIMIT 1
        """,
        (
            q_hash,
            question_id
        )
    ).fetchone()

    if duplicate:

```



```

conn.close()

return (
    False,
    "duplicate"
)

cursor = conn.execute(
    """
    UPDATE questions

    SET question = ?,
        option_a = ?,
        option_b = ?,
        option_c = ?,
        option_d = ?,
        answer = ?,
        explanation = ?,
        exam = ?,
        subject = ?,
        source = ?,
        hash = ?,
        updated_at = ?

    WHERE id = ?
    """,
    (
        data["question"],
        data["option_a"],
        data["option_b"],
        data["option_c"],
        data["option_d"],
        data["answer"],
        data.get("explanation",

```

```

""),
        data.get("exam", ""),
        data.get("subject", ""),
        data.get("source", ""),
        q_hash,
        utcnow(),
        question_id,
    )
)

conn.commit()

changed = (
    cursor.rowcount > 0
)

conn.close()

if not changed:
    return False, "not_found"

return True, "updated"

#
=====
=====
# QUESTION SELECTION
#
=====
=====

def get_quiz_questions(
    user_id,

```

```

        count=10,
        exam=None,
        subject=None,
        quiz_id=None
    ):
        """
        User को ऐसे questions देता है जिन्हें उसने पहले
        use नहीं किया।

        यही function question repetition रोकने
        का मुख्य हिस्सा है।
        """

        try:
            user_id = int(user_id)
        except Exception:
            return []

        try:
            count = int(count)
        except Exception:
            count = DEFAULT_QUIZ_COUNT

        # Quiz question count अब unlimited है -
        # सिर्फ न्यूनतम 1 जरूरी है
        count = max(1, count)

        with DB_LOCK:

            conn = db()

            params = [
                user_id
            ]

```

```

if quiz_id is not None:

    try:
        quiz_id = int(
            quiz_id
        )
    except Exception:
        conn.close()
        return []

    query = """
        SELECT q.*
        FROM questions q
        INNER JOIN quiz_questions
qq
            ON qq.question_id =
q.id

        WHERE qq.quiz_id = ?
        AND q.active = 1
        AND NOT EXISTS (
            SELECT 1
            FROM quiz_history h
            WHERE h.user_id = ?
            AND h.question_id =
q.id
        )
    """

    params = [
        quiz_id,
        user_id
    ]

```

```

else:

    query = """
        SELECT q.*
        FROM questions q
        WHERE q.active = 1

        AND NOT EXISTS (
            SELECT 1
            FROM quiz_history h
            WHERE h.user_id = ?
            AND h.question_id =
q.id
        )
        """

    if exam:

        query += """
            AND LOWER(COALESCE(q.exam,
''))
            = LOWER(?)
        """

        params.append(
            str(exam).strip()
        )

    if subject:

        query += """
            AND
LOWER(COALESCE(q.subject, ''))
            = LOWER(?)
        """

```

```

        """

        params.append(
            str(subject).strip()
        )

        query += """
            ORDER BY RANDOM()
            LIMIT ?
        """

        params.append(
            count
        )

        rows = conn.execute(
            query,
            tuple(params)
        ).fetchall()

        conn.close()

    return rows

```

```

#
=====
=====
# MARK QUESTION USED
#
=====
=====

def mark_question_used(

```

```

        user_id,
        question_id,
        selected_answer="",
        correct=0,
        score=0
    ):
        """
        Question को user history में permanently
        reserve करता है।

        INSERT OR IGNORE इसलिए है ताकि double
        callback से
        question दोबारा history में न जाए।
        """

        try:
            user_id = int(
                user_id
            )

            question_id = int(
                question_id
            )

        except Exception:
            return False

        with DB_LOCK:

            conn = db()

            try:

                cursor = conn.execute(

```

```

        """
        INSERT OR IGNORE INTO
quiz_history
        (
            user_id,
            question_id,
            selected_answer,
            correct,
            score,
            answered_at
        )
        VALUES (?, ?, ?, ?, ?, ?)
        """
        (
            user_id,
            question_id,
            selected_answer or "",
            int(bool(correct)),
            float(score),
            utcnow()
        )
    )

    conn.commit()

    inserted = (
        cursor.rowcount > 0
    )

    finally:

        conn.close()

    return inserted

```



```
#
=====
=====
# ENSURE USER
#
=====
=====

def ensure_user(
    telegram_user
):

    if telegram_user is None:
        return False

    user_id = int(
        telegram_user.id
    )

    username = (
        telegram_user.username
        or ""
    )

    first_name = (
        telegram_user.first_name
        or ""
    )

    last_name = (
        telegram_user.last_name
        or ""
    )
```

```

    )

    now = utcnow()

    with DB_LOCK:

        conn = db()

        conn.execute(
            """
            INSERT INTO users
            (
                id,
                username,
                first_name,
                last_name,
                created_at,
                updated_at
            )
            VALUES (?, ?, ?, ?, ?, ?)

            ON CONFLICT(id)
            DO UPDATE SET
                username =
excluded.username,
                first_name =
excluded.first_name,
                last_name =
excluded.last_name,
                updated_at =
excluded.updated_at
            """,
            (
                user_id,

```

```
        username,  
        first_name,  
        last_name,  
        now,  
        now  
    )  
)
```

```
conn.commit()  
conn.close()
```

```
return True
```

```
#  
=====
```

```
# ADMIN CHECK
```

```
#  
=====
```

```
def is_admin(  
    user_id  
):  
  
    try:  
        return int(  
            user_id  
        ) in ADMIN_IDS  
  
    except Exception:  
        return False
```

```

#
=====
=====
# RESET USER HISTORY
#
=====
=====

def reset_user_history(
    user_id
):

    try:
        user_id = int(
            user_id
        )
    except Exception:
        return False

    with DB_LOCK:

        conn = db()

        conn.execute(
            """
            DELETE FROM quiz_history
            WHERE user_id = ?
            """,
            (user_id,)
        )

        conn.commit()
        conn.close()

```

```

        return True

#
=====
=====
# USER STATS
#
=====
=====

def get_stats(
    user_id
):

    try:
        user_id = int(
            user_id
        )
    except Exception:
        return None

    with DB_LOCK:

        conn = db()

        row = conn.execute(
            """
            SELECT *
            FROM users
            WHERE id = ?
            """,
            (user_id,)

```

```

        ).fetchone()

        conn.close()

    return dict(row) if row else None

#
=====
=====
# UPDATE USER STATS
#
=====
=====

def update_user_stats(
    user_id,
    correct=False,
    skipped=False,
    score_delta=0
):

    try:
        user_id = int(
            user_id
        )
    except Exception:
        return False

    with DB_LOCK:

        conn = db()

        conn.execute(

```

```

"""
INSERT OR IGNORE INTO users
(
    id,
    created_at,
    updated_at
)
VALUES (?, ?, ?)
""",
(
    user_id,
    utcnow(),
    utcnow()
)
)

conn.execute(
    """
UPDATE users

SET questions_answered =
    questions_answered + ?,

    correct_answers =
    correct_answers + ?,

    wrong_answers =
    wrong_answers + ?,

    skipped_questions =
    skipped_questions + ?,

    score =
    score + ?,

```

```

        updated_at = ?

        WHERE id = ?
        """
        (
            1 if not skipped else 0,
            1 if correct and not
skipped else 0,
            1 if not correct and not
skipped else 0,
            1 if skipped else 0,
            float(score_delta),
            utcnow(),
            user_id
        )
    )

    conn.commit()
    conn.close()

    return True

#
=====
=====
# QUIZ COMPLETED
#
=====
=====

def increment_quiz_count(
    user_id

```



```

):

    try:
        user_id = int(
            user_id
        )
    except Exception:
        return False

    with DB_LOCK:

        conn = db()

        conn.execute(
            """
            UPDATE users
            SET quizzes_completed =
                quizzes_completed + 1,
                updated_at = ?
            WHERE id = ?
            """,
            (
                utcnow(),
                user_id
            )
        )

        conn.commit()
        conn.close()

    return True

#

```

```

=====
=====
# STREAK SYSTEM
#
=====
=====

def update_streak(
    user_id
):

    try:
        user_id = int(user_id)
    except Exception:
        return None

    today = datetime.now(
        timezone.utc
    ).date()

    with DB_LOCK:

        conn = db()

        row = conn.execute(
            """
            SELECT current_streak,
                   max_streak,
                   last_quiz_date
            FROM users
            WHERE id = ?
            """,
            (user_id,)
        ).fetchone()

```

```

        if not row:
            conn.close()
            return None

        current = row["current_streak"] or
0
        best = row["max_streak"] or 0
        last_date_str =
row["last_quiz_date"]

        if last_date_str:
            try:
                last_date =
datetime.strptime(
                    last_date_str,
                    "%Y-%m-%d"
                ).date()
            except Exception:
                last_date = None
        else:
            last_date = None

        if last_date == today:
            # Aaj already quiz ho chuka
            hai, streak same rahegi
            conn.close()
            return None

        elif last_date == today -
timedelta(days=1):
            current += 1

        else:

```

```

        current = 1

        best = max(best, current)

        conn.execute(
            """
            UPDATE users
            SET current_streak = ?,
                max_streak = ?,
                last_quiz_date = ?
            WHERE id = ?
            """,
            (
                current,
                best,
                today.isoformat(),
                user_id
            )
        )

        conn.commit()
        conn.close()

    if current > 1:
        return f"🔥 {current} दिन की streak जारी है!"

    return None

#
=====
=====
# ACHIEVEMENT BADGES

```

```

#
=====

=====

BADGE_DEFINITIONS = {
    "first_quiz": "पहला Quiz पूरा किया",
    "100_questions": "100 Questions पूरे
    किए",
    "500_questions": "500 Questions पूरे
    किए",
    "streak_7": "7-दिन की Streak",
    "streak_30": "30-दिन की Streak",
    "score_90": "एक Quiz में 90%+ Accuracy",
    "perfect_quiz": "100% Accuracy (Perfect
    Quiz)",
}

def award_badge(
    user_id,
    badge_code
):

    if badge_code not in BADGE_DEFINITIONS:
        return False

    conn = db()

    try:
        cur = conn.execute(
            """
            INSERT OR IGNORE INTO badges
            (
                user_id,

```

```

        badge_code,
        earned_at
    )
VALUES (?, ?, ?)
""",
(
    user_id,
    badge_code,
    utcnow(),
)
)

conn.commit()

return cur.rowcount > 0

except Exception:
    logger.exception(
        "Failed awarding badge"
    )
    return False

finally:
    conn.close()

def check_and_award_badges(
    user_id,
    total,
    correct,
    accuracy
):

    newly_awarded = []

```

```
stats = get_stats(user_id) or {}

answered = stats.get(
    "questions_answered", 0
) or 0

quizzes = stats.get(
    "quizzes_completed", 0
) or 0

streak = stats.get(
    "current_streak", 0
) or 0

checks = []

if quizzes >= 1:
    checks.append("first_quiz")

if answered >= 100:
    checks.append("100_questions")

if answered >= 500:
    checks.append("500_questions")

if streak >= 7:
    checks.append("streak_7")

if streak >= 30:
    checks.append("streak_30")

if total and accuracy >= 90:
    checks.append("score_90")
```

```

        if total and correct == total:
            checks.append("perfect_quiz")

    for code in checks:
        if award_badge(user_id, code):
            newly_awarded.append(
                BADGE_DEFINITIONS[code]
            )

    return newly_awarded

def get_user_badges(
    user_id
):

    conn = db()

    try:
        rows = conn.execute(
            """
            SELECT badge_code, earned_at
            FROM badges
            WHERE user_id = ?
            ORDER BY earned_at DESC
            """,
            (user_id,)
        ).fetchall()

        return [dict(r) for r in rows]

    finally:
        conn.close()

```



```

#
=====
=====
# BOOKMARKS
#
=====
=====

def add_bookmark(
    user_id,
    question_id
):

    conn = db()

    try:
        conn.execute(
            """
            INSERT OR IGNORE INTO bookmarks
            (
                user_id,
                question_id,
                created_at
            )
            VALUES (?, ?, ?)
            """,
            (
                user_id,
                question_id,
                utcnow(),
            )
        )
    )

```

```
        conn.commit()
        return True

    except Exception:
        logger.exception(
            "Failed adding bookmark"
        )
        return False

    finally:
        conn.close()

def remove_bookmark(
    user_id,
    question_id
):

    conn = db()

    try:
        conn.execute(
            """
            DELETE FROM bookmarks
            WHERE user_id = ?
            AND question_id = ?
            """,
            (user_id, question_id)
        )

        conn.commit()
        return True
```

```

        finally:
            conn.close()

def list_bookmarks(
    user_id,
    limit=10
):

    conn = db()

    try:
        rows = conn.execute(
            """
                SELECT q.*
                FROM bookmarks b
                JOIN questions q
                  ON q.id = b.question_id
                WHERE b.user_id = ?
                ORDER BY b.created_at DESC
                LIMIT ?
            """,
            (user_id, limit)
        ).fetchall()

        return [dict(r) for r in rows]

    finally:
        conn.close()

```

```

#
=====
=====

```

```

# MISTAKE NOTEBOOK / WEAK TOPICS
#
=====
=====

def list_mistakes(
    user_id,
    limit=10
):

    conn = db()

    try:
        rows = conn.execute(
            """
            SELECT q.*
            FROM quiz_history h
            JOIN questions q
              ON q.id = h.question_id
            WHERE h.user_id = ?
            AND h.correct = 0
            AND h.selected_answer IS NOT
NULL
            ORDER BY h.answered_at DESC
            LIMIT ?
            """,
            (user_id, limit)
        ).fetchall()

        return [dict(r) for r in rows]

    finally:
        conn.close()

```

```

def get_weak_topics(
    user_id,
    limit=5
):

    conn = db()

    try:
        rows = conn.execute(
            """
            SELECT
                COALESCE(q.subject,
'General') AS subject,
                COUNT(*) AS wrong_count
            FROM quiz_history h
            JOIN questions q
                ON q.id = h.question_id
            WHERE h.user_id = ?
            AND h.correct = 0
            AND h.selected_answer IS NOT
NULL
            GROUP BY subject
            ORDER BY wrong_count DESC
            LIMIT ?
            """,
            (user_id, limit)
        ).fetchall()

        return [dict(r) for r in rows]

    finally:
        conn.close()

```

```

#
=====
=====
# LEADERBOARD
#
=====
=====

def get_leaderboard(
    limit=10
):

    conn = db()

    try:
        rows = conn.execute(
            """
                SELECT id, username,
first_name, score,
                                correct_answers,
questions_answered
                FROM users
                WHERE questions_answered > 0
                ORDER BY score DESC
                LIMIT ?
            """,
            (limit,)
        ).fetchall()

        return [dict(r) for r in rows]

    finally:
        conn.close()

```

```

#
=====
=====
# FEEDBACK / REPORT
#
=====
=====

def add_feedback(
    user_id,
    question_id,
    message="Reported via button"
):

    conn = db()

    try:
        conn.execute(
            """
            INSERT INTO feedback
            (
                user_id,
                question_id,
                message,
                created_at
            )
            VALUES (?, ?, ?, ?)
            """,
            (
                user_id,
                question_id,
                message,

```

```

        utcnow(),
    )
)

conn.commit()
return True

finally:
    conn.close()

#
=====
=====
# REFERRAL SYSTEM
#
=====
=====

def set_referrer(
    user_id,
    referrer_id
):

    if user_id == referrer_id:
        return False

    conn = db()

    try:
        row = conn.execute(
            """
            SELECT referred_by
            FROM users

```



```

        WHERE id = ?
        """
        (user_id,)
    ).fetchone()

    if not row or row["referred_by"]:
        # पहले से referred है, या user
मौजूद नहीं
        return False

    conn.execute(
        """
        UPDATE users
        SET referred_by = ?
        WHERE id = ?
        """,
        (referrer_id, user_id)
    )

    conn.execute(
        """
        UPDATE users
        SET referral_count =

COALESCE(referral_count, 0) + 1
        WHERE id = ?
        """,
        (referrer_id,)
    )

    conn.commit()
    return True

except Exception:

```

```

        logger.exception(
            "Failed setting referrer"
        )
        return False

    finally:
        conn.close()

#
=====
=====
# GROQ CLIENT
#
=====
=====

groq_client = None

if GROQ_API_KEY:

    try:

        groq_client = Groq(
            api_key=GROQ_API_KEY
        )

    except Exception as e:

        logger.exception(
            "Groq initialization failed:
%s",
            e
        )

```

```

#
=====
=====
# SERPAPI CLIENT STATE
#
=====
=====

SERPAPI_ENABLED = bool(SERPAPI_API_KEY)

if not SERPAPI_ENABLED:
    logger.warning(
        "SERPAPI_API_KEY set नहीं है;
/autoquiz web-search के बिना "
        "सिर्फ AI knowledge से questions
बनाएगा।"
    )

#
=====
=====
# GOOGLE CLOUD TTS STATE
#
=====
=====

GOOGLE_TTS_ENABLED =
bool(GOOGLE_TTS_API_KEY)

if not GOOGLE_TTS_ENABLED:
    logger.warning(

```

```
        "GOOGLE_TTS_API_KEY set नहीं है;  
/speak काम नहीं करेगा।"  
    )
```

```
#  
=====
```

```
# EXTRACT JSON FROM GROQ RESPONSE  
#
```

```
def extract_json(  
    text  
):  
    """  
    Groq response में JSON ढूँढता है।  
    """
```

```
    if not text:  
        return None
```

```
    text = str(  
        text  
    ).strip()
```

```
    # Markdown code fence हटाएँ  
    text = re.sub(  
        r"```(?:json)?",  
        "",  
        text,  
        flags=re.IGNORECASE  
    )
```

```
text = text.replace(
    "`",
    ""
).strip()

# Direct JSON
try:

    return json.loads(
        text
    )

except Exception:
    pass

# JSON object/list खोजें
candidates = []

first_array = text.find("[")
last_array = text.rfind("]")

if (
    first_array >= 0
    and last_array > first_array
):
    candidates.append(
        text[
            first_array:
            last_array + 1
        ]
    )

first_object = text.find("{")
```

```

last_object = text.rfind("{}")

if (
    first_object >= 0
    and last_object > first_object
):
    candidates.append(
        text[
            first_object:
            last_object + 1
        ]
    )

for candidate in candidates:

    try:

        return json.lo
        json.loads(
candidate
)

    except Exception:
        continue

return None

#
=====
=====
# NORMALIZE GENERATED QUESTION
#
=====

```

```
=====

def normalize_generated_question(
    item,
    source=""
):

    if not isinstance(item, dict):
        return None

    question = (
        item.get("question")
        or item.get("q")
        or item.get("Question")
    )

    options = item.get("options")

    if isinstance(options, list) and
len(options) >= 4:

        option_a = options[0]
        option_b = options[1]
        option_c = options[2]
        option_d = options[3]

    else:

        option_a = (
            item.get("option_a")
            or item.get("a")
            or item.get("A")
        )
```

```
option_b = (  
    item.get("option_b")  
    or item.get("b")  
    or item.get("B")  
)  
  
option_c = (  
    item.get("option_c")  
    or item.get("c")  
    or item.get("C")  
)  
  
option_d = (  
    item.get("option_d")  
    or item.get("d")  
    or item.get("D")  
)  
  
answer = (  
    item.get("answer")  
    or item.get("correct_answer")  
    or item.get("correct")  
)  
  
data = {  
    "question": question,  
    "option_a": option_a,  
    "option_b": option_b,  
    "option_c": option_c,  
    "option_d": option_d,  
    "answer": answer,  
    "explanation": item.get(  
        "explanation",  
        ""
```



```

    ),
    "exam": item.get(
        "exam",
        ""
    ),
    "subject": item.get(
        "subject",
        ""
    ),
    "source": (
        item.get("source", "")
        or source
    ),
}

```

```

if not valid_question_payload(data):
    return None

```

```

return data

```

```

#
=====
=====
# GROQ GENERATE QUESTIONS
#
=====
=====

```

```

def groq_generate_questions(
    topic,
    count=10,
    context_text="",
    source=""

```

```
):

    if groq_client is None:
        raise RuntimeError(
            "GROQ_API_KEY configured नहीं है।"
        )

    try:
        count = int(count)
    except Exception:
        count = 10

    count = max(
        1,
        min(
            count,
            MAX_IMPORT_QUESTIONS
        )
    )

    context_text = str(
        context_text or ""
    )[:30000]

    prompt = f"""
    आप एक परीक्षा प्रश्न निर्माण विशेषज्ञ हैं।

    विषय:
    {topic}

    आपको {count} अलग-अलग MCQ प्रश्न बनाने हैं।

    भाषा:
    हिंदी।
```

प्रत्येक प्रश्न में ये fields अनिवार्य हैं:

```
question
option_a
option_b
option_c
option_d
answer
explanation
exam
subject
```

नियम:

1. केवल valid MCQ बनाएं।
2. चारों options अलग होने चाहिए।
3. answer केवल A, B, C या D होना चाहिए।
4. प्रश्न factual और परीक्षा उपयोगी हों।
5. प्रश्न आपस में duplicate नहीं होने चाहिए।
6. दिए गए context का उपयोग करें।
7. कोई अतिरिक्त text न दें।
8. केवल JSON array दें।

Format:

```
[
  {
    "question": "प्रश्न",
    "option_a": "विकल्प A",
    "option_b": "विकल्प B",
    "option_c": "विकल्प C",
    "option_d": "विकल्प D",
    "answer": "A",
```

```

        "explanation": "व्याख्या",
        "exam": "CET",
        "subject": "Polity"
    }}
]

Context:
{context_text}
"""

    last_error = None
    response = None
    for attempt in range(max(1,
GROQ_MAX_RETRIES + 1)):
        try:
            response =
groq_client.chat.completions.create(
                model=GROQ_MODEL,
                messages=[
                    {
                        "role": "system",
                        "content": (
                            "आप MCQ
generator हैं। "
                            "केवल valid
JSON array दें।"
                        )
                    },
                    {
                        "role": "user",
                        "content": prompt
                    }
                ],
                temperature=0.4,

```

```

        max_tokens=12000,
    )
    break
except Exception as exc:
    last_error = exc
    if attempt >= GROQ_MAX_RETRIES:
        raise
    delay = GROQ_RETRY_BASE_SECONDS
    * (2 ** attempt)
    logger.warning(
        "Groq request failed
(attempt %s/%s): %s; retrying in %.1fs",
        attempt + 1,
        GROQ_MAX_RETRIES + 1, exc, delay
    )
    time.sleep(delay)

    if response is None:
        raise last_error or
RuntimeError("Groq request failed")

    content =
response.choices[0].message.content or ""

    parsed = extract_json(
        content
    )

    if isinstance(parsed, dict):

        parsed = (
            parsed.get("questions")
            or parsed.get("data")
            or []

```

```
)

if not isinstance(parsed, list):
    return []

result = []
seen_hashes = set()

for item in parsed:

    data =
normalize_generated_question(
    item,
    source=source
)

    if not data:
        continue

    q_hash = question_hash(
        data["question"],
        data["option_a"],
        data["option_b"],
        data["option_c"],
        data["option_d"],
    )

    if q_hash in seen_hashes:
        continue

    seen_hashes.add(
        q_hash
    )
```

```

        result.append(data)

    return result

#
=====
=====
# HTTP FETCH
#
=====
=====

def fetch_url(url):

    try:

        headers = {
            "User-Agent": (
                "Mozilla/5.0 "
                "(Windows NT 10.0; Win64;
x64) "
                "AppleWebKit/537.36 "
                "(KHTML, like Gecko) "
                "Chrome/120.0
Safari/537.36"
            )
        }

        response = requests.get(
            url,
            headers=headers,
            timeout=REQUEST_TIMEOUT,
        )

        response.raise_for_status()

```

```

        return response.text

    except Exception as e:

        logger.exception(
            "URL fetch failed: %s",
            url
        )

        return ""

#
=====
=====
# HTML TO TEXT
#
=====
=====

def html_to_text(
    html_content
):

    if not html_content:
        return ""

    try:

        soup = BeautifulSoup(
            html_content,
            "html.parser"
        )

```



```
# Unwanted elements remove
for tag in soup([
    "script",
    "style",
    "noscript",
    "svg",
    "iframe",
    "nav",
    "footer",
    "header",
]):

    tag.decompose()

text = soup.get_text(
    separator="\n"
)

lines = []

for line in text.splitlines():

    line = normalize_text(
        line
    )

    if line:
        lines.append(line)

return "\n".join(lines)

except Exception:
```

```

        logger.exception(
            "HTML extraction failed"
        )

        return ""

#
=====
=====
# SERPAPI WEB SEARCH
#
=====
=====

def serpapi_search(
    query,
    num_results=None,
    engine="google"
):
    """
        SerpAPI (https://serpapi.com) के ज़रिए
        search results लाता है।
        engine="google" -> सामान्य Google
        search (organic_results)
        engine="google_news" -> Google News
        (news_results), current affairs के लिए
        हर result में title, snippet, link (और
        news हो तो date) होता है।
    """

    if not SERPAPI_API_KEY:
        raise RuntimeError(
            "SERPAPI_API_KEY configured नहीं

```

```

    )

    num_results = int(
        num_results or
        SERPAPI_RESULTS_COUNT
    )

    num_results = max(
        1,
        min(num_results, 20)
    )

    params = {
        "engine": engine,
        "q": query,
        "num": num_results,
        "hl": "hi",
        "gl": "in",
        "api_key": SERPAPI_API_KEY,
    }

    try:

        response = requests.get(

            "https://serpapi.com/search.json",
            params=params,
            timeout=REQUEST_TIMEOUT,
        )

        response.raise_for_status()

        payload = response.json()

```

```
except Exception:

    logger.exception(
        "SerpAPI request failed
(engine=%s): %s",
        engine,
        query
    )

    raise RuntimeError(
        "SerpAPI से data नहीं मिल सका।"
    )

    if isinstance(payload, dict) and
payload.get("error"):
        raise RuntimeError(
            f"SerpAPI error:
{payload.get('error')}"
        )

    results = []

    result_key = (
        "news_results"
        if engine == "google_news"
        else "organic_results"
    )

    for item in (payload.get(result_key) or
[]):

        title = normalize_text(
            item.get("title") or ""
```

```

    )

    snippet = normalize_text(
        item.get("snippet") or
item.get("summary") or ""
    )

    link = (
        item.get("link") or ""
    ).strip()

    date = normalize_text(
        item.get("date") or ""
    )

    source_name = normalize_text(
        (item.get("source") or
{}).get("name")
        if
isinstance(item.get("source"), dict)
        else (item.get("source") or "")
    )

    if not (title or snippet):
        continue

    results.append({
        "title": title,
        "snippet": snippet,
        "link": link,
        "date": date,
        "source": source_name,
    })

```

```

# Google का सीधा "Answer Box" /
knowledge panel भी उपयोगी context देता है
    answer_box = payload.get("answer_box")
or {}

if isinstance(answer_box, dict):

    extra = normalize_text(
        answer_box.get("snippet")
        or answer_box.get("answer")
        or ""
    )

    if extra:
        results.insert(
            0,
            {
                "title":
answer_box.get("title") or "Answer Box",
                "snippet": extra,
                "link":
answer_box.get("link") or "",
                "date": "",
                "source": "",
            }
        )

    return results[:num_results]

def build_context_from_serpapi(
    topic,
    num_results=None,
    fetch_pages=None,

```

```

        engine="google"
    ):
        """
        किसी topic के लिए SerpAPI search results
        (और चुनिंदा pages) से
        एक combined context text बनाता है, जो
        groq_generate_questions()
        को context_text के रूप में दिया जा सके।
        engine="google_news" देने पर Google News
        results (current affairs) आते हैं।
        """

        results = serpapi_search(
            topic,
            num_results=num_results,
            engine=engine,
        )

        if not results:
            return "", []

        fetch_pages = int(
            SERPAPI_FETCH_PAGES
            if fetch_pages is None
            else fetch_pages
        )

        fetch_pages = max(0, min(fetch_pages,
            len(results)))

        parts = []
        sources = []

        for idx, item in enumerate(results):

```

```

        date_bit = f" ({item['date']})" if
item.get("date") else ""
        source_bit = f" [{item['source']}]"
if item.get("source") else ""

        block = (
            f"Source {idx + 1}:
{item['title']}{date_bit}{source_bit}\n"
            f"{item['snippet']}"
        )

        parts.append(block)

        if item.get("link"):
            sources.append(item["link"])

        # पहले कुछ top results के actual pages भी
        fetch करें
        # ज्यादा गहराई वाला context जोड़ते हैं (best-
        effort, silent fail)
        for item in results[:fetch_pages]:

            link = item.get("link")

            if not link:
                continue

            try:

                page_html = fetch_url(link)

                page_text =
html_to_text(page_html)

```



```

        if page_text:
            parts.append(
                f"Full page
({link}):{page_text[:4000]}"
            )

        except Exception:
            continue

    context_text = "\n\n".join(parts)
    [:SERPAPI_CONTEXT_CHAR_LIMIT]

    return context_text, sources

#
=====
=====
# WIKIPEDIA API (free, no key needed)
#
=====
=====

def wikipedia_search(
    topic,
    num_results=None,
    lang=None
):
    """
    MediaWiki API (generator=search) से किसी
    topic पर सबसे relevant
    Wikipedia pages लाता है, हर page का
    plain-text extract (summary) समेत।

```

कोई API key नहीं चाहिए – Wikipedia का API हमेशा free रहा है।

```
"""

    lang = (lang or WIKIPEDIA_LANG).strip()
    or "hi"

    num_results = int(
        num_results or
        WIKIPEDIA_RESULTS_COUNT
    )

    num_results = max(
        1,
        min(num_results, 10)
    )

    params = {
        "action": "query",
        "generator": "search",
        "gsrsearch": topic,
        "gsrlimit": num_results,
        "prop": "extracts",
        "exintro": 1,
        "explaintext": 1,
        "format": "json",
        "redirects": 1,
    }

    headers = {
        "User-Agent": (
            "TelegramQuizBot/1.0 "
            "(educational MCQ generation)"
        )
    }
```

```

    }

    try:

        response = requests.get(

            f"https://{lang}.wikipedia.org/w/api.php",
            params=params,
            headers=headers,
            timeout=REQUEST_TIMEOUT,
        )

        response.raise_for_status()

        payload = response.json()

    except Exception:

        logger.exception(
            "Wikipedia request failed
(lang=%s): %s",
            lang,
            topic
        )

        raise RuntimeError(
            "Wikipedia से data नहीं मिल सका।"
        )

    pages = (
        payload.get("query",
        {}).get("pages", {})
        if isinstance(payload, dict)
        else {}

```

```

)

results = []

for page in pages.values():

    title = normalize_text(
        page.get("title") or ""
    )

    extract = normalize_text(
        page.get("extract") or ""
    )

    if not (title and extract):
        continue

    url = (

f"https://{lang}.wikipedia.org/wiki/"
        f"{title.replace(' ', '_')}}"
    )

    results.append({
        "title": title,
        "extract": extract,
        "url": url,
    })

# अगर चुनी हुई भाषा (जैसे hi) में कुछ न मिले,
तो अंग्रेज़ी Wikipedia
# पर एक बार और कोशिश करते हैं – इससे नए/
कम-कवर topics भी मिल जाते हैं।
if not results and lang != "en":

```

```

        return wikipedia_search(
            topic,
            num_results=num_results,
            lang="en"
        )

    return results[:num_results]

def build_context_from_wikipedia(
    topic,
    num_results=None,
    lang=None
):
    """
    किसी topic के लिए Wikipedia extracts से
    एक combined context text
    बनाता है, जो groq_generate_questions() को
    context_text के रूप में
    दिया जा सके।
    """

    results = wikipedia_search(
        topic,
        num_results=num_results,
        lang=lang,
    )

    if not results:
        return "", []

    parts = []
    sources = []

```

```

        for idx, item in enumerate(results):

            block = (
                f"Source {idx + 1}:"
                f"{item['title']}\n"
                f"{item['extract']}"
            )

            parts.append(block)
            sources.append(item["url"])

        context_text = "\n\n".join(parts)
        [:WIKIPEDIA_CONTEXT_CHAR_LIMIT]

        return context_text, sources

#
=====
=====
# GOOGLE CLOUD TEXT-TO-SPEECH
#
=====
=====

def google_tts_synthesize(
    text,
    lang_code=None,
    voice_name=None,
    audio_encoding="OGG_OPUS"
):
    """
    Google Cloud Text-to-Speech REST API से
    text को audio bytes में बदलता

```

```

    है (/speak command के लिए)। Telegram
    voice notes के लिए OGG_OPUS
    (default) चाहिए होता है; MP3 भी चाहें तो दे
    सकते हैं।
    """

    if not GOOGLE_TTS_API_KEY:
        raise RuntimeError(
            "GOOGLE_TTS_API_KEY configured
    नहीं है।"
        )

    text = str(text or "").strip()

    if not text:
        raise RuntimeError(
            "Convert करने के लिए text खाली
    है।"
        )

    text = text[:GOOGLE_TTS_CHAR_LIMIT]

    lang_code = (
        lang_code
        or GOOGLE_TTS_LANGUAGE_CODE
        or "hi-IN"
    )

    voice = {"languageCode": lang_code}

    voice_name = voice_name or
    GOOGLE_TTS_VOICE_NAME

    if voice_name:

```

```

        voice["name"] = voice_name

    body = {
        "input": {"text": text},
        "voice": voice,
        "audioConfig": {"audioEncoding":
audio_encoding},
    }

    try:

        response = requests.post(

"https://texttospeech.googleapis.com/v1/text:synthesize",
            params={"key":
GOOGLE_TTS_API_KEY},
            json=body,
            timeout=REQUEST_TIMEOUT,
        )

        response.raise_for_status()

        payload = response.json()

    except Exception:

        logger.exception(
            "Google TTS request failed"
        )

        raise RuntimeError(
            "Google TTS से audio नहीं बन
सका।"

```



```

    )

    audio_b64 = payload.get("audioContent")
    if isinstance(payload, dict) else None

    if not audio_b64:
        raise RuntimeError(
            "Google TTS ने कोई audio नहीं
लौटाया।"
        )

    import base64 as _base64

    return _base64.b64decode(audio_b64)

#
=====
=====
# HTML LINKS
#
=====
=====

def extract_links(
    html_content,
    base_url
):

    if not html_content:
        return []

    links = []

```

```
try:

    soup = BeautifulSoup(
        html_content,
        "html.parser"
    )

    from urllib.parse import (
        urljoin,
        urlparse,
    )

    base_domain = urlparse(
        base_url
    ).netloc

    for tag in soup.find_all(
        "a",
        href=True
    ):

        href = tag.get(
            "href",
            ""
        ).strip()

        if not href:
            continue

        full_url = urljoin(
            base_url,
            href
        )
```

```
        parsed = urlparse(
            full_url
        )

        if parsed.scheme not in (
            "http",
            "https"
        ):
            continue

        # Same domain only
        if (
            parsed.netloc
            and parsed.netloc !=
base_domain
        ):
            continue

        if full_url not in links:
            links.append(
                full_url
            )

    return links

except Exception:

    logger.exception(
        "Link extraction failed"
    )

    return []
```

```

#
=====
=====
# CRAWL SOURCE
#
=====
=====

def crawl_source(
    source_name
):

    conn = db()

    source = conn.execute(
        """
        SELECT *
        FROM sources
        WHERE name = ?
        AND enabled = 1
        """,
        (source_name,)
    ).fetchone()

    conn.close()

    if not source:
        raise ValueError(
            f"Source नहीं मिला:
{source_name}"
        )

    base_url = source["url"]

```

```

html_content = fetch_url(
    base_url
)

if not html_content:
    return {
        "source": source_name,
        "url": base_url,
        "text": "",
        "links": [],
    }

text = html_to_text(
    html_content
)

links = extract_links(
    html_content,
    base_url
)

return {
    "source": source_name,
    "url": base_url,
    "text": text[:50000],
    "links": links[:100],
}

#
=====
=====
# CRAWL + GENERATE QUESTIONS
#

```

```

=====
=====

def scrape_and_generate(
    source_name,
    topic=None,
    count=10
):

    data = crawl_source(
        source_name
    )

    source_text = data.get(
        "text",
        ""
    )

    if not source_text:
        return []

    if topic:

        generation_topic = (
            f"{topic}\n\n"
            f"Source: {source_name}"
        )

    else:

        generation_topic = (
            f"इस source की सामग्री से "
            f"परीक्षा उपयोगी MCQ बनाएं: "
            f"{source_name}"

```

```

    )

    questions = groq_generate_questions(
        topic=generation_topic,
        count=count,
        context_text=source_text,
        source=source_name,
    )

    # ध्यान दें: यह function केवल generate किए
    गए question
    # dictionaries लौटाता है, इन्हें database में
    save नहीं करता।
    # Saving (add_question) caller
    (scrape_command) करता है,
    # ताकि हर question सिर्फ एक बार ही save
    हो।
    return questions

#
=====
=====
# EXAM / SUBJECT LINK LIBRARY
#
=====
=====
# User exam या subject के नाम से खुद कुछ free
quiz-website links
# save कर सकता है (/addexamlink,
/addsubjectlink)। बाद में सिर्फ उस
# exam/subject का नाम देकर (/examquiz,
/subjectquiz), bot उन सभी saved
# links का content पढ़कर, उसी पर आधारित

```

authentic MCQ बना देता है।

```
def _normalize_topic_name(
    topic_name
):
    return normalize_text(
        str(topic_name or "")
    ).strip().lower()

def add_topic_source(
    kind,
    topic_name,
    url,
    added_by=None
):

    topic_key =
    _normalize_topic_name(topic_name)

    url = str(url or "").strip()

    if not topic_key or not url:
        raise ValueError(
            "Topic नाम और link दोनों जरूरी हैं।"
        )

    conn = db()

    try:

        existing = conn.execute(
            """
            SELECT id
```



```

        FROM topic_sources
        WHERE kind = ?
        AND topic_name = ?
        AND url = ?
        """
        (kind, topic_key, url)
    ).fetchone()

    if existing:
        return existing["id"], False

    cursor = conn.execute(
        """
        INSERT INTO topic_sources
        (kind, topic_name, url,
added_by, created_at)
        VALUES (?, ?, ?, ?, ?)
        """,
        (kind, topic_key, url,
added_by, utcnow())
    )

    conn.commit()

    return cursor.lastrowid, True

finally:

    conn.close()

def seed_default_topic_sources():
    """
    DEFAULT_TOPIC_SOURCES में दिए गए

```

```

verified, free, no-login websites
    को startup पर एक बार topic_sources
table में डाल देता है, ताकि
    /subjectquiz और /examquiz पहली बार से ही
काम करें – बिना पहले
    /addsubjectlink या /addexamlink चलाए।
add_topic_source() खुद
    duplicate check करता है, इसलिए हर
restart पर दोबारा चलाना safe है।
"""

added_count = 0

for kind, topic_name, url in
DEFAULT_TOPIC_SOURCES:

    try:

        _, is_new = add_topic_source(
            kind,
            topic_name,
            url,
        )

        if is_new:
            added_count += 1

    except Exception:

        logger.exception(
            "Default topic-source seed
failed: %s / %s / %s",
            kind,
            topic_name,

```

```

        url
    )

    if added_count:
        logger.info(
            "Seeded %s default topic-source
link(s).",
            added_count
        )

def get_topic_sources(
    kind,
    topic_name
):

    topic_key =
_normalize_topic_name(topic_name)

    conn = db()

    try:

        rows = conn.execute(
            """
            SELECT *
            FROM topic_sources
            WHERE kind = ?
            AND topic_name = ?
            ORDER BY created_at ASC
            """,
            (kind, topic_key)
        ).fetchall()

```

```

        return [dict(row) for row in rows]

    finally:

        conn.close()

def list_topic_names(
    kind
):

    conn = db()

    try:

        rows = conn.execute(
            """
            SELECT topic_name, COUNT(*) as
link_count
            FROM topic_sources
            WHERE kind = ?
            GROUP BY topic_name
            ORDER BY topic_name ASC
            """,
            (kind,)
        ).fetchall()

        return [dict(row) for row in rows]

    finally:

        conn.close()

```

```

def build_context_from_topic_links(
    kind,
    topic_name,
    max_links=8,
    chars_per_page=4000
):
    """
    किसी exam/subject के लिए save किए गए सभी
    links को fetch करके
    उनका text एक साथ जोड़ता है – यही
    "authentic" context बनता है
    जिस पर groq_generate_questions() आधारित
    होकर questions बनाता है।
    """

    saved = get_topic_sources(
        kind,
        topic_name
    )

    if not saved:
        return "", [], 0

    saved = saved[:max_links]

    parts = []
    sources = []
    fetched_count = 0

    for item in saved:

        url = item["url"]

        try:

```

```
        html_content = fetch_url(url)

        page_text =
html_to_text(html_content)

        except Exception:

            logger.exception(
                "Topic-source fetch failed:
%S",
                url
            )

            continue

        if not page_text:
            continue

        fetched_count += 1

        parts.append(
            f"Source {fetched_count}
({url}):\n"
            f"{page_text[:chars_per_page]}"
        )

        sources.append(url)

    context_text = "\n\n".join(parts)

    return context_text, sources,
fetched_count
```

```

#
=====
=====
# PDF / TXT EXTRACTION
#
=====
=====

def extract_pdf_text(file_path):
    """Extract text from normal PDFs. OCR
    fallback is handled separately."""
    try:
        from pypdf import PdfReader

        reader = PdfReader(str(file_path))
        pages = []

        for page in reader.pages:
            try:
                text = page.extract_text()
            or ""
                if text.strip():
                    pages.append(text)
            except Exception:
                logger.exception("PDF page
text extraction failed")

        return "\n".join(pages)

    except Exception:
        logger.exception("PDF extraction
failed: %s", file_path)
        return ""

```

```

def _ocr_pdf_with_groq(file_path,
max_pages=None):
    """OCR image/scanned PDF pages with
Groq vision when PyMuPDF is installed."""
    if groq_client is None:
        return ""

    try:
        import base64
        import fitz # PyMuPDF
    except Exception:
        logger.exception("PyMuPDF/base64
unavailable for scanned PDF OCR")
        return ""

    try:
        pdf = fitz.open(str(file_path))
        page_limit = len(pdf) if max_pages
is None else min(len(pdf), int(max_pages))
        chunks = []

        # Groq vision requests can accept
multiple images. Keep batches small for
reliability.
        batch_size = 5
        for batch_start in range(0,
page_limit, batch_size):
            content = [
                {
                    "type": "text",
                    "text": (
                        "इन PDF pages की

```


पूरी readable text हिंदी/अंग्रेजी में ज्यों की त्यों निकालें। "

"Page order बनाए रखें। कोई summary या MCQ न बनाएं; केवल OCR text दें।"

```
        ),
    }
]

for page_index in
range(batch_start, min(batch_start +
batch_size, page_limit)):
    page =
pdf.load_page(page_index)
    pix =
page.get_pixmap(matrix=fitz.Matrix(1.5,
1.5), alpha=False)
    image_b64 =
base64.b64encode(pix.tobytes("jpeg",
jpg_quality=80)).decode("ascii")
    content.append({
        "type": "image_url",
        "image_url": {"url":
f"data:image/jpeg;base64,{image_b64}"},
    })

response =
groq_client.chat.completions.create(
    model=GROQ_VISION_MODEL,
    messages=[
        {"role": "system",
"content": "आप एक high-accuracy OCR
assistant हैं। केवल source की text लौटाएं।"},
        {"role": "user",
```

```

"content": content},
    ],
    temperature=0,
    max_tokens=12000,
)
text =
response.choices[0].message.content or ""
if text.strip():
    chunks.append(text.strip())

pdf.close()
return "\n\n".join(chunks)

except Exception:
    logger.exception("Groq vision OCR
failed for PDF: %s", file_path)
    return ""

def extract_txt_text(file_path):

    try:

        path = Path(file_path)

        # पहले UTF-8
        try:

            return path.read_text(
                encoding="utf-8"
            )

        except UnicodeDecodeError:

```

```

        pass

    # fallback encodings
    for encoding in (
        "utf-8-sig",
        "cp1252",
        "latin-1",
    ):

        try:

            return path.read_text(
                encoding=encoding
            )

        except Exception:
            continue

    return ""

except Exception:

    logger.exception(
        "TXT extraction failed: %s",
        file_path
    )

    return ""

def extract_html_text(file_path):

    try:

```

```
raw = extract_txt_text(
    file_path
)

if not raw.strip():
    return ""

soup = BeautifulSoup(
    raw,
    "html.parser"
)

text = soup.get_text(
    separator="\n"
)

lines = [
    line.strip()
    for line in text.splitlines()
    if line.strip()
]

return "\n".join(lines)

except Exception:

    logger.exception(
        "HTML extraction failed: %s",
        file_path
    )

    return ""
```

```

#
=====
=====
# SAVE PDF FILE RECORD
#
=====
=====

def save_pdf_file(
    user_id,
    file_name,
    file_path
):

    conn = db()

    try:

        cursor = conn.execute(
            """
            INSERT INTO pdf_files
            (
                filename,
                path,
                imported_by,
                created_at
            )
            VALUES (?, ?, ?, ?)
            """,
            (
                file_name,
                str(file_path),
                user_id,
                utcnow(),
            )
        )
    
```

```

        )
    )

    conn.commit()

    return cursor.lastrowid

except Exception:

    logger.exception(
        "PDF record save failed"
    )

    return None

finally:

    conn.close()

```

```

#
=====
=====
# DOCUMENT CACHE / FILE HASH
#
=====
=====

def file_sha256(file_path, chunk_size=1024
* 1024):
    """Calculate a file fingerprint without
loading the whole file into RAM."""
    digest = hashlib.sha256()
    try:

```

```

        with open(file_path, "rb") as fh:
            while True:
                chunk = fh.read(chunk_size)
                if not chunk:
                    break
                digest.update(chunk)
            return digest.hexdigest()
    except Exception:
        logger.exception("File hash
calculation failed: %s", file_path)
        return ""

def get_document_cache(file_hash):
    if not file_hash:
        return None
    with DB_LOCK:
        conn = db()
        try:
            row = conn.execute(
                """
                SELECT file_hash, filename,
source_type, question_ids, status
                FROM document_import_cache
                WHERE file_hash = ?
                LIMIT 1
                """,
                (file_hash,),
            ).fetchone()
            if not row:
                return None
        try:
            ids =
json.loads(row["question_ids"] or "[]")

```

```

        except Exception:
            ids = []
            ids = [int(x) for x in ids if
str(x).isdigit()]
            return {
                "file_hash":
row["file_hash"],
                "filename": row["filename"]
or "",
                "source_type":
row["source_type"] or "",
                "question_ids": ids,
                "status": row["status"] or
"completed",
            }
        finally:
            conn.close()

def save_document_cache(file_hash,
filename, source_type, question_ids,
status="completed"):
    if not file_hash:
        return
    ids_json = json.dumps([int(x) for x in
(question_ids or []) if str(x).isdigit()])
    now = utcnow()
    with DB_LOCK:
        conn = db()
        try:
            conn.execute(
                """
                INSERT INTO
document_import_cache

```



```

        (file_hash, filename,
source_type, question_ids, status,
created_at, updated_at)
        VALUES (?, ?, ?, ?, ?, ?,
?)
        ON CONFLICT(file_hash) DO
UPDATE SET
            filename =
excluded.filename,
            source_type =
excluded.source_type,
            question_ids =
excluded.question_ids,
            status =
excluded.status,
            updated_at =
excluded.updated_at
        "",
        (file_hash, filename,
source_type, ids_json, status, now, now),
    )
    conn.commit()
finally:
    conn.close()

```

```

def get_cached_question_ids(file_hash):
    """Return only active questions that
    still exist; stale IDs are discarded."""
    cached = get_document_cache(file_hash)
    if not cached or cached.get("status")
    != "completed":
        return []
    ids = cached.get("question_ids", [])

```

```

        if not ids:
            return []
        placeholders = ",".join("?" for _ in
ids)
        with DB_LOCK:
            conn = db()
            try:
                rows = conn.execute(
                    f"SELECT id FROM questions
WHERE active = 1 AND id IN
({placeholders})",
                    tuple(ids),
                ).fetchall()
                existing = {int(r["id"]) for r
in rows}
                return [qid for qid in ids if
qid in existing]
            finally:
                conn.close()

#
=====
=====
# IMPORT QUESTIONS FROM TEXT
#
=====
=====

def import_questions_from_text(
    text,
    source="import",
    count=None
):

```

```
"""Generate and save MCQs from the
complete document.
```

```
    If count is supplied, preserve the old
    fixed-count behavior.
```

```
    If count is None, automatically process
    the whole text in chunks and
    generate as many useful non-duplicate
    questions as the configured limit allows.
```

```
    """
    result = {"ids": [], "added": 0,
"duplicate": 0, "failed": 0, "total": 0}
    if not text or not str(text).strip():
        return result
```

```
    text = str(text)
```

```
    # Legacy/manual fixed-count import.
    if count is not None:
        try:
            count = int(count)
        except Exception:
            count = DEFAULT_QUIZ_COUNT
        count = max(1, min(count,
MAX_IMPORT_QUESTIONS))
        question_batches = [text[:50000]]
        counts = [count]
    else:
        # Automatic mode: do not truncate
        the document to the first 50k chars.
        chunk_size = max(4000,
AUTO_IMPORT_CHUNK_CHARS)
        per_chunk = max(1,
AUTO_IMPORT_QUESTIONS_PER_CHUNK)
```

```

        max_questions = max(1,
min(AUTO_IMPORT_MAX_QUESTIONS,
MAX_IMPORT_QUESTIONS))
        question_batches = [
            text[i:i + chunk_size]
            for i in range(0, len(text),
chunk_size)
            if text[i:i +
chunk_size].strip()
        ]
        counts = [min(per_chunk,
max_questions)] * len(question_batches)

        generated_hashes = set()

        for chunk_index, (chunk, batch_count)
in enumerate(zip(question_batches, counts),
start=1):
            if not result["ids"] and
chunk_index == 1:
                pass
                remaining = MAX_IMPORT_QUESTIONS -
result["added"]
                if count is None:
                    remaining = min(remaining,
AUTO_IMPORT_MAX_QUESTIONS -
result["added"])
                if remaining <= 0:
                    break
                batch_count = min(batch_count,
remaining)

            try:
                questions =

```

```

groq_generate_questions(
    topic=(
        "दिए गए अध्ययन सामग्री के
इस भाग से केवल source-grounded, "
        "महत्वपूर्ण परीक्षा उपयोगी MCQ
तैयार करें। "
        "जहाँ source में पहले से
प्रश्न हैं, उन्हें भी पहचानकर duplicate न बनाएं।"
    ),
    count=batch_count,
    context_text=chunk,
    source=source,
)
except Exception:
    logger.exception("MCQ
generation failed for %s chunk %s", source,
chunk_index)
    result["failed"] += 1
    continue

    result["total"] += len(questions)

    for question in questions:
        try:
            q_hash = question_hash(
                question["question"],
                question["option_a"], question["option_b"],
                question["option_c"],
                question["option_d"]
            )
            if q_hash in
generated_hashes:
                result["duplicate"] +=
1

```

```

        continue

generated_hashes.add(q_hash)

        question_id, status =
add_question(question)
        if question_id and status
== "added":
            result["added"] += 1

result["ids"].append(question_id)
        elif question_id and status
== "duplicate":
            result["duplicate"] +=
1
        else:
            result["failed"] += 1
    except Exception:
        logger.exception("Imported
question save failed")
        result["failed"] += 1

    return result

#
=====
=====
# IMPORT PDF
#
=====
=====

def import_pdf(

```

```

        user_id,
        file_path,
        file_name,
        count=None
    ):
        empty_result = {"ids": [], "added": 0,
                        "duplicate": 0, "failed": 0, "total": 0,
                        "cached": False}

        # Same PDF दोबारा आने पर OCR/Groq
        generation बिल्कुल दोबारा नहीं चलेगा.
        # SHA-256 streaming hash RAM usage को
        लगभग constant रखता है.
        document_hash = file_sha256(file_path)
        cached_ids =
        get_cached_question_ids(document_hash)
        if cached_ids:
            return {
                "ids": cached_ids,
                "added": 0,
                "duplicate": len(cached_ids),
                "failed": 0,
                "total": len(cached_ids),
                "cached": True,
            }

        text = extract_pdf_text(file_path)

        # Scanned/image PDFs often return
        little or no text through pypdf.
        # Automatically fall back to Groq
        vision OCR instead of rejecting the PDF.
        if len(text.strip()) < 100:
            ocr_text =

```

```

_ocr_pdf_with_groq(file_path)
    if ocr_text.strip():
        text = ocr_text

    if not text.strip():
        return empty_result

    save_pdf_file(
        user_id=user_id,
        file_name=file_name,
        file_path=file_path,
    )

    result = import_questions_from_text(
        text=text,
        source=file_name,
        count=count,
    )

    # Empty/failed imports are not cached,
    # so a temporary Groq/network failure
    # can be retried normally on the next
    # upload.
    if document_hash and result.get("ids"):
        save_document_cache(
            document_hash,
            file_name,
            "pdf",
            result.get("ids", []),
            status="completed",
        )

    result["cached"] = False
    return result

```



```

#
=====
=====
# IMPORT TXT
#
=====
=====

def import_txt(
    file_path,
    file_name,
    count=None
):

    text = extract_txt_text(
        file_path
    )

    empty_result = {
        "ids": [],
        "added": 0,
        "duplicate": 0,
        "failed": 0,
        "total": 0,
    }

    if not text.strip():
        return empty_result

    return import_questions_from_text(
        text=text,
        source=file_name,

```

```

        count=count,
    )

#
=====
=====
# IMPORT HTML
#
=====
=====

def import_html(
    file_path,
    file_name,
    count=None
):

    text = extract_html_text(
        file_path
    )

    empty_result = {
        "ids": [],
        "added": 0,
        "duplicate": 0,
        "failed": 0,
        "total": 0,
    }

    if not text.strip():
        return empty_result

    return import_questions_from_text(

```

```

        text=text,
        source=file_name,
        count=count,
    )

#
=====
=====
# QUESTION TEXT PARSER
#
=====
=====

def parse_question_text(text):

    if not text:
        return None

    text = str(text).strip()

    question_match = re.search(
        r"(?:^\n)\s*(?:Question\s*:\s*)?"
        r"(.+?)"
        r"\n\s*A[\]\.\.:]\s*(.+?)"
        r"\n\s*B[\]\.\.:]\s*(.+?)"
        r"\n\s*C[\]\.\.:]\s*(.+?)"
        r"\n\s*D[\]\.\.:]\s*(.+?)"
        r"(?:\n|$)",
        text,
        re.IGNORECASE | re.DOTALL,
    )

    if not question_match:

```

```

        return None

    question =
question_match.group(1).strip()

    option_a =
question_match.group(2).strip()
    option_b =
question_match.group(3).strip()
    option_c =
question_match.group(4).strip()
    option_d =
question_match.group(5).strip()

    answer_match = re.search(
        r"(?:Answer|Correct\s*Answer)"
        r"\s*[:\-\]? \s*"
        r"([ABCD])\b",
        text,
        re.IGNORECASE,
    )

    if not answer_match:
        return None

    answer = (
        answer_match
        .group(1)
        .upper()
    )

    explanation_match = re.search(
        r"Explanation\s*[:\-\]? \s*(.+?)(?
        =\n\s*(?:Exam|Subject)\s*[:|$)",

```

```

        text,
        re.IGNORECASE | re.DOTALL,
    )

    exam_match = re.search(
        r"Exam\s*[:\-\]?s*(.+?)(?=\n|$)",
        text,
        re.IGNORECASE,
    )

    subject_match = re.search(
        r"Subject\s*[:\-\]?s*(.+?)(?
= \n|$)",
        text,
        re.IGNORECASE,
    )

    data = {
        "question": question,
        "option_a": option_a,
        "option_b": option_b,
        "option_c": option_c,
        "option_d": option_d,
        "answer": answer,
        "explanation": (
            explanation_match.group(1).strip()
            if explanation_match
            else ""
        ),
        "exam": (
            exam_match.group(1).strip()
            if exam_match
            else ""

```

```

        ),
        "subject": (
            subject_match.group(1).strip()
            if subject_match
            else ""
        ),
        "source": "manual",
    }

    if not valid_question_payload(data):
        return None

    return data
#
=====
=====
# QUIZ QUESTION SELECTION
#
=====
=====

def get_quiz_questions(
    user_id,
    count=10,
    exam=None,
    subject=None,
    quiz_id=None
):

    try:
        count = int(count)
    except Exception:
        count = DEFAULT_QUIZ_COUNT

```

```

count = max(1, count)

conn = db()

try:

    params = [user_id]

    conditions = [
        "q.active = 1",
        """
        NOT EXISTS (
            SELECT 1
            FROM quiz_history h
            WHERE h.user_id = ?
            AND h.question_id = q.id
        )
        """
    ]

    if exam:

        conditions.append(
            "LOWER(TRIM(COALESCE(q.exam, ''))) =
            LOWER(TRIM(?))"
        )

        params.append(exam)

    if subject:

        conditions.append(

```

```

"LOWER(TRIM(COALESCE(q.subject, ''))) =
LOWER(TRIM(?))"
        )

        params.append(subject)

    if quiz_id:

        conditions.append(
            """
            q.id IN (
                SELECT question_id
                FROM quiz_questions
                WHERE quiz_id = ?
            )
            """
        )

        params.append(quiz_id)

    sql = f"""
    SELECT q.*
    FROM questions q
    WHERE {" AND
".join(conditions)}
    ORDER BY RANDOM()
    LIMIT ?
    """

    params.append(count)

    rows = conn.execute(
        sql,
        tuple(params)
    )

```



```

        ).fetchall()

        return [
            dict(row)
            for row in rows
        ]

    finally:

        conn.close()

#
=====
=====
# MARK QUESTION AS USED
#
=====
=====

def mark_question_used(
    user_id,
    question_id,
    selected_answer=None,
    correct=None,
    score_delta=0
):

    conn = db()

    try:

        conn.execute(
            """

```

```

        INSERT INTO quiz_history
        (
            user_id,
            question_id,
            selected_answer,
            correct,
            score,
            answered_at
        )
        VALUES (?, ?, ?, ?, ?, ?)

        ON CONFLICT(user_id,
question_id)
        DO UPDATE SET
            selected_answer =
excluded.selected_answer,
            correct = excluded.correct,
            score = excluded.score,
            answered_at =
excluded.answered_at
            "",
        (
            user_id,
            question_id,
            selected_answer,
            1 if correct else 0,
            float(score_delta),
            utcnow(),
        )
    )

    conn.commit()

    return True

```

```

except Exception:

    logger.exception(
        "Failed marking question used"
    )

    return False

finally:

    conn.close()


#
=====
=====
# CHECK WHETHER QUESTION WAS USED
#
=====
=====

def question_was_used(
    user_id,
    question_id
):

    conn = db()

    try:

        row = conn.execute(
            """
            SELECT 1

```

```

        FROM quiz_history
        WHERE user_id = ?
        AND question_id = ?
        LIMIT 1
        """
        (
            user_id,
            question_id,
        )
    ).fetchone()

    return row is not None

finally:

    conn.close()

```

```

#
=====
=====
# GET UNSEEN QUESTION COUNT
#
=====
=====

def get_unseen_question_count(
    user_id,
    exam=None,
    subject=None
):

    conn = db()

```

```

try:

    params = [user_id]

    conditions = [
        "q.active = 1",
        """
        NOT EXISTS (
            SELECT 1
            FROM quiz_history h
            WHERE h.user_id = ?
            AND h.question_id = q.id
        )
        """
    ]

    if exam:

        conditions.append(

"LOWER(TRIM(COALESCE(q.exam, ''))) =
LOWER(TRIM(?))"
        )

        params.append(exam)

    if subject:

        conditions.append(

"LOWER(TRIM(COALESCE(q.subject, ''))) =
LOWER(TRIM(?))"
        )

```

```

        params.append(subject)

    sql = f"""
        SELECT COUNT(*)
        FROM questions q
        WHERE {" AND "
.join(conditions)}
        """

    row = conn.execute(
        sql,
        tuple(params)
    ).fetchone()

    return int(row[0] or 0)

finally:

    conn.close()

#
=====
=====
# SAVE QUIZ
#
=====
=====

def create_quiz(
    user_id,
    title,
    exam=None,
    subject=None,

```

```

        negative_mark=None
    ):

        conn = db()

        try:

            # NOTE: `quizzes` table के actual
            columns हैं:
            # name, description, created_by,
            active, created_at, updated_at.
            # (पहले यहाँ
            title/user_id/exam/subject/negative_mark
            columns में
            # insert होता था, जो table में मौजूद ही
            नहीं थे – इसी वजह से हर बार
            # "Quiz create नहीं हो सका" error
            आता था और आगे कोई quiz नहीं बनता था।)

            description_parts = []

            if exam:

                description_parts.append(f"Exam: {exam}")

            if subject:

                description_parts.append(f"Subject:
                {subject}")

            description = " |
            ".join(description_parts)

            now = utcnow()

```

```
cursor = conn.execute(
    """
    INSERT INTO quizzes
    (
        name,
        description,
        created_by,
        active,
        created_at,
        updated_at
    )
    VALUES (?, ?, ?, 1, ?, ?)
    """,
    (
        title,
        description,
        user_id,
        now,
        now,
    )
)

conn.commit()

return cursor.lastrowid

except Exception:

    logger.exception(
        "Quiz creation failed"
    )

    return None
```



```

        finally:

            conn.close()

#
=====
=====
# ADD QUESTION TO QUIZ
#
=====
=====

def add_question_to_quiz(
    quiz_id,
    question_id,
    position
):

    conn = db()

    try:

        conn.execute(
            """
            INSERT OR IGNORE INTO
quiz_questions
            (
                quiz_id,
                question_id,
                position
            )
            VALUES (?, ?, ?)

```

```

        """
        (
            quiz_id,
            question_id,
            position,
        )
    )

    conn.commit()

    return True

except Exception:

    logger.exception(
        "Failed adding question to
quiz"
    )

    return False

finally:

    conn.close()

#
=====
=====
# SAVE QUIZ QUESTIONS
#
=====
=====

```

```

def save_quiz_questions(
    quiz_id,
    question_ids
):

    for position, question_id in enumerate(
        question_ids,
        start=1
    ):

        add_question_to_quiz(
            quiz_id,
            question_id,
            position
        )

    return True


#
=====
=====
# CREATE QUIZ FROM A LIST OF QUESTION IDS
# (used to auto-build a quiz right after
# PDF/TXT import,
# HTML scrape, or AI generation)
#
=====
=====

def create_quiz_from_question_ids(
    user_id,
    title,
    question_ids,

```

```

        exam=None,
        subject=None
    ):

        if not question_ids:
            return None

        quiz_id = create_quiz(
            user_id=user_id,
            title=title,
            exam=exam,
            subject=subject,
        )

        if not quiz_id:
            return None

        save_quiz_questions(
            quiz_id,
            question_ids
        )

        return quiz_id

#
=====
=====
# GET SAVED QUIZ
#
=====
=====

def get_quiz(

```

```

        quiz_id
    ):

        conn = db()

        try:

            row = conn.execute(
                """
                SELECT *
                FROM quizzes
                WHERE id = ?
                """,
                (quiz_id,)
            ).fetchone()

            if not row:
                return None

            data = dict(row)

            # बाकी code में कुछ जगह quiz["title"]
            पढ़ा जाता है, जबकि actual
            # column "name" है – backward-compat
            के लिए alias जोड़ रहे हैं।
            data.setdefault("title",
            data.get("name"))

            return data

        finally:

            conn.close()

```

```

#
=====
=====
# GET SAVED QUIZ QUESTIONS
#
=====
=====

def get_saved_quiz_questions(
    quiz_id
):

    conn = db()

    try:

        rows = conn.execute(
            """
            SELECT q.*
            FROM questions q
            INNER JOIN quiz_questions qq
                ON qq.question_id = q.id
            WHERE qq.quiz_id = ?
            AND q.active = 1
            ORDER BY qq.position ASC
            """,
            (quiz_id,)
        ).fetchall()

        return [
            dict(row)
            for row in rows
        ]

```

```

        finally:

            conn.close()

#
=====
=====
# GET ONLY UNSEEN SAVED QUIZ QUESTIONS
#
=====
=====

def get_unseen_saved_quiz_questions(
    user_id,
    quiz_id,
    count=10
):

    try:
        count = int(count)
    except Exception:
        count = DEFAULT_QUIZ_COUNT

    conn = db()

    try:

        rows = conn.execute(
            """
            SELECT q.*
            FROM questions q
            INNER JOIN quiz_questions qq

```

```

        ON qq.question_id = q.id
        WHERE qq.quiz_id = ?
        AND q.active = 1

        AND NOT EXISTS (
            SELECT 1
            FROM quiz_history h
            WHERE h.user_id = ?
            AND h.question_id = q.id
        )

        ORDER BY qq.position ASC
        LIMIT ?
        "",
        (
            quiz_id,
            user_id,
            count,
        )
    ).fetchall()

    return [
        dict(row)
        for row in rows
    ]

finally:

    conn.close()

```

```
#
```

```
=====
=====
```



```

# CREATE QUIZ AND RANDOM QUESTIONS
#
=====

def create_random_quiz(
    user_id,
    title="New Quiz",
    count=10,
    exam=None,
    subject=None
):

    questions = get_quiz_questions(
        user_id=user_id,
        count=count,
        exam=exam,
        subject=subject,
    )

    if not questions:
        return None, []

    quiz_id = create_quiz(
        user_id=user_id,
        title=title,
        exam=exam,
        subject=subject,
    )

    if not quiz_id:
        return None, []

    question_ids = [

```

```

        q["id"]
    for q in questions
]

save_quiz_questions(
    quiz_id,
    question_ids
)

return quiz_id, questions

#
=====
=====
# START QUIZ SESSION
#
=====
=====

def create_quiz_session(
    user_id,
    questions,
    quiz_id=None,
    time_limit=None
):

    session_id = uuid.uuid4().hex[:12]

    ACTIVE_QUIZZES[
        session_id
    ] = {
        "user_id": user_id,
        "quiz_id": quiz_id,

```

```

        "questions": questions,
        "index": 0,
        "score": 0,
        "correct": 0,
        "wrong": 0,
        "skipped": 0,
        "started_at": time.time(),
        "answered": set(),
        "time_limit": clamp_quiz_time(
            time_limit or get_quiz_time()
        ),
        "current_poll_id": None,
    }

```

```

    return session_id

```

```

#
=====
=====
# GET QUIZ SESSION
#
=====
=====

def get_quiz_session(
    session_id
):

    return ACTIVE_QUIZZES.get(
        session_id
    )

```

```

#
=====
=====
# DELETE QUIZ SESSION
#
=====
=====

def remove_quiz_session(
    session_id
):

    ACTIVE_QUIZZES.pop(
        session_id,
        None
    )

#
=====
=====
# GET CURRENT QUESTION
#
=====
=====

def get_current_question(
    session_id
):

    session = get_quiz_session(
        session_id
    )

```

```

    if not session:
        return None

    index = session.get(
        "index",
        0
    )

    questions = session.get(
        "questions",
        []
    )

    if index < 0 or index >= len(
        questions
    ):
        return None

    return questions[index]
#
=====
=====
# QUIZ DISPLAY
#
=====
=====

ANSWER_LETTERS = ["A", "B", "C", "D"]

async def send_quiz_question(
    context,
    chat_id,
    session_id

```

```

):

    session = get_quiz_session(
        session_id
    )

    if not session:
        return

    question = get_current_question(
        session_id
    )

    if not question:

        await send_scorecard(
            context,
            chat_id,
            session
        )

        remove_quiz_session(
            session_id
        )

        return

    index = session["index"] + 1
    total = len(session["questions"])

    options = [
        str(question["option_a"])[0:100],
        str(question["option_b"])[0:100],
        str(question["option_c"])[0:100],

```

```

        str(question["option_d"])[ :100],
    ]

    correct_answer = str(
        question["answer"]
    ).strip().upper()

    correct_option_id = (
ANSWER_LETTERS.index(correct_answer)
        if correct_answer in ANSWER_LETTERS
        else 0
    )

    explanation = (
        question.get(
            "explanation",
            ""
        )
        or ""
    ).strip()

    # Telegram poll explanation limit ~200
    characters
    poll_explanation = (
        explanation[:195] + "..."
        if len(explanation) > 195
        else explanation
    )

    time_limit = clamp_quiz_time(
        session.get("time_limit")
        or get_quiz_time()
    )

```

```

question_text = (
    f"प्रश्न {index}/{total}\n\n"
    f"{question['question']}"
)[:290]

message = await context.bot.send_poll(
    chat_id=chat_id,
    question=question_text,
    options=options,
    type=Poll.QUIZ,

correct_option_id=correct_option_id,
    is_anonymous=False,
    explanation=poll_explanation or
None,
    open_period=time_limit,
    reply_markup=InlineKeyboardMarkup(
        [
            [
                InlineKeyboardButton(
                    "👉 Save",
                    callback_data=f"bm:
{question['id']}"
                ),
                InlineKeyboardButton(
                    "⚠️ Report",
                    callback_data=f"fb:
{question['id']}"
                ),
            ],
            [
                InlineKeyboardButton(
                    "Stop Quiz",

```



```

callback_data=f"stopquiz:{session_id}"
        )
    ]
    ],
    ),
)

ACTIVE_POLLS[
    message.poll.id
] = {
    "session_id": session_id,
    "question_id": question["id"],
    "chat_id": chat_id,
    "message_id": message.message_id,
    "answered": False,
}

session["current_poll_id"] =
message.poll.id

asyncio.create_task(
    quiz_timeout_watcher(
        context,
        session_id,
        message.poll.id,
        time_limit
    )
)

#
=====
=====

```

```

# QUIZ TIMEOUT WATCHER
#
=====

=====

async def quiz_timeout_watcher(
    context,
    session_id,
    poll_id,
    time_limit
):

    await asyncio.sleep(
        time_limit + 2
    )

    poll_info = ACTIVE_POLLS.get(
        poll_id
    )

    if not poll_info:
        return

    if poll_info.get("answered"):
        return

    session = get_quiz_session(
        session_id
    )

    if not session:

        ACTIVE_POLLS.pop(
            poll_id,

```

```
        None
    )

    return

    if session.get(
        "current_poll_id"
    ) != poll_id:

        ACTIVE_POLLS.pop(
            poll_id,
            None
        )

    return

    poll_info["answered"] = True

    question_id = poll_info["question_id"]

    session["skipped"] += 1

    session["answered"].add(
        question_id
    )

    mark_question_used(
        session["user_id"],
        question_id
    )

    chat_id = poll_info["chat_id"]

    try:
```

```

        await context.bot.send_message(
            chat_id=chat_id,
            text="🕒 समय समाप्त हो गया। अगला
प्रश्न..."
        )

    except Exception:
        logger.exception(
            "Timeout notice failed"
        )

    ACTIVE_POLLS.pop(
        poll_id,
        None
    )

    session["index"] += 1

    await advance_quiz(
        context,
        session_id,
        chat_id
    )

#
=====
=====
# ADVANCE QUIZ / SCORECARD
#
=====
=====

```

```
async def advance_quiz(
    context,
    session_id,
    chat_id
):

    session = get_quiz_session(
        session_id
    )

    if not session:
        return

    if session["index"] < len(
        session["questions"]
    ):

        await send_quiz_question(
            context,
            chat_id,
            session_id
        )

    else:

        await send_scorecard(
            context,
            chat_id,
            session
        )

        remove_quiz_session(
            session_id
        )
```

```
async def send_scorecard(
    context,
    chat_id,
    session
):

    total = len(
        session["questions"]
    )

    correct = session["correct"]
    wrong = session["wrong"]
    skipped = session["skipped"]
    score = session["score"]

    accuracy = (
        (correct / total) * 100
        if total else 0
    )

    user_id = session["user_id"]

    increment_quiz_count(
        user_id
    )

    streak_text = update_streak(
        user_id
    )

    new_badges = check_and_award_badges(
        user_id,
```

```

        total=total,
        correct=correct,
        accuracy=accuracy
    )

    text = (
        "🏆 Quiz पूरा हो गया!\n\n"
        f"कुल प्रश्न: {total}\n"
        f"सही: {correct}\n"
        f"गलत: {wrong}\n"
        f"छूटे/Skip: {skipped}\n"
        f"Accuracy: {accuracy:.1f}%\n"
        f"स्कोर: {score:.2f}"
    )

    if streak_text:
        text += f"\n\n{streak_text}"

    if new_badges:
        text += (
            "\n\n🏅 नया Badge मिला:\n"
            + "\n".join(
                f"• {b}" for b in
new_badges
            )
        )

    await context.bot.send_message(
        chat_id=chat_id,
        text=text
    )

#

```

```
=====
=====
# /QUIZ
#
=====
=====

async def quiz_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)


    args = context.args

    count = DEFAULT_QUIZ_COUNT

    time_limit = None

    if args:

        try:
            count = int(args[0])
        except Exception:
            count = DEFAULT_QUIZ_COUNT

    if len(args) > 1:
```



```

        try:
            time_limit =
clamp_quiz_time(
                int(args[1])
            )
        except Exception:
            time_limit = None

        # Quiz question count अब unlimited है -
        सिर्फ न्यूनतम 1 जरूरी है
        count = max(1, count)

        questions = get_quiz_questions(
            user_id=user.id,
            count=count
        )

        if not questions:

            unseen = get_unseen_question_count(
                user.id
            )

            if unseen == 0:

                await
update.message.reply_text(
                    "आपके लिए सभी उपलब्ध
questions "
                    "पहले ही इस्तेमाल हो चुके हैं।
\n\n"
                    "नई questions add/import
करें।"
                )

```

```

        else:

            await
            update.message.reply_text(
                "Quiz के लिए questions
उपलब्ध नहीं हैं।"
            )

            return

        session_id = create_quiz_session(
            user_id=user.id,
            questions=questions,
            time_limit=time_limit
        )

        await send_quiz_question(
            context,
            update.effective_chat.id,
            session_id
        )

#
=====
=====
# /QUIZID
#
=====
=====

async def quizid_command(
    update,

```

```
context
):

    user = update.effective_user

    ensure_user(user)

    if not context.args:

        await update.message.reply_text(
            "Usage:\n"
            "/quizid QUIZ_ID [seconds]"
        )

        return

    try:

        quiz_id = int(
            context.args[0]
        )

    except Exception:

        await update.message.reply_text(
            "Invalid Quiz ID."
        )

        return
```

```

time_limit = None

if len(context.args) > 1:

    try:
        time_limit = clamp_quiz_time(
            int(context.args[1])
        )
    except Exception:
        time_limit = None

quiz = get_quiz(
    quiz_id
)

if not quiz:

    await update.message.reply_text(
        "Quiz नहीं मिला।"
    )

    return

questions =
get_unseen_saved_quiz_questions(
    user_id=user.id,
    quiz_id=quiz_id,
    count=10_000_000
)

if not questions:

    await update.message.reply_text(
        "इस quiz के सभी questions "
    )

```

```

        "आप पहले ही कर चुके हैं।"
    )

    return

    session_id = create_quiz_session(
        user_id=user.id,
        questions=questions,
        quiz_id=quiz_id,
        time_limit=time_limit
    )

    await send_quiz_question(
        context,
        update.effective_chat.id,
        session_id
    )

#
=====
=====
# POLL ANSWER HANDLER (native quiz poll)
#
=====
=====

async def poll_answer_handler(
    update,
    context
):

    poll_answer = update.poll_answer

```

```
if not poll_answer:
    return

poll_id = poll_answer.poll_id

poll_info = ACTIVE_POLLS.get(
    poll_id
)

if not poll_info:
    return

if poll_info.get("answered"):
    return

session_id = poll_info["session_id"]

session = get_quiz_session(
    session_id
)

if not session:

    ACTIVE_POLLS.pop(
        poll_id,
        None
    )

    return

user_id = (
    poll_answer.user.id
    if poll_answer.user
    else None
```

```

    )

    if user_id != session["user_id"]:
        return

    question_id = poll_info["question_id"]

    if question_id in session["answered"]:

        ACTIVE_POLLS.pop(
            poll_id,
            None
        )

        return

    poll_info["answered"] = True

    session["answered"].add(
        question_id
    )

    index = session["index"]
    question = session["questions"][index]

    correct_answer = str(
        question["answer"]
    ).strip().upper()

    selected_ids = poll_answer.option_ids
or []

    selected = (
        ANSWER_LETTERS[selected_ids[0]]

```

```

        if selected_ids and selected_ids[0]
< len(ANSWER_LETTERS)
            else None
        )

    mark_question_used(
        session["user_id"],
        question_id,
        selected_answer=selected,
        correct=(selected ==
correct_answer),
    )

    if selected == correct_answer:

        session["correct"] += 1
        session["score"] += 1

        update_user_stats(
            session["user_id"],
            correct=True,
            score_delta=1
        )

        result_text = "🎉 सही उत्तर! बधाई
हो।"

    else:

        session["wrong"] += 1

        negative_mark = get_negative_mark()

        session["score"] -= negative_mark

```



```
update_user_stats(
    session["user_id"],
    correct=False,
    score_delta=-negative_mark
)

result_text = (
    "❌ गलत उत्तर!\n"
    f"सही उत्तर: {correct_answer}"
)

explanation = (
    question.get(
        "explanation",
        ""
    )
    or ""
).strip()

response = result_text

if explanation:

    response += (
        f"\n\nव्याख्या:\n"
        f"{explanation}"
    )

chat_id = poll_info["chat_id"]

try:

    await context.bot.send_message(
```

```

        chat_id=chat_id,
        text=response
    )

except Exception:
    logger.exception(
        "Answer response failed"
    )

ACTIVE_POLLS.pop(
    poll_id,
    None
)

session["index"] += 1

await advance_quiz(
    context,
    session_id,
    chat_id
)

#
=====
=====
# STOP QUIZ CALLBACK
#
=====
=====

async def stop_quiz_callback(
    update,
    context

```

```
):  
  
    query = update.callback_query  
  
    await query.answer()  
  
    data = query.data or ""  
  
    parts = data.split(":")  
  
    if len(parts) != 2:  
        return  
  
    session_id = parts[1]  
  
    session = get_quiz_session(  
        session_id  
    )  
  
    if not session:  
        return  
  
    if query.from_user.id !=  
session["user_id"]:  
  
        await query.answer(  
            "यह quiz आपका नहीं है।",  
            show_alert=True  
        )  
  
        return  
  
    poll_id = session.get(  
        "current_poll_id"
```

```

    )

    if poll_id:

        ACTIVE_POLLS.pop(
            poll_id,
            None
        )

        try:

            await context.bot.stop_poll(
chat_id=query.message.chat_id,
message_id=query.message.message_id
            )

        except Exception:
            pass

        remove_quiz_session(
            session_id
        )

        await query.message.reply_text(
            "Quiz रोक दिया गया।"
        )

#
=====
=====
# BOOKMARK CALLBACK

```

```

#
=====

=====

async def bookmark_callback(
    update,
    context
):

    query = update.callback_query

    data = query.data or ""

    parts = data.split(":")

    if len(parts) != 2:
        await query.answer()
        return

    try:
        question_id = int(parts[1])
    except Exception:
        await query.answer()
        return

    user_id = query.from_user.id

    ensure_user(query.from_user)

    added = add_bookmark(
        user_id,
        question_id
    )

```

```

        await query.answer(
            "🚫 Question save हो गया।  

/mybookmarks से देखें।"
            if added else
            "यह question पहले से saved है।",
            show_alert=False
        )

#
=====
=====
# FEEDBACK / REPORT CALLBACK
#
=====
=====

async def feedback_callback(
    update,
    context
):

    query = update.callback_query

    data = query.data or ""

    parts = data.split(":")

    if len(parts) != 2:
        await query.answer()
        return

    try:
        question_id = int(parts[1])

```

```

except Exception:
    await query.answer()
    return

user_id = query.from_user.id

ensure_user(query.from_user)

add_feedback(
    user_id,
    question_id
)

await query.answer(
    "⚠️ Report दर्ज हो गई। धन्यवाद!",
    show_alert=True
)

#
=====
=====
# START COMMAND
#
=====
=====

async def start_command(
    update,
    context
):

    user = update.effective_user

```

```

ensure_user(user)

# Referral link handling: /start
ref_<user_id>
args = context.args

if args and args[0].startswith("ref_"):

    try:
        referrer_id = int(
            args[0][4:]
        )
        set_referrer(
            user.id,
            referrer_id
        )
    except Exception:
        pass

admin_text = ""

if is_admin(user.id):
    admin_text = (
        "\n\nAdmin commands भी
available हैं।"
    )

await update.message.reply_text(
    "Quiz Bot चालू है!\n\n"
    "Commands:\n"
    "/quiz - Quiz शुरू करें\n"
    "/stats - अपनी stats देखें\n"
    "/newquiz - नया quiz बनाएं\n"

```



```

        "/quizid ID - Saved quiz शुरू करें\n"
        "/dashboard - Detailed dashboard
देखें\n"
        "/leaderboard - Top users देखें\n"
        "/mybookmarks - Saved questions
देखें\n"
        "/mymistakes - गलत questions दोबारा
देखें\n"
        "/revision - सिर्फ गलत questions से
quiz\n"
        "/weaktopics - अपने weak topics
जानें\n"
        "/myreferrals - अपना referral link
पाएं\n"
        "/help - सभी commands देखें"
        + admin_text
    )

```

```

#
=====
=====
# STATS COMMAND
#
=====
=====

async def stats_command(
    update,
    context
):

    user = update.effective_user

```

```

ensure_user(user)

stats = get_stats(
    user.id
)

if not stats:
    await update.message.reply_text(
        "अभी आपकी कोई stats उपलब्ध नहीं
        है।"
    )
    return

answered =
stats.get("questions_answered", 0) or 0
    correct = stats.get("correct_answers",
0) or 0
    wrong = stats.get("wrong_answers", 0)
or 0
    score = stats.get("score", 0) or 0
    quizzes =
stats.get("quizzes_completed", 0) or 0
    streak = stats.get("current_streak", 0)
or 0
    max_streak = stats.get("max_streak", 0)
or 0

accuracy = (
    (correct / answered) * 100
    if answered else 0

```

```

    )

    await update.message.reply_text(
        "आपकी Quiz Stats\n\n"
        f"कुल Quiz: {quizzes}\n"
        f"कुल Questions: {answered}\n"
        f"सही: {correct}\n"
        f"गलत: {wrong}\n"
        f"Accuracy: {accuracy:.1f}%\n"
        f"स्कोर: {score:.2f}\n"
        f"🔥 Streak: {streak} दिन (सबसे
लंबा: {max_streak})"
    )

#
=====
=====
# MY BOOKMARKS
#
=====
=====

async def mybookmarks_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)

    rows = list_bookmarks(
        user.id,

```

```

        limit=10
    )

    if not rows:
        await update.message.reply_text(
            "आपने अभी तक कोई question save
नहीं किया।\n\n"
            "Quiz खेलते समय 🚫 Save button
दबाकर "
            "questions save कर सकते हैं।"
        )
    return

    lines = ["🚫 आपके Saved Questions:\n"]

    for i, q in enumerate(rows, 1):
        lines.append(
            f"{i}. {q['question'][:80]}\n"
            f"    उत्तर: {q['answer']}"
        )

    await update.message.reply_text(
        "\n\n".join(lines)
    )

#
=====
=====
# MY MISTAKES
#
=====
=====

```

```

async def mymistakes_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)

    rows = list_mistakes(
        user.id,
        limit=10
    )

    if not rows:
        await update.message.reply_text(
            "आपकी mistake notebook खाली है।
बढ़िया! 🎉"
        )
        return

    lines = ["📖 आपकी Mistake
Notebook:\n"]

    for i, q in enumerate(rows, 1):
        lines.append(
            f"{i}. {q['question'][:80]}\n"
            f"    सही उत्तर: {q['answer']}"
        )

    lines.append(
        "\n\n/revision भेजकर इन्हीं questions
से "
        "quiz शुरू करें।"
    )

```

```

    )

    await update.message.reply_text(
        "\n\n".join(lines)
    )

#
=====
=====
# WEAK TOPICS
#
=====
=====

async def weaktopics_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)

    rows = get_weak_topics(
        user.id,
        limit=5
    )

    if not rows:
        await update.message.reply_text(
            "अभी तक कोई weak topic नहीं
मिला। "
            "थोड़े और quiz खेलिए।"

```

```

    )
    return

    lines = ["📊 आपके Weak Topics:\n"]

    for i, r in enumerate(rows, 1):
        lines.append(
            f"{i}. {r['subject']} – "
            f"{r['wrong_count']} गलत"
        )

    await update.message.reply_text(
        "\n".join(lines)
    )

```

```

#
=====
=====
# REVISION MODE
#
=====
=====

async def revision_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)

    rows = list_mistakes(

```

```

        user.id,
        limit=50
    )

    if not rows:
        await update.message.reply_text(
            "आपकी mistake notebook खाली है,
"
            "revision के लिए कोई question
नहीं है।"
        )
        return

    questions = [
        dict(r) for r in rows
    ]

    session_id = create_quiz_session(
        user_id=user.id,
        questions=questions
    )

    await update.message.reply_text(
        f"📖 Revision Quiz शुरू हो रहा है "
        f"({len(questions)} questions)..."
    )

    await send_quiz_question(
        context,
        update.effective_chat.id,
        session_id
    )

```



```

#
=====
=====
# LEADERBOARD
#
=====
=====

async def leaderboard_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)

    rows = get_leaderboard(
        limit=10
    )

    if not rows:
        await update.message.reply_text(
            "अभी तक leaderboard खाली है।"
        )
        return

    lines = ["🏆 Top 10 Leaderboard:\n"]

    medals = ["🥇", "🥈", "🥉"]

    for i, r in enumerate(rows, 1):
        name = (
            r.get("first_name")

```

```

        or r.get("username")
        or f"User {r['id']}")
    )

    prefix = (
        medals[i - 1]
        if i <= 3 else f"{i}."
    )

    lines.append(
        f"{prefix} {name} – "
        f"Score: {r['score']:.1f} "
        f"({r['correct_answers']} सही)"
    )

    await update.message.reply_text(
        "\n".join(lines)
    )

#
=====
=====
# PERSONAL DASHBOARD
#
=====
=====

async def dashboard_command(
    update,
    context
):

    user = update.effective_user

```

```

ensure_user(user)

stats = get_stats(user.id)

if not stats:
    await update.message.reply_text(
        "अभी आपका कोई data उपलब्ध नहीं है।"
    )
    return

answered = stats.get(
    "questions_answered", 0
) or 0

correct = stats.get(
    "correct_answers", 0
) or 0

wrong = stats.get(
    "wrong_answers", 0
) or 0

score = stats.get("score", 0) or 0

quizzes = stats.get(
    "quizzes_completed", 0
) or 0

streak = stats.get(
    "current_streak", 0
) or 0

```

```

max_streak = stats.get(
    "max_streak", 0
) or 0

referrals = stats.get(
    "referral_count", 0
) or 0

accuracy = (
    (correct / answered) * 100
    if answered else 0
)

badges = get_user_badges(user.id)

weak = get_weak_topics(
    user.id,
    limit=3
)

text = (
    "📊 आपका Dashboard\n\n"
    f"कुल Quiz: {quizzes}\n"
    f"कुल Questions: {answered}\n"
    f"सही: {correct} | गलत: {wrong}\n"
    f"Accuracy: {accuracy:.1f}%\n"
    f"स्कोर: {score:.2f}\n"
    f"🔥 Streak: {streak} दिन "
    f"(सबसे लंबा: {max_streak})\n"
    f"👥 Referrals: {referrals}\n"
    f"🏆 Badges: {len(badges)}"
)

if weak:

```

```

        text += "\n\nWeak Topics:\n" +
"\n".join(
            f"• {w['subject']}"
            (w['wrong_count']} गलत)"
            for w in weak
        )

    await update.message.reply_text(text)

#
=====
=====
# MY REFERRALS
#
=====
=====

async def myreferrals_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)

    stats = get_stats(user.id) or {}

    referrals = stats.get(
        "referral_count", 0
    ) or 0

    bot_username = context.bot.username

```

```

link = (
    f"https://t.me/{bot_username}"
    f"?start=ref_{user.id}"
    if bot_username else
    f"(bot username पता नहीं चला, "
    f"आपकी referral id: {user.id})"
)

await update.message.reply_text(
    "👥 Referral Program\n\n"
    f"अभी तक आपने {referrals} लोगों को
invite किया है\n\n"
    f"अपना referral link शेयर
करें:\n{link}"
)

#
=====
=====
# EXAM MODE
#
=====
=====

async def exam_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)

```

```

args = context.args

if not args:
    await update.message.reply_text(
        "किस exam का mock test चाहिए?"
        "\n\n"
        "उदाहरण: /exam RAS"
    )
    return

exam_name = " ".join(args)

count = min(
    50,
    MAX_IMPORT_QUESTIONS
)

questions = get_quiz_questions(
    user_id=user.id,
    count=count,
    exam=exam_name
)

if not questions:
    await update.message.reply_text(
        f"'{exam_name}' exam के लिए कोई
        "
        f"unseen questions उपलब्ध नहीं
        हैं!"
    )
    return

session_id = create_quiz_session(

```

```

        user_id=user.id,
        questions=questions
    )

    await update.message.reply_text(
        f"📝 {exam_name} Mock Test शुरू हो
रहा है "
        f"({len(questions)} questions)..."
    )

    await send_quiz_question(
        context,
        update.effective_chat.id,
        session_id
    )

#
=====
=====
# HELP COMMAND
#
=====
=====

async def help_command(
    update,
    context
):

    text = ""
    Quiz Bot Commands

    User Commands:

```


/start

Bot status और basic menu

/quiz

Random quiz शुरू करें

/quiz 20

20 questions का quiz

/stats

अपनी performance देखें

/dashboard

पूरा dashboard देखें (streak, badges, weak topics)

/leaderboard

Top 10 users देखें

/mybookmarks

Saved (🔖) questions देखें

/mymistakes

गलत किए गए questions देखें

/revision

सिर्फ गलत questions से quiz

/weaktopics

अपने कमज़ोर topics जानें

/exam NAME

उस exam का mock test

`/myreferrals`

अपना referral link पाएं

`/quizid ID`

Saved quiz शुरू करें

`/newquiz`

नया quiz बनाएं

`/help`

Commands की पूरी list

Admin Commands:

`/add`

नया question add करें

`/edit ID`

Question edit करें

`/delete ID`

Question delete करें

`/generate 10 Topic`

AI से questions बनाएं

`/autoquiz [COUNT] TOPIC`

SerpAPI web-search + AI से quiz अपने आप
बनाकर शुरू करें

`/newsquiz [COUNT] TOPIC`

SerpAPI Google News से current-affairs quiz
अपने आप बनाकर शुरू करें

/wikiquiz [COUNT] TOPIC
Wikipedia से factual/history/polity quiz
अपने आप बनाकर शुरू करें

/websearch QUERY
Quiz बनाए बिना सिर्फ SerpAPI results देखें

/scrape SOURCE [COUNT]
Saved website source (rajras/rbse/ncert
आदि) से quiz बनाकर शुरू करें

/speak TEXT
Text को voice note में बदलें (Google TTS)

/addexamlink EXAM_NAME LINK
Exam के लिए free quiz-website link save करें

/addsubjectlink SUBJECT_NAME LINK
Subject के लिए free quiz-website link save
करें

/examquiz [COUNT] EXAM_NAME
Saved exam links से authentic quiz अपने आप
बनाकर शुरू करें

/subjectquiz [COUNT] SUBJECT_NAME
Saved subject links से authentic quiz अपने
आप बनाकर शुरू करें

/examlist, /subjectlist
Saved exams/subjects की list देखें

/examlinks EXAM_NAME, /subjectlinks

SUBJECT_NAME

किसी exam/subject के saved links देखें

/scrape SOURCE

Website से content लेकर questions बनाएं

/pdfimport

PDF से questions import करें

/txtimport

TXT से questions import करें

/pdfinfo

PDF information

/htmlinfo

HTML/source information

/htmlreport

Source report

/poll2q

Telegram quiz poll को question में बदलें

/scrapepoll

Forward किए गए polls process करें

/clone ID

Quiz clone करें

/queue

Clone queue देखें

/negmark 0.25

Negative marking सेट करें

/resetpenalty

Default negative marking लगाएं

/stop

Active operation रोकें

/cancel

Pending operation cancel करें

"""

```
        await update.message.reply_text(
            text.strip()
        )
```

#

=====

=====

ADMIN CHECK

#

=====

=====

```
async def require_admin(
    update
):
```

```
    user = update.effective_user
```

```
    if not user:
        return False
```

```

        if not is_admin(user.id):

            if update.effective_message:

                await
                update.effective_message.reply_text(
                    "यह command केवल admin के
लिए है।"
                )

            return False

        return True

#
=====
=====
# /ADD
#
=====
=====

async def add_command(
    update,
    context
):

    if not await require_admin(update):
        return

    user_id = update.effective_user.id

    PENDING[user_id] = {

```

```

        "action": "add"
    }

    await update.message.reply_text(
        "Question इस format में भेजें:\n\n"
        "Question\n"
        "A) Option A\n"
        "B) Option B\n"
        "C) Option C\n"
        "D) Option D\n"
        "Answer: B\n"
        "Explanation: Explanation\n"
        "Exam: CET\n"
        "Subject: Polity\n\n"
        "Cancel करने के लिए /cancel भेजें।"
    )

```

```

#
=====
=====
# /EDIT
#
=====
=====

```

```

async def edit_command(
    update,
    context
):

    if not await require_admin(update):
        return

```

```
if not context.args:

    await update.message.reply_text(
        "Usage:\n"
        "/edit QUESTION_ID"
    )

    return

try:

    question_id = int(
        context.args[0]
    )

except Exception:

    await update.message.reply_text(
        "Invalid Question ID."
    )

    return

question = get_question(
    question_id
)

if not question:

    await update.message.reply_text(
        "Question नहीं मिला।"
    )

    return
```



```

user_id = update.effective_user.id

PENDING[user_id] = {
    "action": "edit",
    "question_id": question_id,
}

await update.message.reply_text(
    "Question का नया data भेजें:\n\n"
    "Question\n"
    "A) Option A\n"
    "B) Option B\n"
    "C) Option C\n"
    "D) Option D\n"
    "Answer: B\n"
    "Explanation: Explanation\n"
    "Exam: CET\n"
    "Subject: Polity\n\n"
    "Cancel: /cancel"
)

#
=====
=====
# /DELETE
#
=====
=====

async def delete_command(
    update,
    context

```

```
):  
  
    if not await require_admin(update):  
        return  
  
    if not context.args:  
  
        await update.message.reply_text(  
            "Usage:\n"  
            "/delete QUESTION_ID"  
        )  
  
        return  
  
    try:  
  
        question_id = int(  
            context.args[0]  
        )  
  
    except Exception:  
  
        await update.message.reply_text(  
            "Invalid Question ID."  
        )  
  
        return  
  
    question = get_question(  
        question_id  
    )  
  
    if not question:
```

```

        await update.message.reply_text(
            "Question नहीं मिला।"
        )

        return

    success = delete_question(
        question_id
    )

    if success:

        await update.message.reply_text(
            f"Question #{question_id}
delete कर दिया गया।"
        )

    else:

        await update.message.reply_text(
            "Question delete नहीं हो सका।"
        )

#
=====
=====
# PENDING TEXT HANDLER
#
=====
=====

async def pending_text_handler(
    update,

```

```

context
):

    user = update.effective_user

    if not user:
        return

    user_id = user.id

    pending = PENDING.get(
        user_id
    )

    if not pending:
        return

    if not is_admin(user_id):
        return

    text = (
        update.message.text
        or ""
    ).strip()

    if not text:
        return

    # -----
    -----
    # AUTO QUIZ (PDF/TXT/HTML भेजने के बाद
    questions की संख्या)
    # -----
    -----

```

```

        if pending.get("action") ==
"auto_quiz_count":
            # Kept only for old pending state
            after a bot restart/deployment.
            # New document uploads never ask
            for a question count.
            await update.message.reply_text(
                "इस file import को automatic
                mode में चलाने के लिए file दोबारा भेजें\n"
                "अब questions की संख्या पूरी नहीं
                जाएगी।"
            )
            return

        data = parse_question_text(
            text
        )

        if not data:

            await update.message.reply_text(
                "Question format सही नहीं है\n\n"
                "Example:\n"
                "भारत का संविधान कब लागू हुआ?\n"
                "A) 1947\n"
                "B) 1950\n"
                "C) 1952\n"
                "D) 1949\n"
                "Answer: B\n"
                "Explanation: संविधान 26 जनवरी
                1950 को लागू हुआ\n"
                "Exam: CET\n"
            )

```

```

        "Subject: Polity"
    )

    return

    action = pending.get(
        "action"
    )

    if action == "add":

        question_id, status = add_question(
            data
        )

        PENDING.pop(
            user_id,
            None
        )

        if question_id and status ==
"added":

            await
update.message.reply_text(
                "Question successfully add
हो गया!\n\n"
                f"Question ID:
{question_id}"
            )

            elif question_id and status ==
"duplicate":

```

```
        await
update.message.reply_text(
    "यह question पहले से
database में मौजूद है!\n\n"
    f"Existing Question ID:
{question_id}")
    )

    else:

        await
update.message.reply_text(
    "Question add नहीं हो सका।"
    )

    return

if action == "edit":

    question_id = pending.get(
        "question_id"
    )

    if not question_id:

        PENDING.pop(
            user_id,
            None
        )

        return

    success = update_question(
        question_id,
```

```

        data
    )

    if success:

        PENDING.pop(
            user_id,
            None
        )

        await
update.message.reply_text(
            f"Question #{question_id} "
            "successfully update हो
गया।"
        )

    else:

        await
update.message.reply_text(
            "Question update नहीं हो
सका।"
        )

#
=====
=====
# /CANCEL
#
=====
=====

```



```

async def cancel_command(
    update,
    context
):

    user_id = update.effective_user.id

    removed = False

    if user_id in PENDING:

        PENDING.pop(
            user_id,
            None
        )

        removed = True

    if user_id in QUIZ_CREATE:

        QUIZ_CREATE.pop(
            user_id,
            None
        )

        removed = True

    if removed:

        await update.message.reply_text(
            "Current operation cancel कर
            दिया गया।"
        )

```

```

else:

    await update.message.reply_text(
        "कोई pending operation नहीं है।"
    )
    #
=====
=====
# NEW QUIZ COMMAND
#
=====
=====

async def newquiz_command(
    update,
    context
):

    user = update.effective_user

    ensure_user(user)

    # पुराने quiz creation state को साफ करें
    QUIZ_CREATE.pop(
        user.id,
        None
    )

    QUIZ_CREATE[
        user.id

```

```

] = {
    "step": "title"
}

await update.message.reply_text(
    "नया Quiz बनाने के लिए जानकारी दें\\n\\n"
    "पहले Quiz का नाम भेजें\\n\\n"
    "Cancel: /cancel"
)

```

```

#
=====
=====
# NEW QUIZ TEXT FLOW
#
=====
=====

```

```

async def newquiz_text_handler(
    update,
    context
):

    user = update.effective_user

    if not user:
        return

    user_id = user.id

    state = QUIZ_CREATE.get(
        user_id
    )

```

```
if not state:
    return

text = (
    update.message.text
    or ""
).strip()

if not text:
    return

step = state.get(
    "step"
)

# -----
# TITLE
# -----

if step == "title":

    state["title"] = text
    state["step"] = "exam"

    await update.message.reply_text(
        "Exam का नाम भेजें\\n\\n"
        "Example: CET\\n\\n"
        "अगर नहीं देना है तो - भेजें।"
    )

    return
```

```

# -----
-----
# EXAM
# -----
-----

if step == "exam":

    state["exam"] = (
        ""
        if text == "-"
        else text
    )

    state["step"] = "subject"

    await update.message.reply_text(
        "Subject भेजें\\n\\n"
        "Example: Polity\\n\\n"
        "अगर नहीं देना है तो - भेजें।"
    )

    return

# -----
-----
# SUBJECT
# -----
-----

if step == "subject":

    state["subject"] = (

```

```

        """
        if text == "-":
            else text
        )

        state["step"] = "count"

        await update.message.reply_text(
            "कितने questions चाहिए?\n\n"
            "Example: 10"
        )

        return

# -----
# COUNT
# -----

if step == "count":

    try:

        count = int(text)

    except Exception:

        await
update.message.reply_text(
            "कृपया केवल number भेजें!\n"
            "Example: 10"
        )

```

```

        return

    if count < 1:

        await
        update.message.reply_text(
            "Questions की संख्या कम से
            कम 1 होनी चाहिए।"
        )

        return

    # Quiz question count अब unlimited
    है - सिर्फ न्यूनतम 1 जरूरी है
    state["count"] = count

    title = state["title"]
    exam = state.get("exam") or None
    subject = state.get("subject") or
    None

    await update.message.reply_text(
        "Quiz बनाया जा रहा है...\n"
        "कृपया प्रतीक्षा करें।"
    )

    try:

        quiz_id, questions =
        create_random_quiz(
            user_id=user_id,
            title=title,
            count=count,
            exam=exam,

```

```

        subject=subject,
    )

    except Exception:

        logger.exception(
            "New quiz creation failed"
        )

        await
update.message.reply_text(
    "Quiz create करते समय error
आया।"
)

    QUIZ_CREATE.pop(
        user_id,
        None
    )

    return

    QUIZ_CREATE.pop(
        user_id,
        None
    )

    if not quiz_id:

        await
update.message.reply_text(
    "Quiz create नहीं हो सका।
\n\n"
    "हो सकता है आपके लिए "

```



```

        "unseen questions उपलब्ध नहीं
        हैं।"
    )

    return

    await update.message.reply_text(
        "Quiz successfully create हो
        गया।\n\n"
        f"Quiz ID: {quiz_id}\n"
        f"Title: {title}\n"
        f"Questions:
        {len(questions)}\n\n"
        f"Start करने के लिए:\n"
        f"/quizid {quiz_id}"
    )

#
=====
=====
# NEGATIVE MARKING
#
=====
=====

async def negmark_command(
    update,
    context
):

    if not await require_admin(update):
        return

```

```

if not context.args:

    current = get_negative_mark()

    await update.message.reply_text(
        f"Current negative marking:
{current}\n\n"
        "Set करने के लिए:\n"
        "/negmark 0.25"
    )

    return

try:

    value = float(
        context.args[0]
    )

except Exception:

    await update.message.reply_text(
        "Invalid value.\n\n"
        "Example:\n"
        "/negmark 0.25"
    )

    return

if value < 0:

    await update.message.reply_text(
        "Negative marking 0 से कम नहीं हो
सकती।"

```

```

    )

    return

    if value > 10:

        await update.message.reply_text(
            "Value बहुत ज्यादा है।"
        )

        return

    set_setting(
        "negative_mark",
        str(value)
    )

    await update.message.reply_text(
        f"Negative marking set कर दी गई:
{value}"
    )

#
=====
=====
# RESET NEGATIVE MARKING
#
=====
=====

async def resetpenalty_command(
    update,
    context

```

```

):

    if not await require_admin(update):
        return

    set_setting(
        "negative_mark",
        str(DEFAULT_NEGATIVE_MARK)
    )

    await update.message.reply_text(
        "Negative marking reset हो गई!\n\n"
        f"Current value:
{DEFAULT_NEGATIVE_MARK}"
    )

#
=====
=====
# QUIZ TIMER (PER QUESTION)
#
=====
=====

async def quiztime_command(
    update,
    context
):

    if not await require_admin(update):
        return

    if not context.args:

```

```

        current = get_quiz_time()

        await update.message.reply_text(
            f"Current quiz timer: {current}
seconds\n\n"
            "Set करने के लिए:\n"
            "/quiztime 30\n\n"
            f"Allowed range:
{MIN_QUIZ_TIME_SECONDS}-"
            f"{MAX_QUIZ_TIME_SECONDS} सेकंड"
        )

        return

    try:

        value = int(
            context.args[0]
        )

    except Exception:

        await update.message.reply_text(
            "Invalid value.\n\n"
            "Example:\n"
            "/quiztime 30"
        )

        return

    value = clamp_quiz_time(
        value
    )

```

```

        set_setting(
            "quiz_time_seconds",
            str(value)
        )

        await update.message.reply_text(
            f"Quiz timer set कर दिया गया:
{value} सेकंड"
        )

    async def resetquiztime_command(
        update,
        context
    ):

        if not await require_admin(update):
            return

        set_setting(
            "quiz_time_seconds",
            str(DEFAULT_QUIZ_TIME_SECONDS)
        )

        await update.message.reply_text(
            "Quiz timer reset हो गया!\n\n"
            f"Current value:
{DEFAULT_QUIZ_TIME_SECONDS} सेकंड"
        )

#
=====

```

```

=====
# RESET USER QUIZ HISTORY
#
=====
=====

async def resethistory_command(
    update,
    context
):

    if not await require_admin(update):
        return

    if context.args:

        try:
            target_id =
int(context.args[0])
        except Exception:

            await
update.message.reply_text(
            "Invalid user id.\n\n"
            "Usage:\n"
            "/resethistory (अपने
लिए)\n"
            "/resethistory USER_ID
(किसी और के लिए)"
            )

        return

    else:

```

```

        target_id =
update.effective_user.id

        reset_user_history(
            target_id
        )

        await update.message.reply_text(
            f"Quiz history reset कर दी गई
(user: {target_id})\n\n"
            "अब सभी questions दोबारा उपलब्ध हैं।"
        )

#
=====
=====
# STOP COMMAND
#
=====
=====

async def stop_command(
    update,
    context
):

    user_id = update.effective_user.id

    stopped = []

    # Pending operations
    if user_id in PENDING:

```



```
PENDING.pop(  
    user_id,  
    None  
)  
  
stopped.append(  
    "pending operation"  
)  
  
# Quiz creation  
if user_id in QUIZ_CREATE:  
  
    QUIZ_CREATE.pop(  
        user_id,  
        None  
    )  
  
    stopped.append(  
        "quiz creation"  
    )  
  
# Active quiz sessions  
session_ids = []  
  
for session_id, session in list(  
    ACTIVE QUIZZES.items()  
):  
  
    if session.get(  
        "user_id"  
    ) == user_id:  
  
        session_ids.append(  
            session_id
```

```

    )

    for session_id in session_ids:

        remove_quiz_session(
            session_id
        )

        stopped.append(
            "active quiz"
        )

    if stopped:

        await update.message.reply_text(
            "Current operations stop कर दी
गई!"
        )

    else:

        await update.message.reply_text(
            "आपकी कोई active operation नहीं
है!"
        )

#
=====
=====
# CLONE QUEUE STATUS
#
=====
=====

```

```
async def queue_command(
    update,
    context
):

    if not await require_admin(update):
        return

    with CLONE_LOCK:

        queue_copy = list(
            CLONE_QUEUE
        )

        if not queue_copy:

            await update.message.reply_text(
                "Clone queue खाली है।"
            )

            return

        lines = [
            "Clone Queue",
            ""
        ]

        for index, item in enumerate(
            queue_copy,
            start=1
        ):

            lines.append(
```

```

        f"{index}. "
        f"Quiz ID: {item.get('quiz_id',
'-')}}"
    )

    lines.append(
        f"User ID: {item.get('user_id',
'-')}}"
    )

    lines.append(
        f"Status: {item.get('status',
'queued')}}"
    )

    lines.append("")

    await update.message.reply_text(
        "\n".join(lines)
    )
    #
    =====
    =====
    # AI GENERATE COMMAND
    #
    =====
    =====

    async def generate_command(update,
context):

        if not await require_admin(update):
            return

```

```

        if not context.args:
            await update.message.reply_text(
                "Usage:\n\n"
                "/generate 10 Indian
Polity\n\n"
                "Example:\n"
                "/generate 20 Rajasthan
History"
            )
            return

        try:
            count = int(context.args[0])
        except Exception:
            await update.message.reply_text(
                "पहला argument questions की
संख्या होना चाहिए|\n\n"
                "Example:\n"
                "/generate 10 Indian Polity"
            )
            return

        if count < 1:
            await update.message.reply_text(
                "Questions की संख्या कम से कम 1
होनी चाहिए|"
            )
            return

        count = min(count,
MAX_IMPORT_QUESTIONS)

        topic = " ".join(
            context.args[1:]

```

```

).strip()

if not topic:
    await update.message.reply_text(
        "Topic देना जरूरी है!\n\n"
        "Example:\n"
        "/generate 10 Indian Polity"
    )
    return

    await update.message.reply_text(
        f"AI {count} questions generate कर
रहा है...\n\n"
        f"Topic: {topic}"
    )

    try:

        questions =
groq_generate_questions(
            topic=topic,
            count=count
        )

    except Exception as e:

        logger.exception(
            "AI generation failed"
        )

        await update.message.reply_text(
            "AI question generation में error
आया!\n\n"
            f"Error: {str(e)[:500]}"

```

```

    )

    return

    if not questions:

        await update.message.reply_text(
            "AI से कोई valid question प्राप्त
नहीं हुआ।"
        )

    return

    added = 0
    duplicate = 0
    failed = 0
    new_ids = []

    for question in questions:

        try:

            question_id, status =
add_question(
                question
            )

            if question_id and status ==
"added":
                added += 1
                new_ids.append(question_id)
            elif question_id and status ==
"duplicate":
                duplicate += 1

```

```

        else:
            failed += 1

    except Exception:

        logger.exception(
            "Question save failed"
        )

        failed += 1

    summary_lines = [
        "AI Generation Complete",
        "",
        f"Requested: {count}",
        f"Generated: {len(questions)}",
        f"Added: {added}",
        f"Duplicate: {duplicate}",
        f"Failed: {failed}",
    ]

    if new_ids:

        quiz_id =
        create_quiz_from_question_ids(
            user_id=update.effective_user.id,
            title=f"AI: {topic}"[:100],
            question_ids=new_ids,
        )

        if quiz_id:

            summary_lines += [

```



```

        "",
        f"Quiz बन गया (Quiz ID:
{quiz_id})",
        f"Start करने के लिए:",
        f"/quizid {quiz_id}",
    ]

    await update.message.reply_text(
        "\n".join(summary_lines)
    )

#
=====
=====
# AUTOQUIZ COMMAND (SERPAPI WEB SEARCH + AI
GENERATION)
#
=====
=====

async def autoquiz_command(update,
context):
    """
    /autoquiz [COUNT] TOPIC

    Command भेजते ही, बिना किसी और
    interaction के:
        1. SerpAPI से topic पर web search किया
        जाता है।
        2. Search results (+ कुछ pages) से
        context बनाया जाता है।
        3. उस context के आधार पर Groq से MCQ
        questions बनते हैं।

```

4. Questions DB में save होकर quiz बनता है।

5. Quiz session तुरंत शुरू हो जाता है (पहला question auto-भेजा जाता है)।

```
"""

if not await require_admin(update):
    return

if not SERPAPI_API_KEY:
    await update.message.reply_text(
        "SERPAPI_API_KEY environment
variable set नहीं है!\n"
        "Web-search के बिना यह command
काम नहीं करेगा!\n\n"
        "इसके बजाय आप /generate COUNT
TOPIC इस्तेमाल कर सकते हैं।"
    )
    return

if groq_client is None:
    await update.message.reply_text(
        "GROQ_API_KEY configured नहीं है,
इसलिए questions generate नहीं हो सकते।"
    )
    return

if not context.args:
    await update.message.reply_text(
        "Usage:\n\n"
        "/autoquiz [COUNT] TOPIC\n\n"
        "Examples:\n"
        "/autoquiz Rajasthan Current
Affairs\n"
```

```

        "/autoquiz 15 Indian
        Polity\n\n"
        "COUNT न देने पर डिफ़ॉल्ट questions
        बनेंगे।"
    )
    return

    args = list(context.args)

    count = DEFAULT_QUIZ_COUNT

    if args and args[0].isdigit():
        count = int(args[0])
        args = args[1:]

    count = max(
        1,
        min(count, MAX_IMPORT_QUESTIONS)
    )

    topic = " ".join(args).strip()

    if not topic:
        await update.message.reply_text(
            "Topic देना जरूरी है!\n\n"
            "Example:\n"
            "/autoquiz 10 Rajasthan GK"
        )
        return

    await update.message.reply_text(
        f"🔍 SerpAPI से \"{topic}\" पर web
        search किया जा रहा है...\n"
        f"इसके बाद AI {count} questions

```

```

generate करेगा और quiz अपने आप शुरू हो जाएगा।
\n"
        "कृपया प्रतीक्षा करें..."
    )

    try:

        context_text, sources = await
asyncio.to_thread(
            build_context_from_serpapi,
            topic,
        )

    except Exception as e:

        logger.exception(
            "SerpAPI search failed for
autoquiz: %s",
            topic
        )

        await update.message.reply_text(
            "Web search में error आया!\n\n"
            f"Error: {str(e)[:500]}"
        )

    return

    if not context_text:
        await update.message.reply_text(
            "इस topic के लिए कोई useful web
result नहीं मिला!\n"
            "कृपया topic थोड़ा अलग तरीके से
लिखकर दोबारा try करें।"

```

```

    )
    return

    try:

        questions = await
    asyncio.to_thread(
        groq_generate_questions,
        topic,
        count,
        context_text,
        "serpapi",
    )

    except Exception as e:

        logger.exception(
            "AI generation from SerpAPI
context failed: %s",
            topic
        )

        await update.message.reply_text(
            "AI question generation में error
आया।\n\n"
            f"Error: {str(e)[:500]}"
        )

        return

    if not questions:
        await update.message.reply_text(
            "AI से कोई valid question प्राप्त
नहीं हुआ।\n"

```

```

        "कृपया दोबारा try करें या topic
        बदलें।"
    )
    return

    added = 0
    duplicate = 0
    failed = 0
    new_ids = []

    for question in questions:

        try:

            question_id, status =
add_question(
                question
            )

            if question_id and status ==
"added":
                added += 1
                new_ids.append(question_id)
            elif question_id and status ==
"duplicate":
                duplicate += 1
            else:
                failed += 1

        except Exception:

            logger.exception(
                "Question save failed
(autoquiz)"

```

```

    )

    failed += 1

    if not new_ids:
        await update.message.reply_text(
            "Questions generate हुए लेकिन  
सभी duplicate/invalid निकले।\n"
            f"Generated: {len(questions)} |  
Duplicate: {duplicate} | Failed: {failed}"
        )
        return

    quiz_id =
    create_quiz_from_question_ids(
        user_id=update.effective_user.id,
        title=f"AutoQuiz: {topic}"[:100],
        question_ids=new_ids,
    )

    if not quiz_id:
        await update.message.reply_text(
            "Questions बन गए, लेकिन Quiz  
create नहीं हो सका।"
        )
        return

    quiz_questions =
    get_saved_quiz_questions(quiz_id)

    if not quiz_questions:
        await update.message.reply_text(
            "Quiz में questions नहीं मिले।"
        )

```

```

        return

    session_id = create_quiz_session(
        user_id=update.effective_user.id,
        questions=quiz_questions,
        quiz_id=quiz_id,
    )

    source_note = ""

    if sources:
        top_sources = "\n".join(f"• {link}"
    for link in sources[:3])
        source_note = f"\n\nTop
sources:\n{top_sources}"

    await update.message.reply_text(
        "✅ AutoQuiz तैयार है!\n\n"
        f"Topic: {topic}\n"
        f"Generated: {len(questions)}\n"
        f"Added: {added}\n"
        f"Duplicate: {duplicate}\n"
        f"Failed: {failed}\n"
        f"Quiz ID: {quiz_id}"
        f"{source_note}\n\n"
        "पहला question शुरू हो रहा है..."
    )

    await send_quiz_question(
        context,
        update.effective_chat.id,
        session_id,
    )

```



```

#
=====
=====
# WEBSEARCH COMMAND (RAW SERPAPI RESULTS,
NO QUIZ)
#
=====
=====

async def websearch_command(update,
context):
    """
    /websearch QUERY

    Sirf SerpAPI (Google) results dikhata
    hai – koi quiz nahi banta!
    Ye check karne ke liye ki /autoquiz
    chalane se pehle topic par
    kaafi content मिल रहा है ya nahi!
    """

    if not await require_admin(update):
        return

    if not SERPAPI_API_KEY:
        await update.message.reply_text(
            "SERPAPI_API_KEY environment
variable set नहीं है।"
        )
        return

    if not context.args:
        await update.message.reply_text(

```

```

        "Usage:\n\n"
        "/websearch QUERY\n\n"
        "Example:\n"
        "/websearch Rajasthan GK 2026"
    )
    return

    query = " ".join(context.args).strip()

    await update.message.reply_text(
        f"🔍 \"{query}\" search किया जा रहा है..."
    )

    try:

        results = await asyncio.to_thread(
            serpapi_search,
            query,
        )

    except Exception as e:

        logger.exception(
            "SerpAPI websearch failed: %s",
            query
        )

        await update.message.reply_text(
            "Search में error आया!\n\n"
            f"Error: {str(e)[:500]}"
        )

    return

```

```

    if not results:
        await update.message.reply_text(
            "कोई result नहीं मिला।"
        )
        return

    lines = [f"🔍 Results: {query}\n"]

    for idx, item in enumerate(results,
start=1):

        lines.append(
            f"{idx}. {item['title']}\n"
            f"{item['snippet']}\n"
            f"{item.get('link', '')}\n"
        )

    text = "\n".join(lines)[:4000]

    await update.message.reply_text(text)

#
=====
=====
# NEWSQUIZ COMMAND (SERPAPI GOOGLE NEWS +
AI GENERATION)
#
=====
=====

async def newsquiz_command(update,
context):

```

```

"""
/newsquiz [COUNT] TOPIC

/autoquiz जैसा ही, लेकिन SerpAPI के
Google News engine से
latest current-affairs news उठाता है और
उसी पर current-affairs
quiz बनाकर तुरंत शुरू कर देता है।
"""

if not await require_admin(update):
    return

if not SERPAPI_API_KEY:
    await update.message.reply_text(
        "SERPAPI_API_KEY environment
variable set नहीं है।"
    )
    return

if groq_client is None:
    await update.message.reply_text(
        "GROQ_API_KEY configured नहीं है,
इसलिए questions generate नहीं हो सकते।"
    )
    return

if not context.args:
    await update.message.reply_text(
        "Usage:\n\n"
        "/newsquiz [COUNT] TOPIC\n\n"
        "Examples:\n"
        "/newsquiz Rajasthan Current
Affairs\n"

```

```

        "/newsquiz 15 India Current
Affairs September 2026"
    )
    return

    args = list(context.args)

    count = DEFAULT_QUIZ_COUNT

    if args and args[0].isdigit():
        count = int(args[0])
        args = args[1:]

    count = max(
        1,
        min(count, MAX_IMPORT_QUESTIONS)
    )

    topic = " ".join(args).strip()

    if not topic:
        await update.message.reply_text(
            "Topic देना जरूरी है!\n\n"
            "Example:\n"
            "/newsquiz 10 Rajasthan Current
Affairs"
        )
        return

    await update.message.reply_text(
        f"📰 SerpAPI Google News से \"{
{topic}\" की latest news लाई जा रही है...\n"
        f"इसके बाद AI {count} current-
affairs questions generate करेगा और quiz

```

```

अपने आप शुरू हो जाएगा।\n"
    "कृपया प्रतीक्षा करें..."
)

try:

    context_text, sources = await
asyncio.to_thread(
    build_context_from_serpapi,
    topic,
    None,
    None,
    "google_news",
)

except Exception as e:

    logger.exception(
        "SerpAPI news search failed for
newsquiz: %s",
        topic
    )

    await update.message.reply_text(
        "News search में error आया।\n\n"
        f"Error: {str(e)[:500]}"
    )

    return

if not context_text:
    await update.message.reply_text(
        "इस topic के लिए कोई recent
news नहीं मिली।\n"

```

```

        "कृपया topic थोड़ा अलग तरीके से
लिखकर दोबारा try करें।"
    )
    return

    try:

        questions = await
asyncio.to_thread(
            groq_generate_questions,
            topic,
            count,
            context_text,
            "serpapi_news",
        )

    except Exception as e:

        logger.exception(
            "AI generation from news
context failed: %s",
            topic
        )

        await update.message.reply_text(
            "AI question generation में error
आया!\n\n"
            f"Error: {str(e)[:500]}"
        )

    return

    if not questions:
        await update.message.reply_text(

```

```

        "AI से कोई valid question प्राप्त
        नहीं हुआ।\n"
        "कृपया दोबारा try करें या topic
        बदलें।"
    )
    return

    added = 0
    duplicate = 0
    failed = 0
    new_ids = []

    for question in questions:

        try:

            question_id, status =
add_question(
                question
            )

            if question_id and status ==
"added":
                added += 1
                new_ids.append(question_id)
            elif question_id and status ==
"duplicate":
                duplicate += 1
            else:
                failed += 1

        except Exception:

            logger.exception(

```



```

        "Question save failed
(newsquiz)"
    )

    failed += 1

    if not new_ids:
        await update.message.reply_text(
            "Questions generate हुए लेकिन
सभी duplicate/invalid निकले\\n"
            f"Generated: {len(questions)} |
Duplicate: {duplicate} | Failed: {failed}"
        )
        return

    quiz_id =
create_quiz_from_question_ids(
    user_id=update.effective_user.id,
    title=f"NewsQuiz: {topic}"[:100],
    question_ids=new_ids,
)

    if not quiz_id:
        await update.message.reply_text(
            "Questions बन गए, लेकिन Quiz
create नहीं हो सका।"
        )
        return

    quiz_questions =
get_saved_quiz_questions(quiz_id)

    if not quiz_questions:
        await update.message.reply_text(

```

```

        "Quiz में questions नहीं मिले।"
    )
    return

    session_id = create_quiz_session(
        user_id=update.effective_user.id,
        questions=quiz_questions,
        quiz_id=quiz_id,
    )

    source_note = ""

    if sources:
        top_sources = "\n".join(f"• {link}"
for link in sources[:3])
        source_note = f"\n\nTop news
sources:\n{top_sources}"

    await update.message.reply_text(
        "✅ NewsQuiz तैयार है!\n\n"
        f"Topic: {topic}\n"
        f"Generated: {len(questions)}\n"
        f"Added: {added}\n"
        f"Duplicate: {duplicate}\n"
        f"Failed: {failed}\n"
        f"Quiz ID: {quiz_id}"
        f"{source_note}\n\n"
        "पहला question शुरू हो रहा है..."
    )

    await send_quiz_question(
        context,
        update.effective_chat.id,
        session_id,

```

```

    )

#
=====
=====
# WIKIQUIZ COMMAND (WIKIPEDIA + AI
GENERATION)
#
=====
=====

async def wikiquiz_command(update,
context):
    """
    /wikiquiz [COUNT] TOPIC

    Wikipedia (free, no key) से topic पर
    summary उठाता है, उसी context
    पर Groq से MCQ बनाता है और quiz तुरंत शुरू
    कर देता है – /autoquiz की
    तरह ही, पर source Wikipedia है
    (encyclopedic/factual topics: history,
    polity, geography, science वगैरह के लिए
    ज्यादा भरोसेमंद)।
    """

    if not await require_admin(update):
        return

    if groq_client is None:
        await update.message.reply_text(
            "GROQ_API_KEY configured नहीं है,
            इसलिए questions generate नहीं हो सकते।"

```

```

    )
    return

if not context.args:
    await update.message.reply_text(
        "Usage:\n\n"
        "/wikiquiz [COUNT] TOPIC\n\n"
        "Examples:\n"
        "/wikiquiz Rajasthan History\n"
        "/wikiquiz 15 Indian
Constitution"
    )
    return

args = list(context.args)

count = DEFAULT_QUIZ_COUNT

if args and args[0].isdigit():
    count = int(args[0])
    args = args[1:]

count = max(
    1,
    min(count, MAX_IMPORT_QUESTIONS)
)

topic = " ".join(args).strip()

if not topic:
    await update.message.reply_text(
        "Topic देना जरूरी है!\n\n"
        "Example:\n"
        "/wikiquiz 10 Rajasthan

```

```

History"
    )
    return

    await update.message.reply_text(
        f"📖 Wikipedia से \"{topic}\" पर
जानकारी लाई जा रही है...\n"
        f"इसके बाद AI {count} questions
generate करेगा और quiz अपने आप शुरू हो जाएगा।
\n"
        "कृपया प्रतीक्षा करें..."
    )

    try:

        context_text, sources = await
asyncio.to_thread(
            build_context_from_wikipedia,
            topic,
        )

    except Exception as e:

        logger.exception(
            "Wikipedia search failed for
wikiquiz: %s",
            topic
        )

        await update.message.reply_text(
            "Wikipedia search में error आया।
\n\n"
            f"Error: {str(e)[:500]}"
        )

```

```

        return

    if not context_text:
        await update.message.reply_text(
            "इस topic के लिए Wikipedia पर  
कोई page नहीं मिला।\n"
            "कृपया topic थोड़ा अलग तरीके से  
लिखकर दोबारा try करें।"
        )
        return

    try:

        questions = await
        asyncio.to_thread(
            groq_generate_questions,
            topic,
            count,
            context_text,
            "wikipedia",
        )

    except Exception as e:

        logger.exception(
            "AI generation from Wikipedia  
context failed: %s",
            topic
        )

        await update.message.reply_text(
            "AI question generation में error  
आया।\n\n"

```

```

        f"Error: {str(e)[:500]}"
    )

    return

    if not questions:
        await update.message.reply_text(
            "AI से कोई valid question प्राप्त
नहीं हुआ।\n"
            "कृपया दोबारा try करें या topic
बदलें।"
        )
    return

    added = 0
    duplicate = 0
    failed = 0
    new_ids = []

    for question in questions:

        try:

            question_id, status =
add_question(
                question
            )

            if question_id and status ==
"added":
                added += 1
                new_ids.append(question_id)
            elif question_id and status ==
"duplicate":

```

```

        duplicate += 1
    else:
        failed += 1

    except Exception:

        logger.exception(
            "Question save failed
(wikiquiz)"
        )

        failed += 1

    if not new_ids:
        await update.message.reply_text(
            "Questions generate हुए लेकिन
सभी duplicate/invalid निकले\\n"
            f"Generated: {len(questions)} |
Duplicate: {duplicate} | Failed: {failed}"
        )
        return

    quiz_id =
create_quiz_from_question_ids(
        user_id=update.effective_user.id,
        title=f"WikiQuiz: {topic}"[:100],
        question_ids=new_ids,
    )

    if not quiz_id:
        await update.message.reply_text(
            "Questions बन गए, लेकिन Quiz
create नहीं हो सका।"
        )

```



```

        return

        quiz_questions =
get_saved_quiz_questions(quiz_id)

        if not quiz_questions:
            await update.message.reply_text(
                "Quiz में questions नहीं मिले।"
            )
            return

        session_id = create_quiz_session(
            user_id=update.effective_user.id,
            questions=quiz_questions,
            quiz_id=quiz_id,
        )

        source_note = ""

        if sources:
            top_sources = "\n".join(f"• {link}"
for link in sources[:3])
            source_note = f"\n\nWikipedia
sources:\n{top_sources}"

        await update.message.reply_text(
            "✅ WikiQuiz तैयार है!\n\n"
            f"Topic: {topic}\n"
            f"Generated: {len(questions)}\n"
            f"Added: {added}\n"
            f"Duplicate: {duplicate}\n"
            f"Failed: {failed}\n"
            f"Quiz ID: {quiz_id}"
            f"{source_note}\n\n"

```

```

        "पहला question शुरू हो रहा है..."
    )

    await send_quiz_question(
        context,
        update.effective_chat.id,
        session_id,
    )

#
=====
=====
# SPEAK COMMAND (GOOGLE CLOUD TTS)
#
=====
=====

async def speak_command(update, context):
    """
    /speak TEXT

    Google Cloud Text-to-Speech से दिया गया
    text एक voice note में बदलकर
    Telegram पर भेजता है। हिंदी exam-content
    सुनकर revise करने के लिए उपयोगी।
    """

    if not GOOGLE_TTS_API_KEY:
        await update.message.reply_text(
            "GOOGLE_TTS_API_KEY environment
            variable set नहीं है।"
        )
    return

```

```

    if not context.args:
        await update.message.reply_text(
            "Usage:\n\n"
            "/speak TEXT\n\n"
            "Example:\n"
            "/speak राजस्थान का सबसे बड़ा जिला
जैसलमेर है।"
        )
        return

    text = " ".join(context.args).strip()

    if len(text) > GOOGLE_TTS_CHAR_LIMIT:
        await update.message.reply_text(
            f"Text बहुत लंबा है (अधिकतम
{GOOGLE_TTS_CHAR_LIMIT} characters)। "
            "कृपया छोटा करके भेजें।"
        )
        return

    try:

        audio_bytes = await
        asyncio.to_thread(
            google_tts_synthesize,
            text,
        )

    except Exception as e:

        logger.exception(
            "Google TTS synthesis failed"
        )

```

```

        await update.message.reply_text(
            "Audio बनाने में error आया!\n\n"
            f"Error: {str(e)[:500]}"
        )

    return

from io import BytesIO

audio_file = BytesIO(audio_bytes)
audio_file.name = "speech.ogg"

await update.message.reply_voice(
    voice=audio_file,
    caption=text[:200],
)

#
=====
=====
# EXAM / SUBJECT LINK LIBRARY COMMANDS
#
=====
=====

def _split_topic_name_and_url(args):
    """
    Command args में से पहला http(s) link ढूंढता
    है – बाकी सारे words
    मिलकर topic name (exam/subject) बनता है।
    Example: ["RAS", "Pre", "2027",
    "https://x.com"] ->

```

```

        ("RAS Pre 2027",
"https://x.com")
        """

        url = None
        name_words = []

        for word in args:
            if url is None and
word.lower().startswith(("http://",
"https://")):
                url = word.strip()
            else:
                name_words.append(word)

        topic_name = "
".join(name_words).strip()

        return topic_name, url

async def addexamlink_command(update,
context):

    if not await require_admin(update):
        return

    if not context.args:
        await update.message.reply_text(
            "Usage:\n\n"
            "/addexamlink EXAM_NAME
LINK\n\n"
            "Example:\n"
            "/addexamlink RAS Pre 2027

```

```

https://example.com/ras-quiz"
    )
    return

    topic_name, url =
_split_topic_name_and_url(context.args)

    if not topic_name or not url:
        await update.message.reply_text(
            "Exam का नाम और एक http(s)
link दोनों देना जरूरी है।"
        )
        return

    existing = get_topic_sources("exam",
topic_name)

    if len(existing) >=
TOPIC_LINKS_MAX_SAVED:
        await update.message.reply_text(
            f"\{topic_name}\\" के लिए पहले से
{TOPIC_LINKS_MAX_SAVED} links save हैं, "
            "जो limit है।"
        )
        return

    try:

        _, is_new = add_topic_source(
            "exam",
            topic_name,
            url,

added_by=update.effective_user.id,

```

```

    )

except Exception:

    logger.exception(
        "addexamlink failed"
    )

    await update.message.reply_text(
        "Link save करने में error आया।"
    )

    return

if is_new:
    await update.message.reply_text(
        f"✅ Link save हो गया!\n\n"
        f"Exam: {topic_name}\n"
        f"Total links: {len(existing) +
1}\n\n"
        f"Quiz बनाने के लिए:\n"
        f"/examquiz {topic_name}"
    )
else:
    await update.message.reply_text(
        "यह link पहले से इस exam के लिए
save है।"
    )

async def addsubjectlink_command(update,
context):

    if not await require_admin(update):

```

```

        return

    if not context.args:
        await update.message.reply_text(
            "Usage:\n\n"
            "/addsubjectlink SUBJECT_NAME LINK\n\n"
            "Example:\n"
            "/addsubjectlink Indian Polity https://example.com/polity-quiz"
        )
        return

    topic_name, url =
_split_topic_name_and_url(context.args)

    if not topic_name or not url:
        await update.message.reply_text(
            "Subject का नाम और एक http(s) link दोनों देना जरूरी है।"
        )
        return

    existing = get_topic_sources("subject",
topic_name)

    if len(existing) >=
TOPIC_LINKS_MAX_SAVED:
        await update.message.reply_text(
            f"\n{topic_name}\n" के लिए पहले से
{TOPIC_LINKS_MAX_SAVED} links save हैं, "
            "जो limit है।"
        )
        return

```



```

try:

    _, is_new = add_topic_source(
        "subject",
        topic_name,
        url,

added_by=update.effective_user.id,
    )

except Exception:

    logger.exception(
        "addsubjectlink failed"
    )

    await update.message.reply_text(
        "Link save करने में error आया।"
    )

    return

if is_new:
    await update.message.reply_text(
        f"✅ Link save हो गया!\n\n"
        f"Subject: {topic_name}\n"
        f"Total links: {len(existing) +
1}\n\n"
        f"Quiz बनाने के लिए:\n"
        f"/subjectquiz {topic_name}"
    )
else:
    await update.message.reply_text(

```

```
        "यह link पहले से इस subject के  
        लिए save है।"  
    )
```

```
async def examlinks_command(update,  
context):  
  
    if not await require_admin(update):  
        return  
  
    if not context.args:  
        await update.message.reply_text(  
            "Usage:\n\n"  
            "/examlinks EXAM_NAME"  
        )  
        return  
  
    topic_name = "  
".join(context.args).strip()  
  
    saved = get_topic_sources("exam",  
topic_name)  
  
    if not saved:  
        await update.message.reply_text(  
            f"\n{topic_name}\n" के लिए कोई  
link save नहीं है\n\n"  
            f"जोड़ने के लिए:\n"  
            f"/addexamlink {topic_name}  
LINK"  
        )  
        return
```

```

        lines = [f"

```

```

        f"\{topic_name}\\" के लिए कोई
link save नहीं है\\n\\n"
        f"जोड़ने के लिए:\\n"
        f"/addsubjectlink {topic_name}
LINK"
    )
    return

    lines = [f"

```

```

    )
    return

    lines = ["📖 Saved Exams:\n"]

    for row in rows:
        lines.append(
            f"• {row['topic_name']}
({row['link_count']} links)"
        )

    await update.message.reply_text(
        "\n".join(lines)[:4000]
    )

async def subjectlist_command(update,
context):

    if not await require_admin(update):
        return

    rows = list_topic_names("subject")

    if not rows:
        await update.message.reply_text(
            "अभी तक कोई subject link save
नहीं किया गया।\n\n"
            "/addsubjectlink SUBJECT_NAME
LINK"
        )
        return

    lines = ["📖 Saved Subjects:\n"]

```

```

        for row in rows:
            lines.append(
                f"• {row['topic_name']}
({row['link_count']} links)"
            )

        await update.message.reply_text(
            "\n".join(lines)[:4000]
        )

    async def _run_topic_link_quiz(
        update,
        context,
        kind,
        topic_name,
        count
    ):
        """
        /examquiz और /subjectquiz दोनों का
        common flow: saved links से
        content जोड़ना, Groq से questions बनाना,
        save करना, quiz बनाकर
        auto-start करना।
        """

        label = "Exam" if kind == "exam" else
        "Subject"

        await update.message.reply_text(
            f"🔗 \"{topic_name}\" के लिए save
            किए गए links पढ़े जा रहे हैं...\n"
            f"इसके बाद AI {count} questions

```

```

generate करेगा और quiz अपने आप शुरू हो जाएगा।
\n"
        "कृपया प्रतीक्षा करें..."
    )

    try:

        context_text, sources,
        fetched_count = await asyncio.to_thread(
            build_context_from_topic_links,
            kind,
            topic_name,
            TOPIC_LINKS_MAX_PER_QUIZ,
            TOPIC_LINKS_CHARS_PER_PAGE,
        )

    except Exception as e:

        logger.exception(
            "Topic-link fetch failed (%s):
%s",
            kind,
            topic_name
        )

        await update.message.reply_text(
            "Saved links से content लाने में
error आया।\n\n"
            f"Error: {str(e)[:500]}"
        )

    return

    if not context_text:

```

```

        cmd = "addexamlink" if kind ==
"exam" else "addsubjectlink"
        await update.message.reply_text(
            f"\n{topic_name}\n" के saved
links से कोई content नहीं मिला "
            "(links काम नहीं कर रहे या खाली
हैं)\n\n"
            f"पहले links check/add करें:\n"
            f"/{cmd} {topic_name} LINK"
        )
        return

    try:

        questions = await
asyncio.to_thread(
            groq_generate_questions,
            topic_name,
            count,
            context_text,
            f"topic_links_{kind}",
        )

    except Exception as e:

        logger.exception(
            "AI generation from topic links
failed (%s): %s",
            kind,
            topic_name
        )

        await update.message.reply_text(
            "AI question generation में error

```



```

आया|\n\n"
        f"Error: {str(e)[:500]}"
    )

    return

    if not questions:
        await update.message.reply_text(
            "AI से कोई valid question प्राप्त
नहीं हुआ|\n"
            "कृपया दोबारा try करें!"
        )
    return

    added = 0
    duplicate = 0
    failed = 0
    new_ids = []

    for question in questions:

        try:

            question_id, status =
add_question(question)

            if question_id and status ==
"added":
                added += 1
                new_ids.append(question_id)
            elif question_id and status ==
"duplicate":
                duplicate += 1
            else:

```

```

        failed += 1

    except Exception:

        logger.exception(
            "Question save failed
(topic_links_%s)",
            kind
        )

        failed += 1

    if not new_ids:
        await update.message.reply_text(
            "Questions generate हुए लेकिन
सभी duplicate/invalid निकले\\n"
            f"Generated: {len(questions)} |
Duplicate: {duplicate} | Failed: {failed}"
        )
        return

    quiz_id =
create_quiz_from_question_ids(
        user_id=update.effective_user.id,
        title=f"{label}: {topic_name}"
[:100],
        question_ids=new_ids,
    )

    if not quiz_id:
        await update.message.reply_text(
            "Questions बन गए, लेकिन Quiz
create नहीं हो सका।"
        )

```

```

        return

    quiz_questions =
    get_saved_quiz_questions(quiz_id)

    if not quiz_questions:
        await update.message.reply_text(
            "Quiz में questions नहीं मिले।"
        )
        return

    session_id = create_quiz_session(
        user_id=update.effective_user.id,
        questions=quiz_questions,
        quiz_id=quiz_id,
    )

    source_note = ""

    if sources:
        top_sources = "\n".join(f"• {link}"
    for link in sources[:4])
        source_note = f"\n\nSources used
    ({fetched_count}): \n{top_sources}"

    await update.message.reply_text(
        f"✅ {label} Quiz तैयार है!\n\n"
        f"{label}: {topic_name}\n"
        f"Generated: {len(questions)}\n"
        f"Added: {added}\n"
        f"Duplicate: {duplicate}\n"
        f"Failed: {failed}\n"
        f"Quiz ID: {quiz_id}"
        f"{source_note}\n\n"
    )

```

```

        "पहला question शुरू हो रहा है..."
    )

    await send_quiz_question(
        context,
        update.effective_chat.id,
        session_id,
    )

async def examquiz_command(update,
context):
    """
    /examquiz [COUNT] EXAM_NAME

    /addexamlink से save किए गए उस exam के
    सभी website links पढ़कर
    उन्हीं पर based authentic quiz बनाता है और
    तुरंत शुरू कर देता है।
    """

    if not await require_admin(update):
        return

    if groq_client is None:
        await update.message.reply_text(
            "GROQ_API_KEY configured नहीं है,
            इसलिए questions generate नहीं हो सकते।"
        )
        return

    if not context.args:
        await update.message.reply_text(
            "Usage:\n\n"

```

```

        "/examquiz [COUNT]
EXAM_NAME\n\n"
        "पहले links save करें:\n"
        "/addexamlink EXAM_NAME
LINK\n\n"
        "Example:\n"
        "/examquiz 20 RAS Pre 2027"
    )
    return

    args = list(context.args)

    count = DEFAULT_QUIZ_COUNT

    if args and args[0].isdigit():
        count = int(args[0])
        args = args[1:]

    count = max(1, min(count,
MAX_IMPORT_QUESTIONS))

    topic_name = " ".join(args).strip()

    if not topic_name:
        await update.message.reply_text(
            "Exam का नाम देना जरूरी है।"
        )
        return

    if not get_topic_sources("exam",
topic_name):
        await update.message.reply_text(
            f"\n{topic_name}\n के लिए अभी
तक कोई link save नहीं है\n\n"

```

```

        f"पहले link जोड़ें:\n"
        f"/addexamlink {topic_name}
LINK\n\n"
        "(चाहें तो एक साथ कई links भी जोड़
सकते हैं – हर बार यही command "
        "अलग-अलग link के साथ चलाएं)"
    )
    return

    await _run_topic_link_quiz(
        update,
        context,
        "exam",
        topic_name,
        count,
    )

async def subjectquiz_command(update,
context):
    """
    /subjectquiz [COUNT] SUBJECT_NAME

    /addsubjectlink से save किए गए उस
subject के सभी website links पढ़कर
उन सबका मिला-जुला (authentic) content
इस्तेमाल करके quiz बनाता है
और तुरंत शुरू कर देता है।
    """

    if not await require_admin(update):
        return

    if groq_client is None:

```

```

        await update.message.reply_text(
            "GROQ_API_KEY configured नहीं है,  
इसलिए questions generate नहीं हो सकते।"
        )
        return

    if not context.args:
        await update.message.reply_text(
            "Usage:\n\n"
            "/subjectquiz [COUNT]  
SUBJECT_NAME\n\n"
            "पहले links save करें:\n"
            "/addsubjectlink SUBJECT_NAME  
LINK\n\n"
            "Example:\n"
            "/subjectquiz 20 Indian Polity"
        )
        return

    args = list(context.args)

    count = DEFAULT_QUIZ_COUNT

    if args and args[0].isdigit():
        count = int(args[0])
        args = args[1:]

    count = max(1, min(count,
MAX_IMPORT_QUESTIONS))

    topic_name = " ".join(args).strip()

    if not topic_name:
        await update.message.reply_text(

```

```

        "Subject का नाम देना जरूरी है।"
    )
    return

    if not get_topic_sources("subject",
topic_name):
        await update.message.reply_text(
            f"\{topic_name}\\" के लिए अभी
तक कोई link save नहीं है!\n\n"
            f"पहले link जोड़ें:\n"
            f"/addsubjectlink {topic_name}
LINK\n\n"
            "(चाहे तो एक साथ कई links भी जोड़
सकते हैं – हर बार यही command "
            "अलग-अलग link के साथ चलाएं)"
        )
    return

    await _run_topic_link_quiz(
        update,
        context,
        "subject",
        topic_name,
        count,
    )

#
=====
=====
# SCRAPE COMMAND
#
=====
=====

```



```
async def scrape_command(update, context):

    if not await require_admin(update):
        return

    if not context.args:

        await update.message.reply_text(
            "Usage:\n\n"
            "/scrape SOURCE [COUNT]\n\n"
            "Available sources:\n"
            "rajras\n"
            "rbse\n"
            "samyak_rbse\n"
            "ncert\n"
            "online2study\n\n"
            "Example:\n"
            "/scrape rajras 15"
        )

        return

    source_name = (
        context.args[0]
        .strip()
        .lower()
    )

    count = DEFAULT_QUIZ_COUNT

    if len(context.args) > 1 and
context.args[1].isdigit():
        count = int(context.args[1])
```

```

count = max(
    1,
    min(count, MAX_IMPORT_QUESTIONS)
)

conn = db()

source = conn.execute(
    """
    SELECT *
    FROM sources
    WHERE name = ?
    AND enabled = 1
    """,
    (source_name,)
).fetchone()

conn.close()

if not source:

    await update.message.reply_text(
        "Source नहीं मिला या disabled है।
\n\n"
        "Available:\n"
        "rajras\n"
        "rbse\n"
        "samyak_rbse\n"
        "ncert\n"
        "online2study"
    )

return

```

```

        await update.message.reply_text(
            f"Source scrape किया जा रहा है:\n"
            f"{source_name}\n\n"
            f"AI {count} questions generate
करेगा और quiz अपने आप शुरू हो जाएगा।\n"
            "यह process थोड़ा समय ले सकता है।"
        )

    try:

        result = await asyncio.to_thread(
            scrape_and_generate,
            source_name,
            None,
            count,
        )

    except Exception:

        logger.exception(
            "Scrape failed"
        )

        await update.message.reply_text(
            "Scraping में error आया।"
        )

        return

    if not result:

        await update.message.reply_text(
            "Scraping से कोई question

```

```

generate नहीं हुआ।"
    )

    return

    added = 0
    duplicate = 0
    failed = 0
    new_ids = []

    for question in result:

        try:

            question_id, status =
add_question(
                question
            )

            if question_id and status ==
"added":
                added += 1
                new_ids.append(question_id)
            elif question_id and status ==
"duplicate":
                duplicate += 1
            else:
                failed += 1

        except Exception:

            logger.exception(
                "Scraped question save
failed"

```

```

        )

        failed += 1

    if not new_ids:

        await update.message.reply_text(
            "Questions generate हुए लेकिन  
सभी duplicate/invalid निकले\\n"
            f"Generated: {len(result)} |  
Duplicate: {duplicate} | Failed: {failed}"
        )

        return

    quiz_id =
    create_quiz_from_question_ids(
        user_id=update.effective_user.id,
        title=f"Source: {source_name}"
        [:100],
        question_ids=new_ids,
    )

    if not quiz_id:

        await update.message.reply_text(
            "Questions बन गए, लेकिन Quiz  
create नहीं हो सका।"
        )

        return

    quiz_questions =
    get_saved_quiz_questions(quiz_id)

```

```

if not quiz_questions:

    await update.message.reply_text(
        "Quiz में questions नहीं मिले।"
    )

    return

session_id = create_quiz_session(
    user_id=update.effective_user.id,
    questions=quiz_questions,
    quiz_id=quiz_id,
)

await update.message.reply_text(
    "✅ Scraping पूरी हुई, Quiz तैयार है।
\n\n"
    f"Source: {source_name}\n"
    f"Generated: {len(result)}\n"
    f"Added: {added}\n"
    f"Duplicate: {duplicate}\n"
    f"Failed: {failed}\n"
    f"Quiz ID: {quiz_id}\n\n"
    "पहला question शुरू हो रहा है..."
)

await send_quiz_question(
    context,
    update.effective_chat.id,
    session_id,
)

```

```

#
=====
=====
# SOURCE LIST COMMAND
#
=====
=====

async def sources_command(update, context):

    if not await require_admin(update):
        return

    conn = db()

    rows = conn.execute(
        """
        SELECT name, url, enabled
        FROM sources
        ORDER BY name
        """
    ).fetchall()

    conn.close()

    if not rows:

        await update.message.reply_text(
            "कोई source configured नहीं है।"
        )

        return

    lines = [

```

```

        "Configured Sources",
        ""
    ]

    for row in rows:

        status = (
            "Enabled"
            if row["enabled"]
            else "Disabled"
        )

        lines.append(
            f"{row['name']} - {status}"
        )

        lines.append(
            row["url"]
        )

        lines.append("")

    await update.message.reply_text(
        "\n".join(lines)
    )

```

```

#
=====
=====
# SOURCE ADD COMMAND
#
=====
=====

```



```
async def addsource_command(update,
context):

    if not await require_admin(update):
        return

    if len(context.args) < 2:

        await update.message.reply_text(
            "Usage:\n\n"
            "/addsource SOURCE_NAME\n\n"
            "URL\n\n"
            "Example:\n"
            "/addsource example\n\n"
            "https://example.com/"
        )

        return

    name = (
        context.args[0]
        .strip()
        .lower()
    )

    url = (
        context.args[1]
        .strip()
    )

    if not re.match(
        r"^https?://",
        url,
```

```

        re.IGNORECASE
    ):

        await update.message.reply_text(
            "Valid HTTP/HTTPS URL है।"
        )

        return

conn = db()

try:

    conn.execute(
        """
        INSERT INTO sources
        (name, url, enabled)
        VALUES (?, ?, 1)

        ON CONFLICT(name)
        DO UPDATE SET
            url = excluded.url,
            enabled = 1
        """,
        (
            name,
            url
        )
    )

    conn.commit()

except Exception:

```

```

        conn.rollback()

        logger.exception(
            "Source add failed"
        )

        await update.message.reply_text(
            "Source save नहीं हो सका।"
        )

        conn.close()

        return

    conn.close()

    await update.message.reply_text(
        "Source successfully save हो गया।
\n\n"
        f"Name: {name}\n"
        f"URL: {url}"
    )

#
=====
=====
# SOURCE ENABLE / DISABLE
#
=====
=====

async def source_command(update, context):

```

```
if not await require_admin(update):
    return

if len(context.args) < 2:

    await update.message.reply_text(
        "Usage:\n\n"
        "/source NAME on\n"
        "/source NAME off"
    )

    return

name = (
    context.args[0]
    .strip()
    .lower()
)

action = (
    context.args[1]
    .strip()
    .lower()
)

if action not in (
    "on",
    "off"
):

    await update.message.reply_text(
        "केवल on या off इस्तेमाल करें।"
    )
```

```

        return

        enabled = 1 if action == "on" else 0

        conn = db()

        cursor = conn.execute(
            """
            UPDATE sources
            SET enabled = ?
            WHERE name = ?
            """,
            (
                enabled,
                name
            )
        )

        conn.commit()

        changed = cursor.rowcount

        conn.close()

        if not changed:

            await update.message.reply_text(
                "Source नहीं मिला।"
            )

        return

    status = (
        "enabled"

```

```

        if enabled
        else "disabled"
    )

    await update.message.reply_text(
        f"Source '{name}' {status} कर दिया
गया।"
    )
    #
    =====
    =====
    # PDF / TXT IMPORT COMMANDS
    #
    =====
    =====

async def pdfimport_command(update,
context):

    if not await require_admin(update):
        return

    user_id = update.effective_user.id

    PENDING[user_id] = {
        "action": "pdfimport"
    }

    await update.message.reply_text(
        "अब PDF file भेजें।\n\n"
        "PDF के questions database में import
किए जाएंगे।\n\n"
        "Cancel करने के लिए /cancel भेजें।"
    )

```

```

async def txtimport_command(update,
context):

    if not await require_admin(update):
        return

    user_id = update.effective_user.id

    PENDING[user_id] = {
        "action": "txtimport"
    }

    await update.message.reply_text(
        "अब TXT file भेजें\\n\\n"
        "TXT के questions database में import
किए जाएंगे\\n\\n"
        "Cancel करने के लिए /cancel भेजें|"
    )


#
=====
=====
# PDF INFO
#
=====
=====

async def pdfinfo_command(update, context):

    if not await require_admin(update):
        return

```

```

try:

    files = sorted(
        PDF_DIR.iterdir(),
        key=lambda p:
p.stat().st_mtime,
        reverse=True
    )

    except Exception:

        await update.message.reply_text(
            "PDF directory access नहीं हो
सकी।"
        )

        return

pdf_files = [
    p for p in files
    if p.is_file()
    and p.suffix.lower() == ".pdf"
]

if not pdf_files:

    await update.message.reply_text(
        "अभी कोई PDF उपलब्ध नहीं है।"
    )

    return

lines = [

```



```

        "PDF Files",
        ""
    ]

    for index, path in enumerate(
        pdf_files[:30],
        start=1
    ):

        try:

            size_mb = (
                path.stat().st_size
                / (1024 * 1024)
            )

            modified =
datetime.fromtimestamp(
                path.stat().st_mtime
            ).strftime(
                "%Y-%m-%d %H:%M"
            )

        except Exception:

            size_mb = 0
            modified = "-"

        lines.append(
            f"{index}. {path.name}"
        )

        lines.append(
            f"Size: {size_mb:.2f} MB"

```

```

    )

    lines.append(
        f"Modified: {modified}"
    )

    lines.append("")

    if len(pdf_files) > 30:

        lines.append(
            f"... और {len(pdf_files) - 30}
files"
        )

        await update.message.reply_text(
            "\n".join(lines)
        )

#
=====
=====
# CHANNEL AUTO PDF IMPORT
#
# Configured CHANNEL_ID में जो भी नई PDF post
होगी,
# उसे अपने आप download करके questions
generate कर देगा।
#
=====
=====

async def channel_pdf_handler(

```

```
        update,
        context
    ):

        message = (
            update.channel_post
            or update.edited_channel_post
        )

        if not message:
            return

        if not CHANNEL_ID:
            return

        try:
            configured_id = int(CHANNEL_ID)
        except Exception:
            configured_id = None

        chat_id = message.chat_id

        channel_matches = (
            configured_id is not None
            and chat_id == configured_id
        ) or (
            message.chat.username
            and message.chat.username.lower()
            == CHANNEL_ID.lstrip("@").lower()
        )

        if not channel_matches:
            return
```

```
document = message.document

if not document:
    return

filename = (
    document.file_name
    or "unknown.pdf"
)

extension = (
    Path(filename)
    .suffix
    .lower()
)

if extension != ".pdf":
    return

logger.info(
    "Channel PDF detected: %s",
    filename
)

try:
    telegram_file = await
context.bot.get_file(
    document.file_id
)

safe_name = (
    f"{uuid.uuid4().hex}_"
    f"{Path(filename).name}"
)
```

```

    )

    destination = (
        PDF_DIR / safe_name
    )

    await
telegram_file.download_to_drive(
    custom_path=str(destination)
)

    result = import_pdf(
        user_id=0,
        file_path=str(destination),
        file_name=filename,
        count=None,
    )

    added = result.get("added", 0) if
isinstance(result, dict) else 0

    summary = (
        "🍰 Channel PDF Auto-
Import\n\n"
        f"File: {filename}\n"
        f"Questions added: {added}"
    )

    except Exception:

        logger.exception(
            "Channel PDF auto-import
failed"
        )

```

```

        summary = (
            "🍰 Channel PDF Auto-Import
Failed\n\n"
            f"File: {filename}"
        )

    for admin_id in ADMIN_IDS:

        try:

            await context.bot.send_message(
                chat_id=admin_id,
                text=summary
            )

        except Exception:
            logger.exception(
                "Failed notifying admin
%s",
                admin_id
            )

#
=====
=====
# DOCUMENT IMPORT HANDLER
#
=====
=====

async def document_import_handler(
    update,

```

```
context
):

    if not update.message:
        return

    user = update.effective_user

    if not user:
        return

    if not is_admin(user.id):
        return

    document = update.message.document

    if not document:
        return

    filename = (
        document.file_name
        or "unknown"
    )

    extension = (
        Path(filename)
        .suffix
        .lower()
    )

    pending = PENDING.get(
        user.id
    )
```

```

# -----
-----
# अगर कोई import operation pending नहीं
है
# -----
-----

if not pending:

    if extension in (".pdf", ".txt",
".html", ".htm"):
        temp_path = None
        try:
            temp_path = DATA_DIR / f"
{uuid.uuid4().hex}{extension}"
            telegram_file = await
context.bot.get_file(document.file_id)
            await
telegram_file.download_to_drive(custom_path
=str(temp_path))

            await
update.message.reply_text(
                "File मिल गई!\n\n"
                "अब PDF/TXT/HTML को
automatically पढ़कर जितने उपयोगी MCQ संभव हैं "
                "वे generate किए जाएंगे।
\n"
                "Questions की संख्या बताने
की जरूरत नहीं है!\n\n"
                "कृपया processing पूरी होने
तक प्रतीक्षा करें..."
            )

```



```

        if extension == ".pdf":
            result = await
        asyncio.to_thread(
            import_pdf,
            user.id, str(temp_path), filename, None
        )
        elif extension in (".html",
            ".htm"):
            result = await
        asyncio.to_thread(
            import_html,
            str(temp_path), filename, None
        )
        else:
            result = await
        asyncio.to_thread(
            import_txt,
            str(temp_path), filename, None
        )

        new_ids = result.get("ids",
            []) if isinstance(result, dict) else []
        if not new_ids:
            await
        update.message.reply_text(
            "File पढ़ ली गई,  

            लेकिन कोई valid नया MCQ नहीं बन सका!\n"
            "Scanned PDF के लिए  

            Koyeb requirements में PyMuPDF install होना  

            चाहिए!"
        )
        return

    quiz_id =

```

```

create_quiz_from_question_ids(
    user_id=user.id,
    title=f"Auto:
{filename}",
    question_ids=new_ids,
)
if not quiz_id:
    await
update.message.reply_text("Questions बन
गए, लेकिन Quiz create नहीं हो सका।")
    return

    quiz_questions =
get_saved_quiz_questions(quiz_id)
    if not quiz_questions:
        await
update.message.reply_text("Quiz में questions
नहीं मिले।")
        return

        session_id =
create_quiz_session(
            user_id=user.id,

questions=quiz_questions,
            quiz_id=quiz_id,
        )

        added = result.get("added",
0)

        duplicate =
result.get("duplicate", 0)
        failed =
result.get("failed", 0)

```

```

        total = result.get("total",
0)

        cached_note = "\n🔄 यह PDF
पहले process हो चुकी थी; AI processing दोबारा
नहीं चली।" if result.get("cached") else ""
        await
update.message.reply_text(
            "Automatic Quiz तैयार है।
\n\n"
            f"File: {filename}\n"
            f"Generated: {total}\n"
            f"Added: {added}\n"
            f"Duplicate:
{duplicate}\n"
            f"Failed: {failed}\n"
            f"Quiz ID: {quiz_id}
{cached_note}\n\n"
            "अब पहला question शुरू
हो रहा है..."
        )

        await send_quiz_question(
            context,

update.effective_chat.id,
            session_id,
        )

    except Exception:
        logger.exception("Automatic
document import failed: %s", filename)
        await
update.message.reply_text(

```

```

        "File process करते समय
error आया!\n"
        "Log में पूरा error दर्ज है!"
    )
    finally:
        if temp_path:
            try:
                Path(temp_path).unlink(missing_ok=True)
            except Exception:
                pass

    return

    action = pending.get(
        "action"
    )

    # -----
    -----
    # PDF IMPORT
    # -----
    -----

    if action == "pdfimport":

        if extension != ".pdf":

            await
update.message.reply_text(
        "इस operation में केवल PDF
file भेजें!"
    )

```

```

        return

        await update.message.reply_text(
            "PDF download और process की जा
रही है...\n"
            "कृपया प्रतीक्षा करें।"
        )

    try:

        telegram_file = await
context.bot.get_file(
            document.file_id
        )

        safe_name = (
            f"{uuid.uuid4().hex}_"
            f"{Path(filename).name}"
        )

        destination = (
            PDF_DIR / safe_name
        )

        await
telegram_file.download_to_drive(
            custom_path=str(destination)
        )

        # PDF को database में import करें
        result = import_pdf(
            user_id=user.id,
            file_path=str(destination),

```

```
        file_name=filename,
    )

    except Exception:

        logger.exception(
            "PDF import failed"
        )

        PENDING.pop(
            user.id,
            None
        )

        await
    update.message.reply_text(
        "PDF import करते समय error
    आया।"
    )

    return

    PENDING.pop(
        user.id,
        None
    )

    if isinstance(result, dict):

        added = result.get(
            "added",
            0
        )
```

```
duplicate = result.get(
    "duplicate",
    result.get("duplicates", 0)
)

failed = result.get(
    "failed",
    0
)

total = result.get(
    "total",
    added + duplicate + failed
)

new_ids = result.get(
    "ids",
    []
)

summary_lines = [
    "PDF Import Complete",
    "",
    f"File: {filename}",
    f"Total: {total}",
    f"Added: {added}",
    f"Duplicate: {duplicate}",
    f"Failed: {failed}",
]

if new_ids:

    quiz_id =
    create_quiz_from_question_ids(
```

```

        user_id=user.id,
        title=f"PDF:
{filename}",
        question_ids=new_ids,
    )

    if quiz_id:

        summary_lines += [
            "",
            f"Quiz बन गया (Quiz
ID: {quiz_id})",
            f"Start करने के
लिए:",
            f"/quizid
{quiz_id}",
        ]

        await
update.message.reply_text(
            "\n".join(summary_lines)
        )

    else:

        await
update.message.reply_text(
            "PDF process हो गई!\n\n"
            f"Result: {result}"
        )

    return

# -----

```



```

-----
# TXT IMPORT
# -----
-----

if action == "txtimport":

    if extension != ".txt":

        await
update.message.reply_text(
    "इस operation में केवल TXT
file भेजें।"
)

    return

    await update.message.reply_text(
        "TXT file process की जा रही
है..."
    )

    temp_path = None

    try:

        temp_path = (
            DATA_DIR
            / f"{uuid.uuid4().hex}.txt"
        )

        telegram_file = await
context.bot.get_file(
    document.file_id

```

```

        )

        await
telegram_file.download_to_drive(
            custom_path=str(temp_path)
        )

        result = import_txt(
            file_path=str(temp_path),
            file_name=filename,
        )

    except Exception:

        logger.exception(
            "TXT import failed"
        )

        PENDING.pop(
            user.id,
            None
        )

        await
update.message.reply_text(
            "TXT import करते समय error
आया।"
        )

        return

    finally:

        if temp_path:

```

```
        try:

            temp_path.unlink(
                missing_ok=True
            )

        except Exception:

            pass

    PENDING.pop(
        user.id,
        None
    )

    if isinstance(result, dict):

        added = result.get(
            "added",
            0
        )

        duplicate = result.get(
            "duplicate",
            result.get("duplicates", 0)
        )

        failed = result.get(
            "failed",
            0
        )

        total = result.get(
```

```

        "total",
        added + duplicate + failed
    )

    new_ids = result.get(
        "ids",
        []
    )

    summary_lines = [
        "TXT Import Complete",
        "",
        f"File: {filename}",
        f"Total: {total}",
        f"Added: {added}",
        f"Duplicate: {duplicate}",
        f"Failed: {failed}",
    ]

    if new_ids:

        quiz_id =
create_quiz_from_question_ids(
            user_id=user.id,
            title=f"TXT:
{filename}",
            question_ids=new_ids,
        )

        if quiz_id:

            summary_lines += [
                "",
                f"Quiz बन गया (Quiz

```

```

ID: {quiz_id})",
                                f"Start करने के
लिए:",
                                f"/quizid
{quiz_id}",
                                ]

        await
update.message.reply_text(
                                "\n".join(summary_lines)
                                )

        else:

            await
update.message.reply_text(
                                "TXT process हो गई!\n\n"
                                f"Result: {result}"
                                )

        return
#
=====
=====
# TELEGRAM POLL HELPERS
#
=====
=====

def extract_poll_question(poll):
    """
        Telegram Poll object से question payload
        तैयार करता है।
    """

```

```
if not poll:
    return None

question_text = (
    poll.question
    or ""
).strip()

options = [
    (option.text or "").strip()
    for option in poll.options
]

if not question_text:
    return None

if len(options) != 4:
    return None

if any(
    not option
    for option in options
):
    return None

correct_index = poll.correct_option_id

if (
    correct_index is None
    or correct_index < 0
    or correct_index >= len(options)
):
    return None
```

```

correct_letter = chr(
    ord("A") + correct_index
)

return {
    "question": question_text,
    "option_a": options[0],
    "option_b": options[1],
    "option_c": options[2],
    "option_d": options[3],
    "answer": correct_letter,
    "explanation": "",
    "exam": "",
    "subject": "",
}

```

```

#
=====
=====
# SAVE POLL QUESTION
#
=====
=====

```

```

def save_poll_question(poll):

    payload = extract_poll_question(
        poll
    )

    if not payload:
        return {

```

```

        "success": False,
        "reason": "invalid_poll",
        "question_id": None,
    }

    question_id, status = add_question(
        payload
    )

    if question_id and status == "added":

        return {
            "success": True,
            "reason": "added",
            "question_id": question_id,
        }

    return {
        "success": False,
        "reason": status if question_id
    else "invalid",
        "question_id": question_id,
    }

#
=====
=====
# POLL2Q COMMAND
#
=====
=====

async def poll2q_command(

```



```

        update,
        context
    ):

        if not await require_admin(update):
            return

        PENDING[
            update.effective_user.id
        ] = {
            "action": "poll2q"
        }

        await update.message.reply_text(
            "अब Telegram Quiz Poll भेजें या\n\n"
            "forward करें\n\n"
            "Bot उसके question, options और\n\n"
            "correct answer "\n\n"
            "को database में save करेगा\n\n"
            "केवल Quiz Poll स्वीकार किया जाएगा।\n\n"
            "Cancel: /cancel"
        )

#
=====
=====
# POLL MESSAGE HANDLER
#
=====
=====

async def poll_message_handler(

```

```
        update,
        context
    ):

        user = update.effective_user

        if not user:
            return

        if not is_admin(user.id):
            return

        poll = None

        if update.message:
            poll = update.message.poll

        elif update.channel_post:
            poll = update.channel_post.poll

        if not poll:
            return

        pending = PENDING.get(
            user.id
        )

        # -----
        -----
        # अगर poll2q mode active नहीं है
        # -----
        -----

        if not pending:
```

```

        await
update.effective_message.reply_text(
            "Poll receive हुआ है!\n\n"
            "इसे question में convert करने के
लिए पहले:\n"
            "/poll2q"
        )

        return

        action = pending.get(
            "action"
        )

        if action not in (
            "poll2q",
            "scrapepoll"
        ):

            return

        result = save_poll_question(
            poll
        )

        if result["success"]:

            await
update.effective_message.reply_text(
                "Poll successfully question में
convert हो गया!\n\n"
                f"Question ID:
{result['question_id']}"

```

```

    )

    elif result["reason"] == "duplicate":

        await
update.effective_message.reply_text(
    "यह question पहले से database में
मौजूद है।\n"
    "Duplicate होने के कारण save नहीं
किया गया।"
    )

    else:

        await
update.effective_message.reply_text(
    "Poll valid Quiz Poll नहीं है।
\n\n"
    "चार options वाला quiz poll भेजें
जिसमें "
    "correct answer मौजूद हो।"
    )

#
=====
=====
# SCRAPE POLL COMMAND
#
=====
=====

async def scrapepoll_command(
    update,

```

```

context
):

    if not await require_admin(update):
        return

    PENDING[
        update.effective_user.id
    ] = {
        "action": "scrapepoll",
        "count": 0,
        "added": 0,
        "duplicate": 0,
        "failed": 0,
    }

    await update.message.reply_text(
        "Poll scraping mode ON है\\n\\n"
        "अब Quiz Polls forward/send करें।"
        "\\n\\n"
        "हर valid poll को database में save"
        "किया जाएगा\\n\\n"
        "Statistics देखने के लिए /stop या"
        "/cancel भेजें।"
    )

#
=====
=====
# SCRAPE POLL STATUS
#
=====
=====

```

```

async def poll_status(
    update,
    context
):

    user_id = update.effective_user.id

    pending = PENDING.get(
        user_id
    )

    if not pending:

        await update.message.reply_text(
            "Poll scraping mode active नहीं है।"
        )

        return

    if pending.get(
        "action"
    ) != "scrapepoll":

        await update.message.reply_text(
            "Poll scraping mode active नहीं है।"
        )

        return

    await update.message.reply_text(
        "Poll Processing Status\n\n"

```

```

        f"Processed: {pending.get('count',
0)}\n"
        f"Added: {pending.get('added',
0)}\n"
        f"Duplicate:
{pending.get('duplicate', 0)}\n"
        f"Failed: {pending.get('failed',
0)}"
    )

#
=====
=====
# UPDATED POLL HANDLER FOR SCRAPING STATS
#
=====
=====

async def process_poll(
    update,
    context
):

    user = update.effective_user

    if not user:
        return

    if not is_admin(user.id):
        return

    poll = None

```

```
if update.message:
    poll = update.message.poll

elif update.channel_post:
    poll = update.channel_post.poll

if not poll:
    return

pending = PENDING.get(
    user.id
)

if not pending:
    return

action = pending.get(
    "action"
)

if action == "scrapepoll":

    pending["count"] = (
        pending.get("count", 0) + 1
    )

    result = save_poll_question(
        poll
    )

    if result["success"]:

        pending["added"] = (
            pending.get("added", 0) + 1
```



```

        )

        elif result["reason"] ==
        "duplicate":

            pending["duplicate"] = (
                pending.get("duplicate", 0)
+ 1
            )

        else:

            pending["failed"] = (
                pending.get("failed", 0) +
1
            )

        await
update.effective_message.reply_text(
            "Poll processed.\n\n"
            f"Processed:
{pending['count']}\n"
            f"Added: {pending['added']}\n"
            f"Duplicate:
{pending['duplicate']}\n"
            f"Failed: {pending['failed']}"
        )

        return

    if action == "poll2q":

        result = save_poll_question(
            poll

```

```

    )

    PENDING.pop(
        user.id,
        None
    )

    if result["success"]:

        await
update.effective_message.reply_text(
            "Poll successfully convert
हो गया!\n\n"
            f"Question ID:
{result['question_id']}"
        )

        elif result["reason"] ==
"duplicate":

            await
update.effective_message.reply_text(
                "Question पहले से database में
मौजूद है।"
            )

        else:

            await
update.effective_message.reply_text(
                "Invalid Quiz Poll!"
            )
        #
=====

```

```

=====
# CLONE QUIZ HELPERS
#
=====
=====

def get_next_clone_job():
    """
    Queue से अगला queued job निकालता है।
    """

    with CLONE_LOCK:

        for item in CLONE_QUEUE:

            if item.get("status") ==
"queued":

                item["status"] =
"processing"

                return item

        return None

def update_clone_job(
    job_id,
    **updates
):

    with CLONE_LOCK:

        for item in CLONE_QUEUE:

```

```

        if item.get("job_id") ==
job_id:

            item.update(
                updates
            )

            return item

    return None

def clone_saved_quiz(
    source_quiz_id,
    owner_id,
    new_title=None
):
    """
    Existing saved quiz को clone करके नया
    quiz बनाता है।
    """

    source = get_quiz(
        source_quiz_id
    )

    if not source:
        raise ValueError(
            "Source quiz नहीं मिला।"
        )

    questions = get_saved_quiz_questions(
        source_quiz_id

```

```

    )

    if not questions:
        raise ValueError(
            "Source quiz में कोई question नहीं
            है।"
        )

    title = (
        new_title
        or f"{source['title']} - Clone"
    )

    new_quiz_id = create_quiz(
        user_id=owner_id,
        title=title,
    )

    if not new_quiz_id:
        raise RuntimeError(
            "New quiz create नहीं हो सका।"
        )

    question_ids = []

    for question in questions:

        if isinstance(
            question,
            sqlite3.Row
        ):
            question_id = question["id"]
        else:
            question_id = (

```

```

        question.get("id")
        if isinstance(
            question,
            dict
        )
        else None
    )

    if question_id:
        question_ids.append(
            question_id
        )

    if not question_ids:
        raise RuntimeError(
            "Quiz questions प्राप्त नहीं हुए।"
        )

    save_quiz_questions(
        new_quiz_id,
        question_ids
    )

    return (
        new_quiz_id,
        len(question_ids)
    )

```

```

#
=====
=====
# CLONE QUEUE WORKER
#

```

```
=====
=====

async def clone_queue_worker(
    application
):

    while True:

        job = get_next_clone_job()

        if not job:

            await asyncio.sleep(
                1
            )

            continue

        job_id = job.get(
            "job_id"
        )

        try:

            new_quiz_id, count = (
                clone_saved_quiz(

source_quiz_id=job["quiz_id"],

owner_id=job["user_id"],

new_title=job.get("title")
                )
```

```

    )

    update_clone_job(
        job_id,
        status="completed",
        new_quiz_id=new_quiz_id,
        question_count=count,
        completed_at=utcnow()
    )

    try:

        await
application.bot.send_message(
            chat_id=job["user_id"],
            text=(
                "Quiz Clone
Complete\n\n"
                f"Original Quiz:
{job['quiz_id']}\n"
                f"New Quiz ID:
{new_quiz_id}\n"
                f"Questions:
{count}\n\n"
                f"Start करने के
लिए:\n"
                f"/quizid
{new_quiz_id}"
            )
        )

    except Exception:

        logger.exception(

```



```

        "Clone completion
message failed"
    )

    except Exception as e:

        logger.exception(
            "Clone job failed"
        )

        update_clone_job(
            job_id,
            status="failed",
            error=str(e)[:500],
            completed_at=utcnow()
        )

    try:

        await
application.bot.send_message(
            chat_id=job["user_id"],
            text=(
                "Quiz clone नहीं हो
सका!\n\n"
                f"Quiz ID:
{job['quiz_id']}\n"
                f"Error: {str(e)
[:500]}"
            )
        )

    except Exception:

```

```

        logger.exception(
            "Clone failure message
failed"
        )

#
=====
=====
# CLONE COMMAND
#
=====
=====

async def clone_command(
    update,
    context
):

    if not await require_admin(update):
        return

    if not context.args:

        await update.message.reply_text(
            "Usage:\n\n"
            "/clone QUIZ_ID\n\n"
            "Optional title:\n"
            "/clone 12 New CET Quiz"
        )

    return

try:

```

```
        quiz_id = int(
            context.args[0]
        )

    except Exception:

        await update.message.reply_text(
            "Invalid Quiz ID."
        )

        return

    source = get_quiz(
        quiz_id
    )

    if not source:

        await update.message.reply_text(
            "Quiz नहीं मिला।"
        )

        return

    questions = get_saved_quiz_questions(
        quiz_id
    )

    if not questions:

        await update.message.reply_text(
            "इस Quiz में कोई question नहीं है।"
        )
```

```

        return

    title = None

    if len(context.args) > 1:

        title = " ".join(
            context.args[1:]
        ).strip()

    job_id = uuid.uuid4().hex[:12]

    job = {
        "job_id": job_id,
        "user_id":
update.effective_user.id,
        "quiz_id": quiz_id,
        "title": title,
        "status": "queued",
        "question_count": len(questions),
        "created_at": utcnow(),
    }

    with CLONE_LOCK:

        CLONE_QUEUE.append(
            job
        )

    await update.message.reply_text(
        "Clone request queue में add हो गई।
\n\n"
        f"Job ID: {job_id}\n"

```

```

        f"Original Quiz: {quiz_id}\n"
        f"Questions: {len(questions)}\n\n"
        "Status देखने के लिए:\n"
        "/queue"
    )

#
=====
=====
# CLONE JOB STATUS
#
=====
=====

async def clone_status_command(
    update,
    context
):

    if not await require_admin(update):
        return

    if not context.args:

        await update.message.reply_text(
            "Usage:\n\n"
            "/clonestatus JOB_ID"
        )

        return

    job_id = (
        context.args[0]

```

```

        .strip()
    )

    with CLONE_LOCK:

        job = next(
            (
                item
                for item in CLONE_QUEUE
                if item.get("job_id") ==
job_id
            ),
            None
        )

        if not job:

            await update.message.reply_text(
                "Clone Job नहीं मिला।"
            )

            return

        text = (
            "Clone Job\n\n"
            f"Job ID: {job.get('job_id')}\n"
            f"Original Quiz:
{job.get('quiz_id')}\n"
            f"Status: {job.get('status')}\n"
            f"Questions:
{job.get('question_count', 0)}\n"
        )

        if job.get("new_quiz_id"):

```

```

        text += (
            f"New Quiz ID: "
            f"{job.get('new_quiz_id')}\n"
        )

    if job.get("error"):

        text += (
            f"\nError: {job.get('error')}"
        )

    await update.message.reply_text(
        text
    )

#
=====
=====
# CLEAR COMPLETED CLONE JOBS
#
=====
=====

async def clearqueue_command(
    update,
    context
):

    if not await require_admin(update):
        return

    with CLONE_LOCK:

```

```

        before = len(
            CLONE_QUEUE
        )

        CLONE_QUEUE[:] = [
            item
            for item in CLONE_QUEUE
            if item.get("status")
            not in (
                "completed",
                "failed"
            )
        ]

        removed = (
            before
            - len(CLONE_QUEUE)
        )

        await update.message.reply_text(
            f"{removed} completed/failed clone
jobs "
            "queue से remove कर दिए गए।"
        )

#
=====
=====
# HTML SOURCE INFO
#
=====
=====

def analyze_html_source(source_name):

```



```

"""
    Source website को fetch करके basic HTML
    information निकालता है।
"""

conn = db()

source = conn.execute(
    """
        SELECT *
        FROM sources
        WHERE name = ?
    """,
    (source_name,)
).fetchone()

conn.close()

if not source:
    return None

url = source["url"]

response = fetch_url(url)

if not response:
    raise RuntimeError(
        "Website fetch नहीं हो सकी।"
    )

html_content = response.text

soup = BeautifulSoup(
    html_content,

```

```
        "html.parser"
    )

    title = ""

    if soup.title:
        title = (
            soup.title.get_text(
                " ",
                strip=True
            )
        )

    description = ""

    meta_description = soup.find(
        "meta",
        attrs={
            "name": "description"
        }
    )

    if meta_description:
        description = (
            meta_description.get(
                "content",
                ""
            )
            or ""
        ).strip()

    links = []

    for tag in soup.find_all(
```

```
        "a",
        href=True
    ):

        href = (
            tag.get(
                "href",
                ""
            )
            or ""
        ).strip()

        text = tag.get_text(
            " ",
            strip=True
        )

        if href:

            links.append(
                {
                    "text": text[:200],
                    "url": href
                }
            )

    images = soup.find_all(
        "img"
    )

    scripts = soup.find_all(
        "script"
    )
```

```
styles = soup.find_all(
    "link",
    rel=lambda value: (
        value
        and "stylesheet" in value
    )
)

headings = []

for level in range(1, 7):

    for heading in soup.find_all(
        f"h{level}"
    ):

        text = heading.get_text(
            " ",
            strip=True
        )

        if text:

            headings.append(
                {
                    "level": level,
                    "text": text[:300]
                }
            )

text_content = html_to_text(
    html_content
)
```

```
words = re.findall(
    r"\S+",
    text_content
)

return {
    "name": source_name,
    "url": url,
    "title": title,
    "description": description,
    "status_code":
response.status_code,
    "content_type":
response.headers.get(
        "content-type",
        ""
    ),
    "html_size": len(
        html_content
    ),
    "text_size": len(
        text_content
    ),
    "word_count": len(
        words
    ),
    "links": links,
    "link_count": len(
        links
    ),
    "images": len(
        images
    ),
    "scripts": len(
```

```
        scripts
    ),
    "stylesheets": len(
        styles
    ),
    "headings": headings,
}
```

```
#
=====
=====
# /htmlinfo
#
=====
=====
```

```
async def htmlinfo_command(
    update,
    context
):

    if not await require_admin(update):
        return

    if not context.args:

        await update.message.reply_text(
            "Usage:\n\n"
            "/htmlinfo SOURCE\n\n"
            "Example:\n"
            "/htmlinfo rajras"
        )
```

```

        return

    source_name = (
        context.args[0]
        .strip()
    )

    try:

        info = analyze_html_source(
            source_name
        )

        if not info:

            await
            update.message.reply_text(
                "Source नहीं मिला\\n\\n"
                "/sources से available
sources देखें।"
            )

        return

    text = (
        "HTML Source Information\\n\\n"
        f"Source: {info['name']}\\n"
        f"URL: {info['url']}\\n"
        f>Title: {info['title']} or
        'N/A'\\n"
        f>Status:
        {info['status_code']}\\n"
        f"Content-Type:
        {info['content_type']}\\n\\n"

```

```

        f"HTML Size:
{info['html_size']} bytes\n"
        f"Text Size:
{info['text_size']} chars\n"
        f"Words:
{info['word_count']}\n"
        f"Links:
{info['link_count']}\n"
        f"Images: {info['images']}\n"
        f"Scripts: {info['scripts']}\n"
        f"Stylesheets:
{info['stylesheets']}\n"
        f"Headings:
{len(info['headings'])}"
    )

    await update.message.reply_text(
        text
    )

except Exception as e:

    logger.exception(
        "HTML info failed"
    )

    await update.message.reply_text(
        "HTML information निकालते समय
error आया:\n\n"
        f"{str(e)[:1000]}"
    )

#

```



```
=====
=====
# HTML REPORT GENERATOR
#
=====
=====

def build_html_report(
    source_name
):

    info = analyze_html_source(
        source_name
    )

    if not info:
        raise ValueError(
            "Source नहीं मिला।"
        )

    report = []

    report.append(
        "<!DOCTYPE html>"
    )

    report.append(
        "<html>"
    )

    report.append(
        "<head>"
    )
```

```
report.append(
    '<meta charset="UTF-8">'
)

report.append(
    f"<title>{html.escape(source_name)}
Report</title>"
)

report.append(
    """
<style>
body {
    font-family: Arial, sans-serif;
    margin: 40px;
    line-height: 1.5;
}

h1, h2 {
    margin-top: 30px;
}

table {
    border-collapse: collapse;
    width: 100%;
}

th, td {
    border: 1px solid #999;
    padding: 8px;
    text-align: left;
}

th {
```

```
        background: #eee;
    }

    .box {
        border: 1px solid #aaa;
        padding: 15px;
        margin-bottom: 20px;
    }

    a {
        word-break: break-all;
    }
</style>
"""
)

report.append(
    "</head>"
)

report.append(
    "<body>"
)

report.append(
    f"<h1>HTML Source Report</h1>"
)

report.append(
    '<div class="box">'
)

report.append(
    f"<p><b>Source:</b> "
```

```
        f"{html.escape(info['name'])}</p>"
    )

    report.append(
        f"<p><b>URL:</b> "
        f"{html.escape(info['url'])}</p>"
    )

    report.append(
        f"<p><b>Title:</b> "
        f"{html.escape(info['title'] or 'N/A')}</p>"
    )

    report.append(
        f"<p><b>Status Code:</b> "
        f"{info['status_code']}</p>"
    )

    report.append(
        f"<p><b>Content Type:</b> "
        f"
{html.escape(info['content_type'])}</p>"
    )

    report.append(
        f"<p><b>HTML Size:</b> "
        f"{info['html_size']} bytes</p>"
    )

    report.append(
        f"<p><b>Text Size:</b> "
        f"{info['text_size']}
characters</p>"
    )
```

```
)

report.append(
    f"<p><b>Word Count:</b> "
    f"{info['word_count']}</p>"
)

report.append(
    f"<p><b>Links:</b> "
    f"{info['link_count']}</p>"
)

report.append(
    f"<p><b>Images:</b> "
    f"{info['images']}</p>"
)

report.append(
    f"<p><b>Scripts:</b> "
    f"{info['scripts']}</p>"
)

report.append(
    f"<p><b>Stylesheets:</b> "
    f"{info['stylesheets']}</p>"
)

report.append(
    "</div>"
)

# -----
-----
# DESCRIPTION
```

```
# -----  
-----  
  
if info["description"]:  
  
    report.append(  
        "<h2>Description</h2>"  
    )  
  
    report.append(  
        "<p>"  
        + html.escape(  
            info["description"]  
        )  
        + "</p>"  
    )  
  
# -----  
-----  
  
# HEADINGS  
# -----  
-----  
  
report.append(  
    "<h2>Headings</h2>"  
)  
  
if info["headings"]:  
  
    report.append(  
        "<table>"  
        "<tr>"  
        "<th>Level</th>"  
        "<th>Heading</th>"
```

```

        "</tr>"
    )

    for heading in info["headings"]:

        report.append(
            "<tr>"
            f"<td>H{heading['level']}"
            "</td>"
            f"<td>"
            f"{html.escape(heading['text'])}</td>"
            "</tr>"
        )

        report.append(
            "</table>"
        )

    else:

        report.append(
            "<p>No headings found.</p>"
        )

    # -----
    # LINKS
    # -----

    report.append(
        "<h2>Links</h2>"
    )

```

```
if info["links"]:

    report.append(
        "<table>"
        "<tr>"
        "<th>#</th>"
        "<th>Text</th>"
        "<th>URL</th>"
        "</tr>"
    )

    for index, link in enumerate(
        info["links"],
        start=1
    ):

        link_url = link["url"]

        report.append(
            "<tr>"
            f"<td>{index}</td>"
            f"<td>"
            {html.escape(link['text'])}</td>"
            f"<td>"
            {html.escape(link_url)}</td>"
            "</tr>"
        )

        report.append(
            "</table>"
        )

else:
```



```

        report.append(
            "<p>No links found.</p>"
        )

        report.append(
            "<hr>"
        )

        report.append(
            "<p>"
            "Generated by Telegram Quiz Bot"
            "</p>"
        )

        report.append(
            "</body>"
        )

        report.append(
            "</html>"
        )

        return "\n".join(
            report
        )

```

```

#
=====
=====
# /htmlreport
#
=====
=====

```

```

async def htmlreport_command(
    update,
    context
):

    if not await require_admin(update):
        return

    if not context.args:

        await update.message.reply_text(
            "Usage:\n\n"
            "/htmlreport SOURCE\n\n"
            "Example:\n"
            "/htmlreport rajras"
        )

        return

    source_name = (
        context.args[0]
        .strip()
    )

    try:

        await update.message.reply_text(
            "HTML report generate हो रही है..."
        )

        report = build_html_report(
            source_name

```

```

    )

    safe_name = re.sub(
        r"^[a-zA-Z0-9_-]+",
        "_",
        source_name
    )

    report_path = (
        DATA_DIR
        / f"{safe_name}_report.html"
    )

    report_path.write_text(
        report,
        encoding="utf-8"
    )

    with report_path.open(
        "rb"
    ) as file:

        await
update.message.reply_document(
    document=file,
    filename=(
        f"
{safe_name}_report.html"
    ),
    caption=(
        f"HTML Report: "
        f"{source_name}"
    )
)

```

```

except Exception as e:

    logger.exception(
        "HTML report failed"
    )

    await update.message.reply_text(
        "HTML report generate नहीं हो
सकी:\n\n"
        f"{str(e)[:1000]}"
    )
    #
=====
=====
# ERROR HANDLER
#
=====
=====

async def error_handler(update, context):
    logger.exception(
        "Telegram update error",
        exc_info=context.error
    )

    try:
        if update and
update.effective_message:
            await
update.effective_message.reply_text(
                "Bot में unexpected error
आया\n\n"
                "कृपया थोड़ी देर बाद दोबारा try

```

```

करें।"

        )
    except Exception:
        logger.exception(
            "Could not send error message"
        )
    #

=====
=====
# BOT STATUS
#
=====
=====

async def status_command(update, context):
    user = update.effective_user

    if user:

        ensure_user(user)

    conn = db()

    try:
        question_count = conn.execute(
            """
            SELECT COUNT(*)
            FROM questions
            WHERE active = 1
            """
        ).fetchone()[0]

```

```

        quiz_count = conn.execute(
            """
            SELECT COUNT(*)
            FROM quizzes
            """
        ).fetchone()[0]

        user_count = conn.execute(
            """
            SELECT COUNT(*)
            FROM users
            """
        ).fetchone()[0]

    finally:
        conn.close()

    await
update.effective_message.reply_text(
    "Bot Status\n\n"
    "Status: Online\n"
    f"Questions: {question_count}\n"
    f"Saved Quizzes: {quiz_count}\n"
    f"Users: {user_count}"
)
#
=====
=====
# APPLICATION
#
=====
=====

```

```
async def post_init(application):

    logger.info(
        "Starting background workers..."
    )

    application.create_task(
        clone_queue_worker(application)
    )

    logger.info(
        "Clone queue worker started."
    )

def build_application():

    if not BOT_TOKEN:

        raise RuntimeError(
            "BOT_TOKEN environment variable
missing."
        )

    application = (
        Application.builder()
        .token(BOT_TOKEN)
        .post_init(post_init)
        .build()
    )

    #
    =====
    =====
```

```
# USER COMMANDS
```

```
#
```

```
=====
```

```
application.add_handler(  
    CommandHandler(  
        "start",  
        start_command  
    )  
)
```

```
application.add_handler(  
    CommandHandler(  
        "status",  
        status_command  
    )  
)
```

```
application.add_handler(  
    CommandHandler(  
        "quiz",  
        quiz_command  
    )  
)
```

```
application.add_handler(  
    CommandHandler(  
        "stats",  
        stats_command  
    )  
)
```

```
application.add_handler(  
    CommandHandler(  
        "quit",  
        quit_command  
    )  
)
```



```
        CommandHandler(  
            "help",  
            help_command  
        )  
    )  
  
    application.add_handler(  
        CommandHandler(  
            "newquiz",  
            newquiz_command  
        )  
    )  
  
    application.add_handler(  
        CommandHandler(  
            "quizid",  
            quizid_command  
        )  
    )  
  
    #  
    =====  
    =====  
    # NEW FEATURES - BOOKMARKS / MISTAKES /  
    LEADERBOARD / ETC  
    #  
    =====  
    =====  
  
    application.add_handler(  
        CommandHandler(  
            "mybookmarks",  
            mybookmarks_command  
        )  
    )
```

```
)

application.add_handler(
    CommandHandler(
        "mymistakes",
        mymistakes_command
    )
)

application.add_handler(
    CommandHandler(
        "weaktopics",
        weaktopics_command
    )
)

application.add_handler(
    CommandHandler(
        "revision",
        revision_command
    )
)

application.add_handler(
    CommandHandler(
        "leaderboard",
        leaderboard_command
    )
)

application.add_handler(
    CommandHandler(
        "dashboard",
        dashboard_command
    )
)
```

```
)  
)  
  
application.add_handler(  
    CommandHandler(  
        "myreferrals",  
        myreferrals_command  
    )  
)  
)
```

```
application.add_handler(  
    CommandHandler(  
        "exam",  
        exam_command  
    )  
)  
)
```

```
#
```

```
=====
```

```
=====
```

```
# ADMIN - QUESTION MANAGEMENT
```

```
#
```

```
=====
```

```
=====
```

```
application.add_handler(  
    CommandHandler(  
        "add",  
        add_command  
    )  
)  
)
```

```
application.add_handler(  
    CommandHandler(  
        "add",  
        add_command  
    )  
)
```

```
        "edit",
        edit_command
    )
)

application.add_handler(
    CommandHandler(
        "delete",
        delete_command
    )
)

application.add_handler(
    CommandHandler(
        "generate",
        generate_command
    )
)

application.add_handler(
    CommandHandler(
        "autoquiz",
        autoquiz_command
    )
)

application.add_handler(
    CommandHandler(
        "websearch",
        websearch_command
    )
)

application.add_handler(
```

```
        CommandHandler(  
            "newsquiz",  
            newsquiz_command  
        )  
    )  
  
    application.add_handler(  
        CommandHandler(  
            "wikiquiz",  
            wikiquiz_command  
        )  
    )  
  
    application.add_handler(  
        CommandHandler(  
            "speak",  
            speak_command  
        )  
    )  
  
    application.add_handler(  
        CommandHandler(  
            "addexamlink",  
            addexamlink_command  
        )  
    )  
  
    application.add_handler(  
        CommandHandler(  
            "addsubjectlink",  
            addsubjectlink_command  
        )  
    )
```

```
application.add_handler(  
    CommandHandler(  
        "examlinks",  
        examlinks_command  
    )  
)  
  
application.add_handler(  
    CommandHandler(  
        "subjectlinks",  
        subjectlinks_command  
    )  
)  
  
application.add_handler(  
    CommandHandler(  
        "examlist",  
        examlist_command  
    )  
)  
  
application.add_handler(  
    CommandHandler(  
        "subjectlist",  
        subjectlist_command  
    )  
)  
  
application.add_handler(  
    CommandHandler(  
        "examquiz",  
        examquiz_command  
    )  
)
```

```
        application.add_handler(  
            CommandHandler(  
                "subjectquiz",  
                subjectquiz_command  
            )  
        )  
  
        #  
        =====  
        =====  
        # ADMIN - SOURCES / SCRAPING  
        #  
        =====  
        =====  
  
        application.add_handler(  
            CommandHandler(  
                "scrape",  
                scrape_command  
            )  
        )  
  
        application.add_handler(  
            CommandHandler(  
                "sources",  
                sources_command  
            )  
        )  
  
        application.add_handler(  
            CommandHandler(  
                "addsource",  
                addsource_command
```

```
)
)

application.add_handler(
    CommandHandler(
        "source",
        source_command
    )
)

application.add_handler(
    CommandHandler(
        "htmlinfo",
        htmlinfo_command
    )
)

application.add_handler(
    CommandHandler(
        "htmlreport",
        htmlreport_command
    )
)
```

```
#
```

```
=====
```

```
=====
```

```
# PDF / TXT
```

```
#
```

```
=====
```

```
=====
```

```
application.add_handler(
    CommandHandler(
```



```

        "pdfimport",
        pdfimport_command
    )
)

application.add_handler(
    CommandHandler(
        "txtimport",
        txtimport_command
    )
)

application.add_handler(
    CommandHandler(
        "pdfinfo",
        pdfinfo_command
    )
)

application.add_handler(
    MessageHandler(
        filters.Document.ALL,
        document_import_handler
    )
)

application.add_handler(
    MessageHandler(
        filters.Document.ALL
        & (
filters.UpdateType.CHANNEL_POST
|
filters.UpdateType.EDITED_CHANNEL_POST

```

```

        ),
        channel_pdf_handler
    )
)

#
=====
=====
# POLL
#
=====
=====

application.add_handler(
    CommandHandler(
        "poll2q",
        poll2q_command
    )
)

application.add_handler(
    CommandHandler(
        "scrapepoll",
        scrapepoll_command
    )
)

application.add_handler(
    MessageHandler(
        filters.POLL,
        process_poll
    )
)

```

```
#
=====
=====
# CLONE
#
=====
=====

    application.add_handler(
        CommandHandler(
            "clone",
            clone_command
        )
    )

    application.add_handler(
        CommandHandler(
            "clonestatus",
            clone_status_command
        )
    )

    application.add_handler(
        CommandHandler(
            "queue",
            queue_command
        )
    )

    application.add_handler(
        CommandHandler(
            "clearqueue",
            clearqueue_command
        )
    )
```

```
)

#
=====
=====
# NEGATIVE MARKING
#
=====
=====

    application.add_handler(
        CommandHandler(
            "negmark",
            negmark_command
        )
    )

    application.add_handler(
        CommandHandler(
            "resetpenalty",
            resetpenalty_command
        )
    )

    application.add_handler(
        CommandHandler(
            "quiztime",
            quiztime_command
        )
    )

    application.add_handler(
        CommandHandler(
            "resetquiztime",
```

```
        resetquiztime_command
    )
)

application.add_handler(
    CommandHandler(
        "resethistory",
        resethistory_command
    )
)

#
=====
=====
# STOP / CANCEL
#
=====
=====

application.add_handler(
    CommandHandler(
        "stop",
        stop_command
    )
)

application.add_handler(
    CommandHandler(
        "cancel",
        cancel_command
    )
)

#
```

```
=====
=====
# QUIZ CALLBACKS
#
=====
=====

    application.add_handler(
        PollAnswerHandler(
            poll_answer_handler
        )
    )

    application.add_handler(
        CallbackQueryHandler(
            stop_quiz_callback,
            pattern=r"^stopquiz:"
        )
    )

    application.add_handler(
        CallbackQueryHandler(
            bookmark_callback,
            pattern=r"^bm:"
        )
    )

    application.add_handler(
        CallbackQueryHandler(
            feedback_callback,
            pattern=r"^fb:"
        )
    )
#
```

```

=====
=====
# TEXT INPUT
#
# Group 0 -> New Quiz
# Group 1 -> Add/Edit question
#
=====
=====

    application.add_handler(
        MessageHandler(
            filters.TEXT &
~filters.COMMAND,
            newquiz_text_handler
        ),
        group=0
    )

    application.add_handler(
        MessageHandler(
            filters.TEXT &
~filters.COMMAND,
            pending_text_handler
        ),
        group=1
    )

#
=====
=====
# ERROR HANDLER
#
=====

```

```

=====

    application.add_error_handler(
        error_handler
    )

    return application
#
=====
=====
# HEALTH SERVER
#
=====
=====

#
=====
=====
# MAIN
#
=====
=====

def main():

    logging.basicConfig(
        level=logging.INFO,
        format=(
            "%(asctime)s | "
            "%(levelname)s | "
            "%(name)s | "
            "%(message)s"
        )
    )

```



```
)

logger.info(
    "Initializing database..."
)

init_db()

logger.info(
    "Database initialized."
)

seed_default_topic_sources()

health_thread = threading.Thread(
    target=run_health_server,
    daemon=True,
    name="health-server"
)

health_thread.start()

logger.info(
    "Health server started on port %s",
    PORT
)

application = build_application()

logger.info(
    "Telegram Quiz Bot starting..."
)

application.run_polling(
```

```
        allowed_updates=Update.ALL_TYPES,  
        drop_pending_updates=True  
    )
```

```
if __name__ == "__main__":  
    main()
```